스마트 제조

류영표 강사
ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com

학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 국제인공지능올림피아드 한국 대표 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

스마트공장 모니터링 프로그램의 필요성

공장의 생산·품질·설비 정보를 자동으로 확인하고 관리하기 위해 **모니터링 프로그램**이 필요합니다.

제조 현장의 데이터 예시 (가상 데이터)

생산 시간	설비번호	생산량	불량 수량	온도	습도
09:00	M001	100	3	35℃	45%
09:00	M002	120	5	37℃	48%
09:00	M003	90	8	43℃	50%
09:00	M004	110	2	36℃	46%
10:00	M001	105	4	36℃	44%
10:00	M002	115	3	38℃	47%
10:00	M003	95	9	45℃	51%
10:00	M004	120	3	37℃	45%

기존 방식의 문제점 (수작업 관리)

- 엑셀 파일을 열어 데이터를 확인해야 함
- 생산량과 불량률을 직접 계산해야 함
- 어떤 설비의 온도가 높은지 바로 알기 어려움
- 매일 반복되는 업무로 시간과 노력이 많이 소요됨

모니터링 프로그램을 사용하면?

- 생산량, 불량률을 자동으로 계산하여 한눈에 확인
- 설비별 생산 현황을 그래프로 비교
- 온도가 기준을 초과하면 즉시 경고 알림
- 원하는 기간과 설비의 데이터를 쉽게 조회

기존 방식과 모니터링 프로그램 비교

구분	기존 방식(엑셀)	모니터링 프로그램
데이터 확인	파일을 열어 확인	웹 화면에서 바로 확인
계산 업무	수식으로 직접 계산	생산량·불량률 자동 계산
현황 비교	표를 직접 비교	그래프로 한눈에 비교
이상 확인	데이터를 찾아 확인	기준 초과 시 경고 알림
데이터 관리	엑셀 파일로 관리	검색·입력·저장 등 통합 관리

기존 데이터 관리 방식의 문제점

공장 관리자가 매일 수행하는 업무를 예로 들어봅시다.

매일 반복되는 수작업 업무

1	데이터 수집	생산설비의 데이터를 엑셀 파일로 수집
2	생산량 합산	각 설비의 생산량을 합산
3	불량 수량 합산	불량 수량을 합산
4	불량률 계산	불량률을 계산 (불량 수량 ÷ 생산량 × 100)
5	온도 이상 확인	온도가 높은 설비가 있는지 확인
6	생산 현황 보고	생산 현황 정리하여 보고

기존 방식과 모니터링 프로그램 비교

구분	기존 방식 (수작업)	모니터링 프로그램 (자동화)
데이터 확인	CSV·엑셀 파일 열기	**웹 화면에서 확인**
생산량 계산	수식 직접 계산	**자동 계산**
불량률 계산	수식 직접 계산	**자동 계산**
설비 비교	표를 직접 비교	**그래프로 비교**
온도 이상 확인	데이터를 찾아 확인	**경고 메시지 표시**
데이터 검색	파일에서 직접 검색	**조건 선택(설비·기간 등)**
데이터 저장	파일 관리	**데이터베이스 활용**

핵심 포인트

모니터링 프로그램은 **데이터 분석을 대신 수행**하고 결과를 사용자에게 **즉시 제공**해 업무를 효율화합니다.

이번 프로젝트 범위

먼저 **CSV 기반의 모니터링**을 구축하고, 이후 가상 센서 데이터와 데이터베이스 기능을 추가하여 확장합니다.

1. 스마트공장 모니터링 시스템 이해

프로젝트 시나리오 : 실습 상황

여러분은 제조기업에 새로 입사한 개발자입니다. 공장 관리자의 요구사항을 반영하여 **모니터링 프로그램**을 설계해 보세요.

공장의 생산설비(예시)

설비번호	설비명	주요 역할
M001	프레스 1호기	제품 가공
M002	프레스 2호기	제품 가공
M003	조립 1호기	부품 조립
M004	조립 2호기	부품 조립

공장 관리자의 요청사항

- 생산설비별 생산량과 불량률을 쉽게 확인할 수 있게 해주세요.
- 공정 온도가 기준을 초과하면 경고가 나타나야 합니다.
- 날짜와 설비를 선택하면 해당 생산 기록을 확인할 수 있으면 좋겠습니다.
- 나중에는 새로운 데이터를 직접 입력하고 저장할 수 있도록 해주세요.

Q1 공장 관리자가 가장 먼저 확인해야 하는 정보는 무엇일까요?

- 전체 생산량
- 불량 제품 개수
- 불량률
- 설비별 생산량
- 온도가 높은 설비

Q2 이러한 정보를 확인하려면 어떤 데이터가 필요할까요?

- 생산량
- 불량 수량
- 설비번호
- 생산 시간
- 온도

Q3 관리자가 쉽게 사용하려면 어떤 기능이 필요할까요?

- 설비 선택 메뉴
- 날짜 선택 기능
- 그래프
- 경고 메시지
- 데이터 검색 기능

학습 확인

관리자의 요구사항을 데이터와 프로그램 기능으로 연결해봅시다.

요구사항	필요한 데이터	계산	프로그램 기능	결과
불량률이 높은 설비 찾기	설비번호, 생산량, 불량 수량	설비별 불량률	설비별 불량률 그래프	불량률이 높은 설비 확인

프로그램의 기본 구조

대부분의 프로그램은 다음과 같은 과정을 거쳐 작동합니다.

구분	의미	스마트공장 예시
입력(Input)	프로그램에 제공하는 정보	생산량, 불량 수량
처리(Process)	입력된 정보를 계산하거나 판단	불량률 계산
출력(Output)	처리 결과를 사용자에게 제공	불량률 5%

예제 1 불량률 계산 프로그램

제품 100개를 생산했고, 그중 불량 제품이 5개일 때 불량률을 계산합니다.

$$\text{불량률} = \frac{\text{불량 수량}}{\text{전체 생산량}} \times 100$$

$$= \frac{5}{100} \times 100 = 5\%$$

입력	처리	출력
• 생산량: 100 • 불량 수량: 5	• 불량 수량 ÷ 생산량 × 100	• 불량률: 5%

학생 실습 문제 불량률을 계산해 보세요.

문제	생산량	불량 수량	불량률(빈칸)
1	100	3	?
2	200	10	?
3	150	15	?
4	500	20	?

정답

문제	불량률
1	3%
2	5%
3	10%
4	4%

예제 2 설비 온도 경고 프로그램

공정 온도가 40℃를 초과하면 경고를 표시합니다.

$$S(T) = \begin{cases} \text{정상}, & T \le 40 \\ \text{경고}, & T > 40 \end{cases}$$

입력	처리	출력
• 설비 온도	• 40℃ 초과 여부 판단	• 40℃ 이하: 정상 • 40℃ 초과: 경고

학생 실습 문제 온도의 결과를 적어보세요.

온도	예상 결과
35℃	정상
39℃	정상
40℃	정상
41℃	경고
45℃	경고

생각해보기

온도가 정확히 40℃이면 왜 정상인가요?

→ 조건이 '40℃ 초과'이기 때문입니다.

예제 3 사용자 선택에 따른 데이터 조회

사용자가 설비를 선택하면 해당 설비의 데이터만 조회합니다. 이러한 기능을 **필터링(Filtering)**이라고 합니다.

입력	처리	출력
M001 선택	M001 데이터 검색	M001 생산 현황
M002 선택	M002 데이터 검색	M002 생산 현황
M003 선택	M003 데이터 검색	M003 생산 현황
M004 선택	M004 데이터 검색	M004 생산 현황

학생 활동 2. 입력·처리·출력 구분하기

다음 프로그램 기능에 대해 입력, 처리, 출력을 구분해 보세요.

활동 방법

1. 각 기능에 필요한 **입력**을 찾습니다.
2. 어떤 **계산·판단**을 하는지 **처리** 내용을 작성합니다.
3. 최종적으로 나타나는 **출력 결과**를 작성합니다.

프로그램 기능	입력 (Input)	처리 (Process)	출력 (Output)
불량률 계산	• 생산량, 불량 수량	• 불량률 계산 $\text{불량률} = \frac{\text{불량 수량}}{\text{생산량}} \times 100$	• 불량률(%)
생산 목표 달성률	• 실제 생산량, 목표 생산량	• 달성률 계산 $\text{달성률} = \frac{\text{실제 생산량}}{\text{목표 생산량}} \times 100$	• 달성률(%)
온도 경고	• 온도	• 기준 초과 여부 판단 $T > 40℃\,?$	• 정상 또는 정상 경고
설비별 데이터 조회	• 설비번호	• 해당 설비 데이터 선택(필터링)	• 선택한 설비의 데이터 (생산량, 불량률, 온도 등)
날짜별 생산량	• 생산 날짜(기간)	• 해당 날짜의 생산량 합산 $\text{총 생산량} = \sum_{t \in \text{기간}} \text{생산량}_t$	• 날짜별 생산량 (또는 기간별 총 생산량)

학습 포인트 | 모든 프로그램은 **입력** → **처리** → **출력** 의 흐름으로 동작합니다.

3. 프로그램 요구사항 정의

모호한 요구사항과 구체적인 요구사항

같은 기능이라도 **구체적으로 작성**해야 AI가 정확하게 이해하고 원하는 프로그램을 만들 수 있습니다.

모호한 요구사항	구체적인 요구사항
생산량을 보여줘	**선택한 기간**의 전체 생산량을 표시해줘.
불량을 분석해줘	**설비별 불량률**을 계산하고 **막대그래프**로 보여줘.
온도가 높으면 알려줘	최근 온도가 **40℃를 초과**하면 **빨간색 경고**를 표시해줘.
데이터를 검색하고 싶어	**설비번호**와 **날짜 범위**로 검색할 수 있게 해줘.
데이터를 저장해줘	입력한 생산 데이터를 **데이터베이스에 저장**해줘.

핵심 메시지

바이브코딩에서 중요한 것은 어려운 프로그래밍 용어가 아닙니다.
원하는 기능을 AI가 이해할 수 있도록 구체적으로 설명하는 것 입니다.

프로그램 사용자와 필요한 정보

이번 프로그램을 사용할 사람은 공장 관리자라고 가정합니다. 사용자별로 필요한 기능이 다를 수 있습니다.

사용자	필요한 정보
생산 관리자 생산 운영 총괄	● 생산량, 생산 목표, 생산 실적
품질 관리자 제품 품질 관리	● 불량 수량, 불량률, 품질 경고
설비 관리자 설비 상태 관리	● 온도, 습도, 설비 상태
데이터 관리자 데이터 관리·운영	● 생산 기록 조회, 입력, 수정

하나의 프로그램으로 통합

이번 수업에서는 사용자별로 필요한 기능을 **하나의 프로그램 안에 통합**하여 구현합니다.

참고 | 실제 공장에서는 사용자별 **접근 권한**이나 **보안 정책**을 별도로 고려해야 합니다.

프로그램 기능 분류하기

프로그램을 4가지 메뉴로 구성합니다.

공장 관리에 필요한 기능을 목적에 따라 네 가지 메뉴로 나누어 구현합니다.

메뉴 1. 생산 현황

목적: 공장의 생산 실적을 확인합니다.

필요한 기능

1. 전체 생산량
2. 정상 제품 수
3. 불량 제품 수
4. 전체 불량률
5. 설비별 생산량 그래프
6. 날짜별 생산량 그래프
7. 생산 목표 달성률

메뉴 2. 품질 관리

목적: 생산 과정에서 발생한 불량을 확인합니다.

필요한 기능

1. 설비별 불량률
2. 날짜별 불량률 변화
3. 불량률이 높은 설비 찾기
4. 허용 불량률 설정
5. 기준 초과 경고
6. 설비별 품질 순위

메뉴 3. 설비 모니터링

목적: 설비의 상태를 확인합니다.

필요한 기능

1. 설비번호 선택
2. 최근 공정 온도
3. 최근 습도
4. 온도 변화 그래프
5. 온도 경고
6. 가상 센서 데이터 생성

메뉴 4. 데이터 관리

목적: 생산 데이터를 조회하고 관리합니다.

필요한 기능

1. 전체 데이터 조회
2. 설비번호 검색
3. 날짜별 검색
4. 생산 데이터 입력
5. 잘못된 입력값 검증
6. 데이터베이스 저장

핵심 포인트 | 메뉴별 **목적**과 **필요한 기능**을 명확히 정의하면 **AI를 활용한 개발**이 훨씬 수월해집니다.

필수 기능과 추가 기능 구분하기

모든 기능을 한꺼번에 개발하면 프로그램이 복잡해집니다. 따라서 처음에는 반드시 필요한 기능부터 구현합니다.

기능	우선순위	이유
생산량 표시	✔ 필수	가장 기본적인 생산 정보
불량률 계산	✔ 필수	품질 상태 확인
설비별 그래프	✔ 필수	설비 간 비교
온도 경고	✔ 필수	이상 상태 확인
설비 선택	✔ 필수	특정 설비 확인
생산 목표 달성률	+ 추가	생산 실적 평가
가상 센서 생성	+ 추가	모니터링 시뮬레이션
데이터베이스 연결	» 후속 개발	영구적인 데이터 저장
웹 배포	» 후속 개발	다른 사용자와 프로그램 공유

우선순위 구분

- ✔ 필수 기능 | 프로그램의 핵심 기능으로 반드시 먼저 구현해야 하는 기능
- + 추가 기능 | 핵심 기능이 완성된 후, 기능을 확장하는 단계에서 구현
- » 후속 개발 | 서비스 안정화 이후에 추가로 개발할 기능

개발 진행 흐름

1단계 필수 기능 구현 → 2단계 추가 기능 확장 → 3단계 후속 기능 개발

핵심 정리 | 우선순위를 정하면 프로그램의 **핵심 기능을 먼저 완성**하고, 이후 기능을 **단계적으로 확장**할 수 있습니다.

학생 실습 3

요구사항 정의서 작성하기

스마트공장 프로그램에 대한 요구사항 정의서를 직접 작성해 봅시다.

스마트공장 프로그램 요구사항 정의서 (학생 작성)

항목	학생 작성 내용
프로그램 이름	
프로그램 사용자	
프로그램 개발 목적	
사용할 데이터	
첫 번째 주요 기능	
두 번째 주요 기능	
세 번째 주요 기능	
필요한 그래프	
경고 발생 조건	
추가하고 싶은 기능	

작성 예시

항목	예시
프로그램 이름	Smart Factory Monitoring
프로그램 사용자	생산 관리자
프로그램 개발 목적	공장의 생산 및 설비 상태 확인
사용할 데이터	생산량, 불량 수량, 온도, 습도
첫 번째 주요 기능	생산량 자동 계산
두 번째 주요 기능	설비별 불량률 비교
세 번째 주요 기능	온도 이상 경고
필요한 그래프	생산량 막대그래프
경고 발생 조건	온도 40°C 초과
추가하고 싶은 기능	생산 목표 달성률

활동 안내

1. 위의 요구사항 정의서 표를 직접 작성해 보세요.
2. 작성한 결과를 이후 **"바이브코딩(AI 코딩)"**의 프롬프트로 사용합니다.

좋은 프롬프트의 기본 구성

AI에게 원하는 결과를 얻기 위해서는 다음 **다섯 가지 요소**를 포함하여 명확하게 설명합니다.

요소	의미	예시
역할	AI가 어떤 역할을 수행할지	Python 개발자
목적	무엇을 만들 것인지	스마트공장 모니터링
기능	어떤 기능이 필요한지	생산량, 불량률, 온도
조건	어떤 환경과 제약이 있는지	Python, Streamlit
출력	어떤 결과를 원하는지	메뉴 구성표

프롬프트 작성 구조 (예시)

역할	너는 초보자를 가르치는 **Python 개발자**야.
목적	**스마트공장 생산 모니터링 프로그램**을 만들려고 해.
기능	**생산량, 불량률, 설비 온도**를 확인하고 싶어.
조건	**Python**과 **Streamlit**을 사용할 예정이야.
출력	필요한 **메뉴와 기능**을 표로 정리해줘.

핵심 정리

이 다섯 가지 요소(역할, 목적, 기능, 조건, 출력)를 포함하여 작성하면
AI가 **요구사항을 정확히 이해**하고, **원하는 결과를 제공**할 가능성이 높아집니다.

실습 4 | AI에게 프로그램 설계 요청하기

바이브코딩을 위해 프로그램의 요구사항을 AI에게 설명하고, **메뉴·기능·데이터·우선순위를** 표로 정리해 봅니다.

바이브코딩 프롬프트 1 (요청 내용)

스마트공장 생산 모니터링 프로그램을 만들기 위해 메뉴, 기능, 필요한 데이터, 필수/추가 기능을 표로 정리해줘.

사용할 데이터 (가상 CSV)

항목	설명
생산 시간	제품을 생산한 시각
설비번호	생산에 사용된 설비 구분
생산량	생산된 제품 수
불량 수량	불량으로 판정된 제품 수
공정 온도	생산 공정의 온도(℃)
습도	생산 공정의 습도(%)

AI가 제안한 프로그램 설계 결과 (예시)

메뉴	기능	설명	필요한 데이터	우선순위
생산 현황	전체 생산량 확인	선택한 기간의 총 생산량을 계산하여 표시	생산량	필수
	정상/불량 제품 수 확인	정상 제품 수와 불량 제품 수를 표시	생산량, 불량 수량	필수
	전체 불량률 계산	전체 기준의 불량률(%)을 계산하여 표시	생산량, 불량 수량	필수
품질 관리	설비별 생산량 비교	설비별 생산량을 그래프로 비교(막대그래프)	설비번호, 생산량	필수
	설비별 불량률 비교	설비별 불량률을 계산하고 그래프로 비교	설비번호, 생산량, 불량 수량	필수
설비 모니터링	공정 온도 모니터링	설비별 온도 변화를 확인(그래프/실시간 표시)	설비번호, 공정 온도	필수
	온도 이상 경고	온도가 기준값(예: 40℃)을 초과하면 경고 표시	공정 온도	필수
데이터 관리	생산 데이터 조회	설비번호와 기간을 조건으로 데이터 조회	생산 시간, 설비번호 등	필수
	생산 데이터 입력	새로운 생산 데이터를 입력하고 저장(임시 저장)	설비번호, 생산량, 불량 수량, 온도, 습도	추가

결과 정리

AI가 제안한 결과를 바탕으로 우리 프로그램에 필요한 **메뉴와 기능, 데이터, 우선순위**를 확인할 수 있습니다.

학생 확인 사항 (AI 결과 점검)

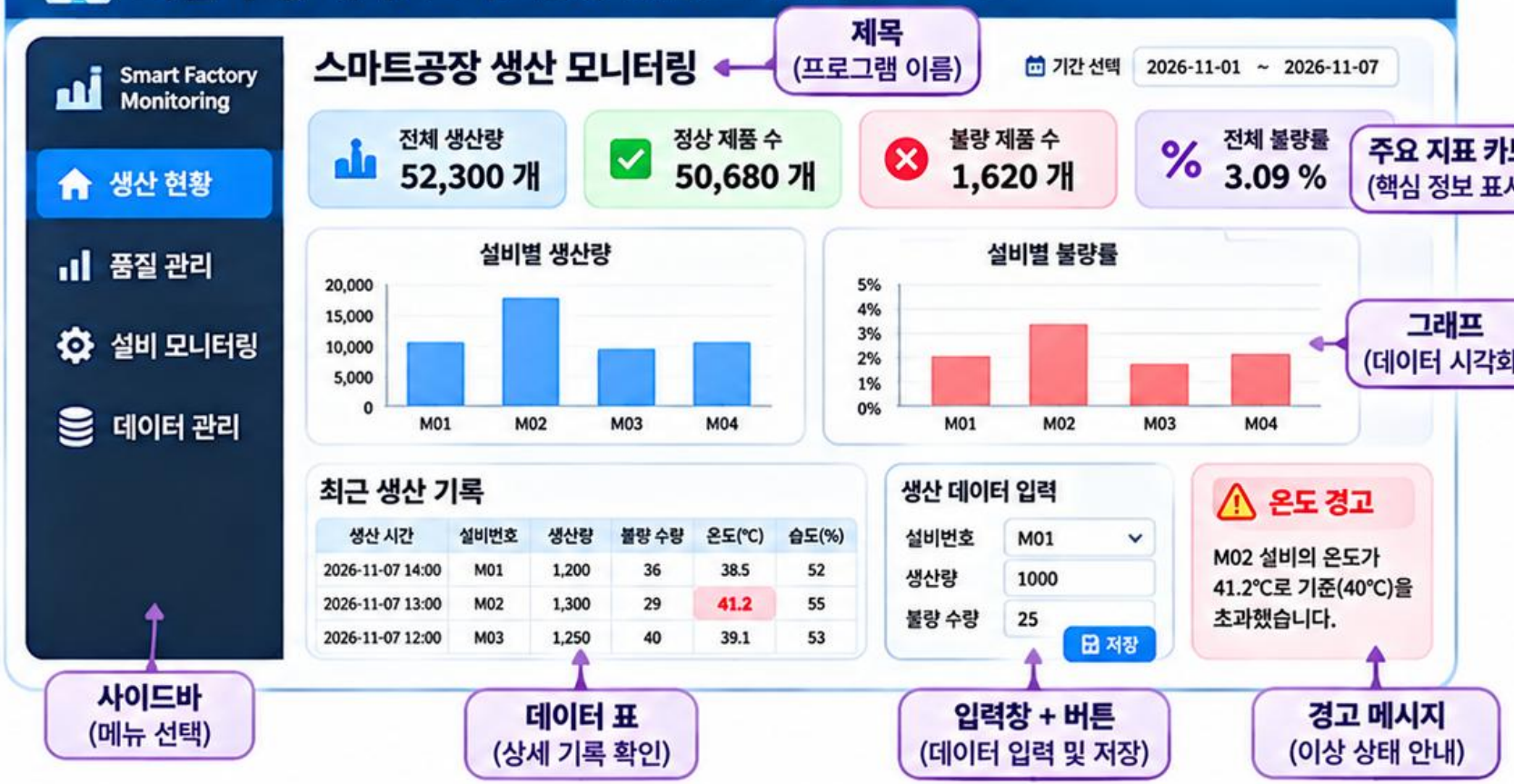
1. 필요한 메뉴가 모두 포함되어 있는가? ☑
2. 생산량과 불량률 기능이 구분되어 있는가? ☑
3. 온도 경고 기능이 포함되어 있는가? ☑
4. 실제 CSV에 없는 데이터를 요구하지 않는가? ☑
5. 불필요하게 복잡한 기능이 포함되어 있지 않은가? ☑

웹 프로그램의 화면 구성 이해

웹 프로그램은 여러 **화면 요소**를 조합하여 데이터를 확인하고, 분석하고, 입력할 수 있는 서비스로 만들 수 있습니다.

화면 요소	역할
제목	프로그램 이름 표시
사이드바	메뉴 선택 (화면 이동)
주요 지표 카드	생산량, 불량률 등 핵심 정보 표시
그래프	생산 현황 등 데이터를 시각화
데이터 표	상세 데이터 기록 확인
입력창	새로운 데이터 입력
버튼	데이터 조회, 저장 등 기능 실행
경고 메시지	온도 초과 등 이상 상태 안내

화면 구성 예시: 스마트공장 생산 모니터링 프로그램



학습 포인트

1. 먼저 어떤 **화면이 필요한지(구성)**와 각 화면에서 **어떤 기능을 할지(내용)**를 생각합니다.
2. 이후, 생각한 화면 구성을 AI에게 설명하여 Streamlit 코드를 생성합니다.

바이브코딩 프롬프트 2

스마트공장 모니터링 화면 구성 설계 (Wireframe)

Streamlit으로 개발할 스마트공장 모니터링 프로그램의 **전체 화면 구조**와 각 **메뉴별 화면**을 설계한 예시입니다.

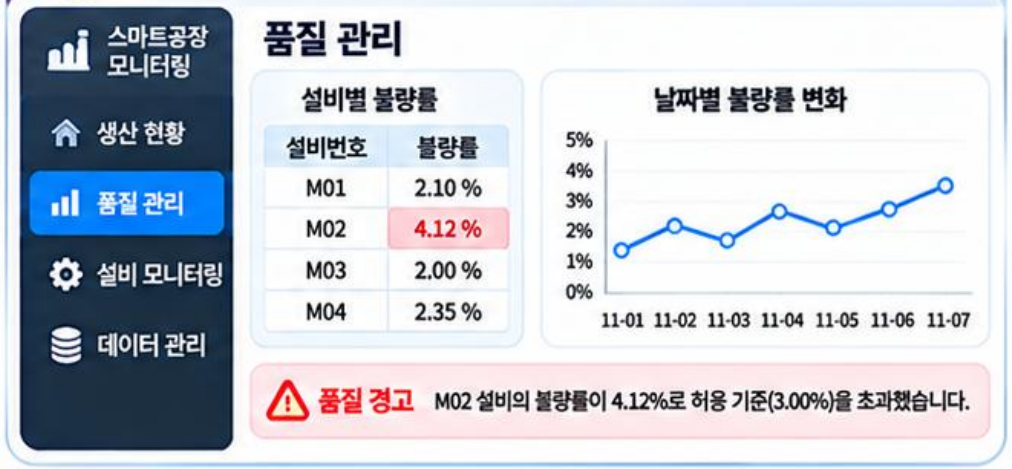
화면 구성 요구사항

- 왼쪽 사이드바 메뉴 배치
- 메뉴 4개 구성
  (생산 현황, 품질 관리,
  설비 모니터링, 데이터 관리)
- 오른쪽 영역에 선택한
  메뉴 내용 표시
- 각 메뉴별 주요 정보와
  기능 구현

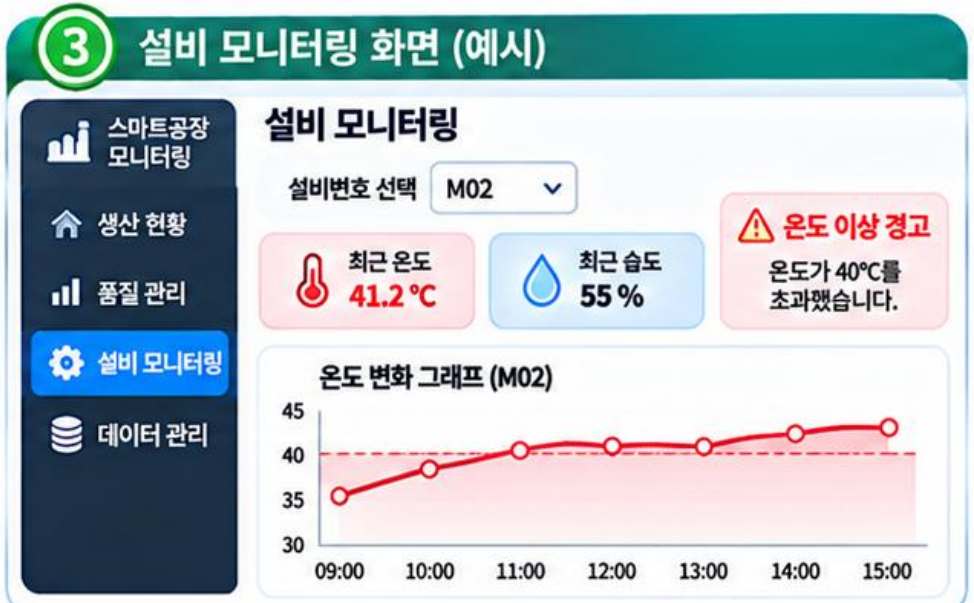
전체 화면 구조

1 생산 현황 화면 (예시)

2 품질 관리 화면 (예시)



3 설비 모니터링 화면 (예시)



4 데이터 관리 화면 (예시)

Wireframe 이해하기

Wireframe(와이어프레임)이란?

프로그램의 실제 디자인을 구현하기 전에 **화면의 구성 요소와 배치를** 간단하게 표현한 **설계도**입니다.

왜 필요할까요?

- 프로그램의 전체 구조를 미리 계획할 수 있어요.
- 메뉴, 데이터, 그래프, 버튼 등의 위치를 정할 수 있어요.
- 개발 전에 필요한 기능을 빠짐없이 확인할 수 있어요.

화면 구성 주요 요소

제목	사이드바 (메뉴)	주요 지표 카드 (숫자 표시)	그래프 (시각화)
데이터 표 (상세 정보)	입력창 (데이터 입력)	버튼 (기능 실행)	경고 메시지 (이상 안내)

생산 현황 화면 예시 (Wireframe)

기억하세요!

이 그림은 완성된 프로그램이 아닙니다.
앞으로 만들 프로그램에서 **어떤 정보가 어디에 나타날 것인지** 미리 정리한 것입니다.

화면 설계 실습

설비 모니터링 화면 설계

특정 설비를 선택하여 **현재 상태(온도, 습도)**를 확인하고, 이상이 발생하면 **경고를 표시하**는 화면을 설계해 봅니다.

화면 구성 요소

- **설비 선택** — 확인할 설비를 선택
- **현재 온도** — 선택한 설비의 최신 온도 표시
- **현재 습도** — 선택한 설비의 최신 습도 표시
- **설비 상태** — 정상/경고 등 설비 상태 표시
- **온도 그래프** — 시간에 따른 온도 변화 확인

설비 모니터링

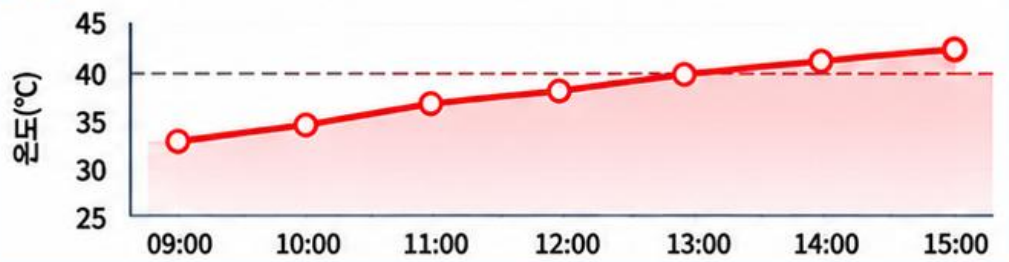
설비 선택 : M003 ▾ ← 설비 선택 메뉴 (드롭다운)

현재 온도 **43℃**

현재 습도 **50%**

설비 상태 : **경고** | 온도 경고 : 기준 온도를 초과했습니다.

시간별 온도 변화



생각해 볼 질문

1. 설비 선택 메뉴는 어디에 배치하는 것이 좋을까요?
2. 현재 온도는 그래프보다 먼저 보여주는 것이 좋을까요?
3. 경고 메시지는 어떤 색상으로 표시하면 좋을까요?

설계 포인트

- 중요한 정보(온도, 상태)는 먼저 강조하기
- 경고는 눈에 띄는 색상(빨강, 주황) 사용하기
- 그래프로 변화 추이를 한눈에 보여주기

핵심 정리

사용자가 쉽게 확인하고 빠르게 이해할 수 있도록 **정보의 배치와 강조 방식**을 미리 계획하는 것이 좋은 화면 설계의 핵심입니다.

실습 6. 기능 추가하기

AI가 만든 설계를 직접 변경하기 (기능 추가 예시)

기존 프로그램에 **원하는 기능을 추가**하여, 더 유용한 서비스로 개선해 봅니다.

추가 기능 선택 목록

선택	기능	설명
A	생산 목표 달성률	목표 대비 설제 생산량 비교
B	설비별 생산량 순위	생산량이 많은 설비 순위 표시
C	온도 경고 기준 변경	사용자가 경고 온도 설정
D	생산 데이터 검색	날짜와 설비번호로 검색
E	데이터 다운로드	조회 결과를 CSV로 저장
F	대시보드 디자인 변경	색상 및 화면 구성 수정

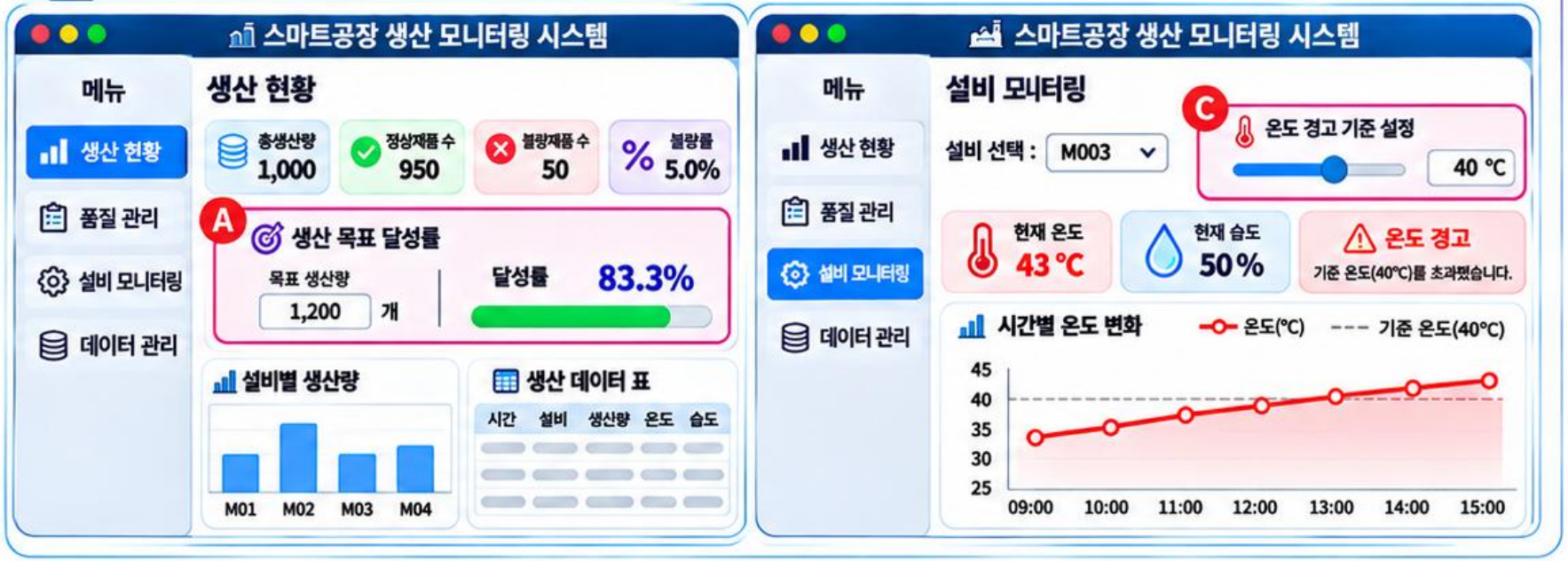
선택한 추가 기능 (예시)

- ✓ A 생산 목표 달성률
- ✓ C 온도 경고 기준 변경

변경된 프로그램 구성표 (요약)

메뉴	주요 기능 (기준 + 추가)	추가된 기능
생산 현황	총생산량, 정상/불량 수, 불량률, 설비별 생산량, 1 생산 목표 달성률	A
품질 관리	설비별 불량률, 날짜별 변화, 품질 경고	—
설비 모니터링	설비 선택, 현재 온도/습도, 온도 그래프, 2 온도 경고 기준 설정	C
데이터 관리	생산 데이터 조회, 설비번호 검색, 날짜 검색, 데이터 입력	—

변경된 화면 설계도 (Wireframe)



핵심 정리

- ✓ A **생산 목표 달성률:** 목표 생산량을 입력하면 실제 생산량과 비교해 달성률을 숫자와 진행률 막대로 확인할 수 있습니다.
- ✓ C **온도 경고 기준 변경:** 사용자가 원하는 온도 기준을 직접 설정하여, 설비의 이상 여부를 유연하게 관리할 수 있습니다.

실습 7. 기능 수정하기

품질 관리 화면 간소화하기

AI가 만든 화면에서 불필요한 기능을 제거하고, **필수 기능만 남겨 더 간단하고 이해하기 쉬운 화면**으로 수정합니다.

수정 전 (AI가 만든 복잡한 화면)

스마트공장 생산 모니터링 시스템

- 생산 현황
- 품질 관리
- 설비 모니터링
- 데이터 관리

품질 관리 (복잡한 구성)

- 전체 불량률 5.0%
- 날짜별 불량률 변화
- 설비별 불량률(%) — M01 M02 M03 M04
- 불량 유형 비율 — 외관 40% / 치수 30% / 기능 20% / 기타 10%
- 불량 발생 내역

일자	설비	불량 수	불량률
11-01	M01	12	4.8%
11-01	M02	18	5.9%
11-01	M03	10	3.9%

- 품질 추세 분석

수정 후 (간단하고 핵심만 있는 화면)

스마트공장 생산 모니터링 시스템

- 생산 현황
- 품질 관리
- 설비 모니터링
- 데이터 관리

품질 관리 (간소화)

- 전체 불량률 5.0%
- 불량률 경고: 기준값(5.0%)을 초과했습니다.
- 설비별 불량률 비교

유지하는 핵심 기능

- **전체 불량률 표시** — 전체 생산 대비 불량률 확인
- **설비별 불량률 그래프** — 각 설비의 불량률을 비교
- **불량률 경고** — 기준값 초과 시 경고 메시지 표시

추가 기능 목록 (추후 개발)

- 날짜별 불량률 변화 추이
- 불량 유형별 비율(외관, 치수 등)
- 불량 발생 내역 상세 표
- 품질 추세 분석 및 예측

핵심 정리

- AI의 설계를 그대로 사용하지 않고, **필요한 기능만 선택**하여 수정할 수 있습니다.
- 불필요한 기능은 추가 기능으로 분리하여, **단계적으로 확장**할 수 있습니다.

Streamlit 기초

Streamlit이란?

Streamlit

Streamlit은 Python으로 데이터 중심 웹 프로그램을 쉽게 만들 수 있는 도구입니다.

복잡한 HTML, CSS, JavaScript 없이도 간단한 화면과 그래프를 구성할 수 있습니다.

주요 명령어와 역할

명령어	역할	화면 예시
st.title()	제목 표시	스마트공장 모니터링
st.write()	문자 및 데이터 표시	생산 현황을 확인할 수 있습니다.
st.metric()	주요 지표 표시 (숫자와 변화량)	총생산량 1,000 ▲ +50
st.button()	버튼 생성 (기능 실행)	데이터 업데이트
st.selectbox()	선택 메뉴 (항목 선택)	설비 선택 : M001
st.dataframe()	데이터 표 표시 (데이터 확인)	시간 / 설비 / 생산량 — 09:00 / M001 / 120

핵심 정리

모든 코드를 암기할 필요는 없습니다.

각 명령어가 화면에서 어떤 역할을 하는지만 이해하면 됩니다.

Google Colab과 Streamlit의 차이와 개발 흐름

Google Colab과 Streamlit 비교

구분	Google Colab	Streamlit
주요 목적	Python 코드 실행 및 데이터 분석	웹 프로그램 제작
실행 방식	코드 셀 실행	웹 서버 실행 (streamlit run)
결과 확인	노트북에 결과 표시	웹브라우저에서 확인
화면 구성	코드·표·그래프 중심	메뉴·버튼·대시보드 중심
주요 활용	데이터 분석, 머신러닝	데이터 대시보드, 웹앱

코드 예시로 이해하기

Google Colab (일반 Python 실행)

```
print("스마트공장 모니터링")
```

결과 (노트북 출력)

스마트공장 모니터링

Streamlit (웹 프로그램)

```
import streamlit as st
st.title("스마트공장 모니터링")
```

결과 (웹브라우저 화면)

스마트공장 모니터링

이번 수업의 개발 방식 (전체 흐름)

1. Google Colab에서 코드 작성(바이브코딩)
   - ✓ AI의 도움으로 Python 코드 생성
   - ✓ app.py 작성
2. app.py 파일로 저장
   - ✓ 완성한 코드를 app.py로 저장
3. GitHub에 파일 저장
   - ✓ GitHub 저장소에 app.py 업로드 버전 관리 가능
4. Streamlit Community Cloud 연결 및 배포
   - ✓ GitHub 저장소를 연결하여 웹앱으로 자동 배포
   - ✓ 별도 서버 없이 사용 가능
5. 웹브라우저에서 확인 및 기능 개선
   - ✓ 배포된 웹 프로그램 실행
   - ✓ 결과 확인 후 코드 수정 및 기능 추가

실습 1 | Google Colab에서 첫 번째 Streamlit 프로그램 만들기

Google Colab에서 Streamlit을 설치하고, 웹 프로그램을 만들기 위한 **기본 환경을 준비**합니다.

STEP 1. Google Colab 실행 및 노트북 준비

1 Google Colab 접속하기

https://colab.research.google.com

2 새 노트북 생성하기

Google Colab

파일 수정 보기 삽입 런타임 도구 도움말 + 새 노트북

3 노트북 이름 변경하기

SmartFactory_02.ipynb

파일 수정 보기 삽입 런타임 도구 도움말

4 첫 번째 코드 셀 생성하기

▶ |

STEP 2. Streamlit 설치하기

Colab의 코드 셀에 다음 명령어를 입력하고 실행합니다.

```
!pip install streamlit
```

명령어 설명

- `pip` : Python 라이브러리를 설치하는 도구입니다.
- `!` : Colab에서 시스템 명령어를 실행한다는 의미입니다.

즉, "우리가 사용할 Streamlit이라는 도구를 설치하는 명령어"입니다.

설치 완료 화면 (예시)

```
Collecting streamlit
  Downloading streamlit-1.40.0-py3-none-any.whl (10.2 MB)
...          ...
Successfully installed streamlit-1.40.0
```

설치가 완료되면 다음 단계로 넘어갑니다.

핵심 정리

- Google Colab에서 새로운 노트북을 만들고, 이름을 **SmartFactory_02.ipynb**로 변경합니다.
- **!pip install streamlit** 명령어로 Streamlit을 설치하여, 웹 프로그램을 만들 준비를 합니다.

실습 1-2 | 바이브코딩으로 첫 번째 프로그램 만들기

ChatGPT에 프롬프트를 입력하여 Streamlit으로 가장 간단한 웹 프로그램을 만들어봅니다.

바이브코딩 프롬프트 (예시)

너는 Python과 Streamlit을 가르치는 프로그래밍 강사야.
나는 Google Colab에서 바이브코딩으로
스마트공장 모니터링 프로그램을 만들려고 해.
현재 Python 기초만 알고 있어.
먼저 가장 간단한 Streamlit 프로그램을 만들어줘.

화면에 표시할 내용

1. **제목:** 스마트공장 생산 모니터링 시스템
2. **설명:** 이 프로그램은 공장의 생산량과 불량률을 확인하기 위한 모니터링 프로그램입니다.
3. **현재 개발 단계:** 기본 화면 제작

생성된 코드 (app.py)

```python
import streamlit as st
# 프로그램 제목
st.title("스마트공장 생산 모니터링 시스템")
# 프로그램 설명
st.write("이 프로그램은 공장의 생산량과 불량률을
          확인하기 위한 모니터링 프로그램입니다.")
# 현재 개발 단계
st.write("현재 개발 단계: 기본 화면 제작")
```

주요 코드 설명

코드	의미
`import streamlit as st`	Streamlit을 사용하기 위한 준비(라이브러리 불러오기)
`st.title()`	프로그램의 제목을 화면에 크게 표시
`st.write()`	문자(설명, 안내문 등)를 화면에 표시

실행 결과 (웹 화면 예시)

스마트공장 생산 모니터링 시스템

이 프로그램은 공장의 생산량과 불량률을 확인하기 위한 모니터링 프로그램입니다.

현재 개발 단계: 기본 화면 제작

핵심 포인트 | ✓ 코드를 모두 외울 필요는 없습니다. ✓ 각 명령어가 화면에서 어떤 역할을 하는지만 이해하면 됩니다.

실습 1-3

Colab에서 app.py 파일 생성하기

Streamlit 프로그램은 Python 파일(.py)로 저장해야 합니다.

1 새 코드 셀에 아래 코드를 입력합니다.

```
%%writefile app.py

import streamlit as st

# 프로그램 제목
st.title("스마트공장 생산 모니터링 시스템")

# 프로그램 설명
st.write("공장의 생산량과 불량률을 확인하는 프로그램입니다.")

# 개발 단계 안내
st.write("현재 개발 단계: 기본 화면 제작")
```

2 코드를 실행합니다. ▶ (실행)

3 실행 결과로 app.py 파일이 생성됩니다.

```
Writing app.py
```

파일 생성 결과 확인

Colab 왼쪽의 파일(🗀) 메뉴를 열어 app.py가 있는지 확인합니다.

파일의 종류와 역할

파일 형식	설명
.ipynb	Colab에서 사용하는 노트북 파일 (코드 실행, 분석, 정리)
.py	Python 프로그램 파일 (Streamlit 웹앱 코드 저장)
.csv	데이터를 저장하는 파일 (생산 데이터, 설비 데이터 등)

참고: 파일 저장 위치

Colab의 파일은 런타임(세션)이 종료되면 사라질 수 있습니다.
따라서 중요한 파일(app.py, 데이터 파일 등)은
GitHub 또는 Google Drive에 저장하세요.

GitHub Google Drive

핵심 정리

- Colab에서는 '%%writefile'을 사용해 Python 파일(app.py)을 생성할 수 있습니다.
- 생성된 app.py는 GitHub 또는 Google Drive에 저장하여 계속 사용할 수 있습니다.

실습 1-4 | 주요 생산지표 표시하기

Streamlit의 `st.metric()`을 사용하여 공장의 주요 생산지표를 화면에 표시해봅니다.

사용할 가상 데이터(고정값)

항목	값
총생산량	1,000개
정상 제품	950개
불량 제품	50개
불량률	5%
평균 공정 온도	38℃

바이브코딩 프롬프트 3-2 (예시)

앞에서 만든 Streamlit 프로그램에
생산 현황을 표시하는 기능을 추가해줘.

다음 숫자를 사용해줘.
- 총생산량: 1000개
- 정상 제품: 950개
- 불량 제품: 50개
- 불량률: 5%
- 평균 공정 온도: 38℃

Streamlit의 st.metric()을 사용하여
숫자를 카드 형태로 보여줘...

실행 결과 화면(예시)

Streamlit

스마트공장 생산 모니터링 시스템

공장의 생산 현황을 확인하는 프로그램입니다.

총생산량	1,000개
정상 제품	950개
불량 제품	50개
불량률	5%
평균 공정 온도	38℃

생성된 코드 (app.py)

```
1  import streamlit as st
2  # 프로그램 제목
3  st.title("스마트공장 생산 모니터링 시스템")
4  # 프로그램 설명
5  st.write("공장의 생산 현황을 확인하는 프로그램입니다.")
6  st.metric("총생산량", "1,000개")
6  st.metric("정상 제품", "950개")
7  st.metric("불량 제품", "50개")
9  st.metric("불량률", "5%")
8  st.metric("평균 공정 온도", "38℃")
```

st.metric()란?

중요한 지표(숫자)를 카드 형태로
강조하여 표시하는 명령어입니다.

`st.metric("지표 이름", "표시할 값")`

"지표 이름"	화면에 표시될 지표의 이름 (예: 총생산량)
"표시할 값"	지표에 표시할 값 (예: 1,000개)

핵심 정리

- st.metric()을 사용하면 주요 지표를 카드 형태로 쉽게 표시할 수 있습니다.
- 지표 이름과 값을 입력하면 자동으로 보기 좋은 카드로 출력됩니다.

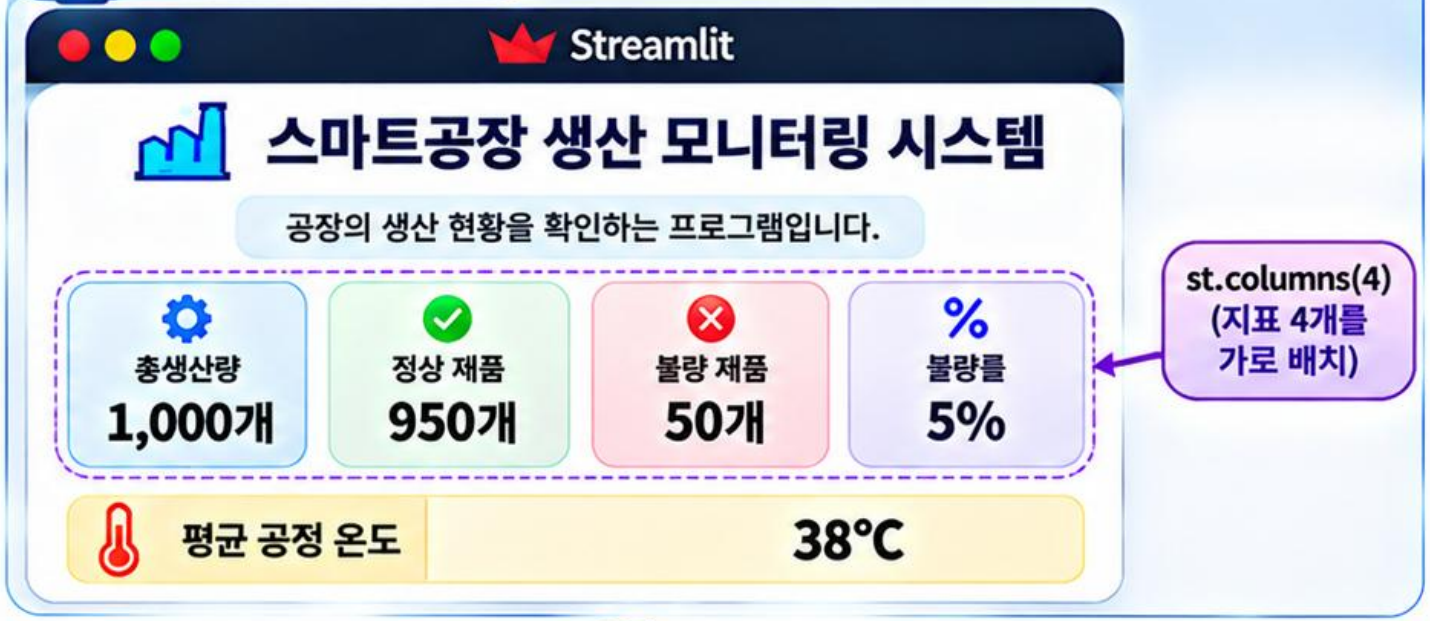
실습 1-5 | 주요 지표를 가로로 배치하기 (st.columns)

st.columns()를 사용하여 여러 개의 지표를 가로로 나란히 배치해 봅니다.

수정된 코드 (app.py)

```python
import streamlit as st

# 프로그램 제목
st.title("스마트공장 생산 모니터링 시스템")
# 프로그램 설명
st.write("공장의 생산 현황을 확인하는 프로그램입니다.")

# 화면을 네 개의열로 분리
col1, col2, col3, col4 = st.columns(4)
# 각 열에 지표 표시
with col1:   st.metric("총생산량",  "1,000개")
with col2:   st.metric("정상 제품", "950개")
with col2:   st.metric("불량 제품", "50개")
with col4:   st.metric("불량률",  "5%")

# 아래애 평균 온도 표시
st.metric("평균 공정 온도",  "38℃")
```

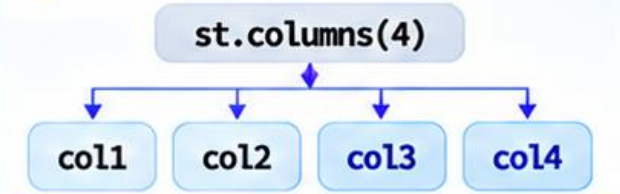
실행 결과 화면



주요 코드 설명

코드	설명
col1, col2, col3, col4 = = st.columns(4)	화면을 4개의 열로 나누어 col1~col4로 저장합니다.
with col1: st.metric("총생산량", "1,000개")	첫 번째 열(col1)에 '총생산량' 지표를 표시합니다.

st.columns() 개념



화면을 가로로 4개의 영역(열)으로 나눕니다.

학생 확인 문제

Q1 지표를 두 개씩 가로로 배치하려면? → st.columns(2)

Q2 화면을 세 개의 열로 나누려면? → st.columns(3)

핵심 정리

- st.columns(n)은 화면을 가로로 n개의 열로 나누어 지표나 내용을 배치하는 명령어입니다.
- 각 열에 st.metric(), st.write() 등을 사용하여 원하는 내용을 표시할 수 있습니다.

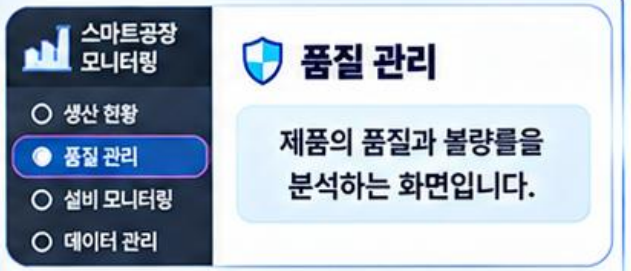
실습 1-6 | 사이드바 메뉴 추가하기

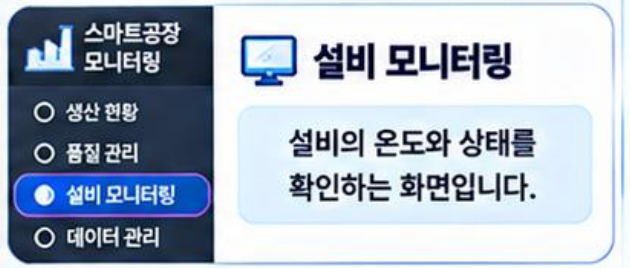
Streamlit의 사이드바와 조건문(if·elif)을 이용하여 4개의 메뉴를 만들고, 메뉴에 따라 다른 화면을 표시해봅니다.

전체 코드 (app.py)

```python
import streamlit as st

st.title("스마트공장 생산 모니터링 시스템")

# 사이드바 메뉴 생성 (4개 메뉴)
menu = st.sidebar.radio(
    "메뉴 선택",
    ["생산 현황", "품질 관리", "설비 모니터링", "데이터 관리"]
)

# 선택한 메뉴에 따라 다른 화면 표시
if menu == "생산 현황":
    st.header("생산 현황")
    col1, col2, col3, col4 = st.columns(4)
    with col1: st.metric("총생산량", "1,000개")
    # ... (나머지 지표 표시)
elif menu == "품질 관리":
    st.header("품질 관리")
    st.write("제품의 품질과 불량률을 분석하는 화면입니다.")
elif menu == "설비 모니터링":
    st.header("설비 모니터링")
    st.write("설비의 온도와 상태를 확인하는 화면입니다.")
elif menu == "데이터 관리":
    st.header("데이터 관리")
    st.write("생산 데이터를 조회하고 관리하는 화면입니다.")
```

실행 결과 (메뉴별 화면 예시)





주요 개념: 사이드바와 메뉴 선택

`st.sidebar.radio()`	왼쪽 사이드바에 선택 메뉴를 생성
`if / elif`	선택한 메뉴에 따라 다른 화면을 표시

메뉴 전환 확인하기

1. 생산 현황 선택 → 생산 지표 카드가 표시되는가? ✓
2. 품질 관리 선택 → 안내 문구가 표시되는가? ✓
3. 설비 모니터링 선택 → 안내 문구가 표시되는가? ✓
4. 데이터 관리 선택 → 안내 문구가 표시되는가? ✓

핵심 정리

- ✓ `st.sidebar.radio()`로 여러 메뉴를 만들고, `if / elif`로 선택한 메뉴에 맞는 화면을 표시합니다.
- ✓ 이전 시간의 생산지표는 '생산 현황' 메뉴에서 계속 사용할 수 있습니다.

실습 | 스마트공장 모니터링 프로그램 만들기 (Streamlit)

Google Colab에서 Streamlit을 사용하여 4개의 메뉴로 구성된 생산 모니터링 웹 프로그램을 만들어봅니다.

1 전체 코드 (Google Colab에서 실행)

```
%%writefile app.py

import streamlit as st

# 1. 웹 프로그램 기본 설정
st.set_page_config(
    page_title="스마트공장 모니터링",  # 브라우저 제목
    page_icon="🏭",   # 아이콘(공장)
    layout="wide"    # 화면을 넓게 사용
)
# 2. 제목과 안내
st.title("스마트공장 생산 모니터링 시스템")
st.write("공장의 생산량과 설비 상태를 확인하는 프로그램입니다.")
st.info("현재 표시되는 데이터는 교육용 가상 데이터입니다.")

# 3. 사이드바 메뉴
menu = st.sidebar.radio(
    "메뉴 선택",
    ["생산 현황", "품질 관리", "설비 모니터링", "데이터 관리"]
)

# 4. 메뉴별 화면 구성
if menu == "생산 현황":
    st.header("생산 현황")
    st.write("공장의 주요 생산지표를 확인합니다.")
    col1, col2, col3, col4 = st.columns(4)
    with col1:  st.metric("총생산량", "1,000개")
    with col2:  st.metric("정상 제품", "950개")
    with col3:  st.metric("불량 제품", "50개")
    with col4:  st.metric("불량률",  "5.0%")
    st.divider()
    st.metric("평균 공정 온도",  "38℃")
    st.write("다음 수업에서 실제 CSV 데이터를 연결합니다.")
elif menu == "품질 관리":
    st.header("품질 관리")
    st.write("제품의 품질과 불량률을 확인하는 화면입니다.")
    st.info("추후 설비별 불량률과 품질 경고 기능을 추가합니다.")
elif menu == "설비 모니터링":
    st.header("설비 모니터링")
    st.write("설비별 온도와 습도를 확인하는 화면입니다.")
    st.info("추후 설비 선택과 온도 경고 기능을 추가합니다.")
```

2 실행 화면 (생산 현황)

3 메뉴별 화면 예시

품질 관리 화면

품질 관리

제품의 품질과 불량률을 확인하는 화면입니다.

추후 설비별 불량률과 품질 경고 기능을 추가합니다.

설비 모니터링 화면

설비 모니터링

설비별 온도와 습도를 확인하는 화면입니다.

추후 설비 선택과 온도 경고 기능을 추가합니다.

데이터 관리 화면

데이터 관리

생산 데이터를 조회하고 관리하는 화면입니다.

추후 데이터 검색과 입력 기능을 추가합니다.

4 주요 명령어와 역할

명령어	역할
st.set_page_config()	웹페이지 기본 설정 (제목, 아이콘, 화면 크기)
st.title()	프로그램의 큰 제목 표시
st.write()	설명 문자 표시
st.info()	안내 메시지를 강조하여 표시
st.sidebar.radio()	사이드바에 메뉴(라디오 버튼) 표시
st.header()	각 화면의 제목(소제목) 표시
st.columns(n)	화면을 n개의 열로 나누어 배치
st.metric()	주요 숫자(지표)를 카드 형태로 표시
st.divider()	구분선을 표시하여 화면을 정리

테스트 방법 (정상 작동 확인)

1. Colab에서 코드를 실행하여 app.py 파일을 생성한다.
2. 다음 명령어로 Streamlit을 실행한다.

```
!streamlit run app.py
```

3. 브라우저에서 프로그램이 열리면 사이드바의 4개 메뉴를 각각 선택한다.
4. 아래 화면이 정상적으로 표시되는지 확인한다.

- **생산 현황:** 1,000개 / 950개 / 50개 / 5.0% / 38℃가 표시되는지
- **품질 관리:** 안내 문구가 표시되는지
- **설비 모니터링:** 안내 문구가 표시되는지
- **데이터 관리:** 안내 문구가 표시되는지

실습 1-8 | app.py 실행 및 결과 확인하기 (GitHub + Streamlit Community Cloud)

Google Colab에서 만든 app.py를 GitHub에 업로드하고, Streamlit Community Cloud로 배포하여 웹에서 실행해봅니다.

1 Google Colab — app.py 작성 및 다운로드 → **2 GitHub** — 코드 저장소에 업로드 (예: smart-factory-monitoring) → **3 Streamlit Community Cloud** — GitHub 저장소를 연결하여 배포 → **웹에서 실행 및 확인** (배포된 URL로 접속)

실습 진행 과정 (상세 단계)

1 Colab에서 app.py 다운로드

```
from google.colab import files
files.download("app.py")
```

2 GitHub 저장소 열기

- GitHub에 로그인하고 새 저장소 생성
- 저장소 이름 예: `smart-factory-monitoring`

3 app.py 업로드

- 저장소에서 **Add file → Upload files**
- Colab에서 다운로드한 app.py 파일 업로드

4 requirements.txt 준비

- 저장소에 requirements.txt 파일 추가

```
requirements.txt
streamlit
```

5 Streamlit Community Cloud에서 배포

- https://share.streamlit.io 접속 및 로그인
- "New app" 클릭 → GitHub 저장소 선택
- 실행 파일: app.py 선택 → Deploy(배포)

최종 결과 화면 예시 (배포된 웹앱)

장점 (추천 이유)

- 웹에서 바로 실행 가능
- 설치 과정이 필요 없음
- 코드 수정 후 바로 반영
- 학생 간 결과 확인·공유가 쉬움

주의할 점

- 코드 수정 후 자동 반영까지 약 1~2분 정도 소요될 수 있음
- 무료 계정은 사용량에 제한이 있음 (교육 실습에는 충분)
- GitHub 저장소를 공개(Public)로 설정해야 배포가 가능함

핵심 정리

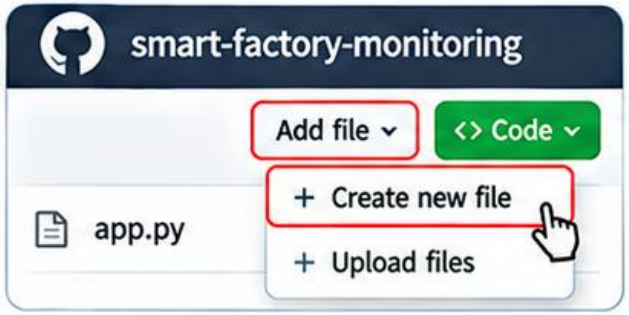
- Colab에서 만든 app.py를 GitHub에 업로드하고, Streamlit Community Cloud로 배포하면 웹브라우저에서 누구나 바로 실행할 수 있습니다.
- 첫 배포 이후에는 GitHub 웹 편집기에서 app.py를 직접 수정하여 더 쉽게 실습을 이어갈 수 있습니다.

실습 1-8 | 상세 과정 4·5 : requirements.txt 준비와 Streamlit 배포

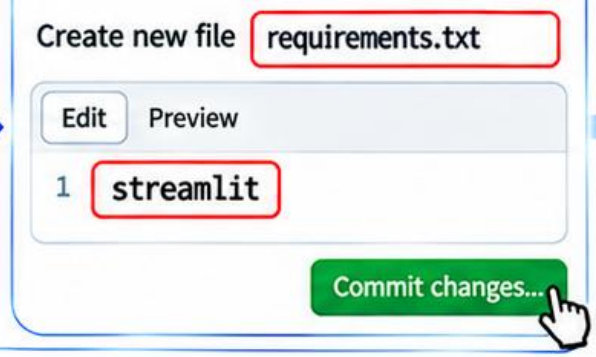
4 requirements.txt 준비 (의존성 설정)

앱을 실행하는데 필요한 라이브러리 정보를 적는 파일입니다. (이번 실습에서는 Streamlit만 사용)

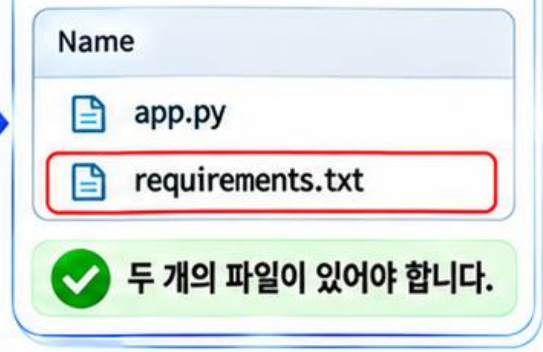
1 GitHub 저장소에서 파일 추가



2 파일 이름과 내용 입력



3 업로드 결과 확인



핵심 설명

- **requirements.txt :** 프로그램에 필요한 라이브러리 목록
- 현재 실습에서는 streamlit만 작성
- GitHub에 app.py와 requirements.txt 두 파일이 있어야 배포 가능

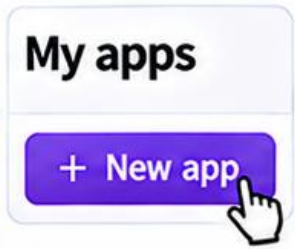
5 Streamlit Community Cloud에서 배포 (웹앱 공개)

GitHub 저장소의 코드를 웹에 배포하여 누구나 브라우저에서 실행할 수 있습니다.

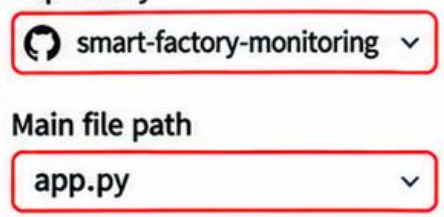
1 Streamlit Community Cloud 접속

웹 브라우저에서 접속 후
GitHub 계정으로 로그인합니다.

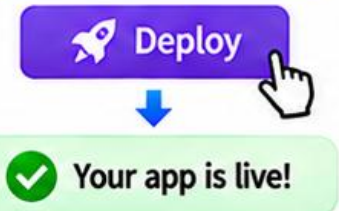
2 새 앱 생성



3 GitHub 저장소와 파일 선택



4 배포 시작 및 완료



배포가 완료되면
웹 주소(URL)가 생성됩니다.

결과 화면 (예시)

핵심 정리

- **4 requirements.txt :** 실행에 필요한 라이브러리(streamlit)를 기록하여 앱이 정상적으로 작동하도록 합니다.
- **5 Streamlit Community Cloud :** GitHub의 코드를 선택해 웹앱으로 배포하고, 생성된 URL로 결과를 확인합니다.

3교시 | CSV 파일 연결하기

데이터 연결 흐름

CSV 파일
(production_data.csv)
생산 데이터가 저장된 파일

→

pandas
DataFrame
데이터를 읽어 표 형태로 처리

→

Streamlit
(웹 화면)
데이터를 웹 화면에 표·지표·그래프로 표시

각 단계의 의미

CSV 파일	• 생산 데이터를 쉼표(,)로 구분해 저장한 파일 (예: production_data.csv)
pandas	• CSV 파일을 읽고, 데이터를 분석·처리하는 Python 라이브러리
DataFrame	• 데이터를 행(기록)과 열(항목)로 구성한 표 형태 (엑셀처럼 다룰 수 있는 자료구조)
Streamlit	• DataFrame의 데이터를 웹 화면에 표, 숫자 카드, 그래프 등으로 보여주는 도구

실제 제조데이터 구조 (production_data.csv)

전체 데이터 개수	10,000건
컬럼 개수	8개
설비 수	100대
제품 종류	20종
시작 시간	2026-11-01 00:00
종료 시간	2026-11-05 03:00

데이터 컬럼 (총 8개)

컬럼명	의미	활용 예시
timestamp	생산 기록 시간	날짜·시간 필터, 기간별 생산량 분석
machine_id	설비번호	설비 선택, 설비별 생산량 비교
product_id	제품번호	제품별 생산량·불량률 분석
production_count	생산량	총생산량 계산, 목표 달성률 계산
defect_count	불량 수량	불량 제품 수, 불량률 계산
temperature	공정 온도	평균 온도, 온도 추이 확인
humidity	습도	설비 환경 분석(습도와 불량률 관계 등)
operating_status	설비 가동 상태	가동(운전)·정지 상태 확인, 가동률 분석

이번 교시에서 활용하는 주요 컬럼 | **timestamp, machine_id, production_count, defect_count, temperature**를 중심으로 생산량 계산, 불량률 분석, 그래프, 필터 기능을 구현합니다.

3교시 | CSV 파일 업로드 및 연결하기 (바이브코딩 실습)

생산 데이터를 CSV로 연결하여 Streamlit 프로그램에서 읽고, 기본 정보를 화면에 표시합니다.

1 GitHub 파일 구조

Colab에서 만든 파일을 GitHub에 업로드하고, CSV 파일도 같은 저장소에 넣습니다.

```
smart-factory-monitoring/
├── app.py                 Streamlit 프로그램
├── production_data.csv    생산 데이터(CSV)
└── requirements.txt       필요한 라이브러리 목록
```

2 requirements.txt

프로그램에 필요한 라이브러리를 작성합니다.

```
streamlit
pandas
plotly
```

의미

- streamlit: 웹 앱 개발 도구
- pandas: 데이터 읽기·처리
- plotly:

3 CSV 읽기 코드와 설명

```
import pandas as pd
df = pd.read_csv("production_data.csv")
```

코드	설명
import pandas as pd	pandas 라이브러리를 pd라는 이름으로 사용
pd.read_csv(...)	CSV 파일을 읽어 DataFrame으로 변환
df	읽은 데이터를 저장하는 변수

4 바이브코딩 프롬프트 6 (복사하여 사용)

너는 Python과 Streamlit을 전문적으로 다루는 개발자야.

나는 앞에서 만든 스마트공장 모니터링 프로그램에 CSV 파일을 연결하려고 해.

기존 프로그램은 다음 네 가지 메뉴로 구성되어 있어.

1. 생산 현황
2. 품질 관리
3. 설비 모니터링
4. 데이터 관리

CSV 파일 이름은 production_data.csv야.
데이터에는 다음 컬럼이 있어.

timestamp, machine_id, product_id, production_count, defect_count, temperature, humidity, operating_status

다음 기능을 구현해줘.

1. pandas로 CSV 파일을 읽어줘.
2. timestamp를 날짜와 시간 형식으로 변환해줘.
3. 생산 현황 메뉴에서 다음 정보를 표시해줘.
   - 전체 데이터 개수
   - 전체 컬럼 개수
   - 데이터 시작 시간
   - 데이터 종료 시간
   - 처음 10개 데이터
4. CSV 파일이 없으면 파일을 업로드할 수 있도록 해줘.
5. 데이터 표는 st.dataframe()을 사용해줘.
6. 기존 4개 메뉴와 디자인은 유지해줘.
7. 기존 생산 현황 코드를 수정해줘.
8. 모든 주석은 한국어로 쉽게 설명해줘.
9. 복잡한 함수나 클래스는 사용하지 마.

Google Colab에서 %%writefile app.py로 파일 생성할 수 있게 전체 코드로 작성해줘.

5 생성된 app.py 예시 (핵심 코드)

```python
import streamlit as st  # Streamlit: 웹 앱 생성 라이브러리
import pandas as pd     # pandas: 데이터 분석 라이브러리

st.set_page_config(page_title="스마트공장 모니터링", layout="wide")
st.title("스마트공장 생산 관리 시스템")

# CSV 파일 읽기 (없으면 업로드)
try:
    df = pd.read_csv("production_data.csv")
except FileNotFoundError:
    st.warning("CSV 파일을 찾을 수 없습니다. 파일을 업로드해주세요.")
    uploaded = st.file_uploader("production_data.csv 업로드", type="csv")
    if uploaded is not None:
        df = pd.read_csv(uploaded)
```

주요 코드 설명

- 필요한 라이브러리 불러오기
- 페이지 제목 및 레이아웃 설정
- CSV 파일 읽기
- 파일이 없으면 업로드 기능 제공

실행 결과 화면 예시 (생산 현황 메뉴)

스마트공장 생산 모니터링

메뉴

- (●) 생산 현황
- ( ) 품질 관리
- ( ) 설비 모니터링
- ( ) 데이터 관리

생산 현황

전체 데이터 개수	전체 컬럼 개수	시작 시간	종료 시간
10,000건	8개	2026-11-01 00:00	2026-11-05 03:00

처음 10개 데이터

	timestamp	machine_id	product_id	production_count	defect_count	humidity	operating_status
0	2026-11-01 00:00:00	M001	P001	100	5	36.8	45.2	운전
1	2026-11-01 00:01:00	M001	P002	120	6	37.1	44.8	운전
2	2026-11-01 00:02:00	M002	P003	110	4	36.9	46.0	운전
4	2026-11-01 00:02:00	M001	P002	110	4	36.9	46.2	운전
	...	...	...	...	...	...	...	...

3교시

주요 생산지표 자동 계산하기

CSV 데이터의 생산량과 불량 수량을 이용하여 총생산량, 정상 제품 수, 불량 제품 수, 불량률을 자동으로 계산합니다.

기존 방식 vs. 자동 계산 방식

기존 (고정값)

```
st.metric("총생산량", "1,000개")
```

변경 (자동 계산)

```
total = df["production_count"].sum()
st.metric("총생산량", f"{total;,}개")
```

생산지표 계산 원리 (수식)

총생산량 (P)	$P = \sum_{i=1}^{n} p_i$	모든 생산량의 합
정상 제품 수 (N)	$N = P - D$	총생산량에서 불량 수량을 뺀 값
불량 제품 수 (D)	$D = \sum_{i=1}^{n} d_i$	모든 불량 수량의 합
불량률 (R)	$R = \frac{D}{P} \times 100$	전체 생산량 중 불량 제품의 비율(%)

예시로 이해하기

설비	생산량 (p_i)
M001	100
M002	120
M003	80
M004	150

계산 과정

- 총생산량: $P = 100 + 120 + 80 + 150 = 450$개
- 불량 수량: 예) $D = 20$개 (가정)
- 정상 제품 수: $N = 450 - 20 = 430$개
- 불량률: $R = \frac{20}{450} \times 4.44\%$

Python 코드 (계산 예시)

```
# 1. 총생산량 계산
total = df["production_count"].sum()

# 2. 불량 제품 수 계산
defect = df["defect_count"].sum()

# 3. 정상 제품 수 계산
normal = total - defect

# 4. 불량률 계산 (0으로 나누는 오류 방지)
defect_rate = defect / total * 100 if total > 0 else 0
```

실제 데이터 결과 (production_data.csv)

전체 생산량 (P)	1,118,576개
불량 제품 수 (D)	38,088개
정상 제품 수 (N)	1,080,488개
전체 불량률 (R)	3.41%

Streamlit 표시 결과 (st.metric)

총생산량	정상 제품 수	불량 제품 수	불량률
1,118,576개	1,080,488개	38,088개	3.41%

핵심 정리

- CSV의 production_count와 defect_count를 사용해 주요 생산지표를 계산한다.
- 불량률은 defect / total × 100이며, 생산량이 0인 경우에는 0으로 처리한다.
- st.metric()을 사용하면 계산한 결과를 카드 형태로 깔끔하게 표시할 수 있다.

3교시

설비별 생산량 그래프 만들기

설비별로 생산량을 합산하고, 생산량이 많은 상위 10개 설비를 막대그래프로 시각화합니다.

학습 목표

- 설비별 생산량을 계산하는 그룹화(Grouping) 이해
- 생산량이 많은 순서로 정렬
- 상위 10개 설비 선택
- Plotly로 막대그래프 시각화

설비별 생산량 계산 원리 (그룹화)

원본 데이터 (일부)

설비번호	생산량
M001	100
M002	120
M001	150
M003	90
M002	130

설비별 합계 결과

설비번호	총생산량
M001	250
M002	250
M003	90

같은 설비번호끼리 생산량을 합산한다.

- M001 : 100 + 150 = 250
- M002 : 120 + 130 = 250
- M003 : 90

그룹화(Grouping)란?

특정 기준(예: 설비번호)으로 데이터를 묶어(그룹화) 각 그룹의 값을 계산하는 기능

예시 설비별 생산량 합계, 평균 온도, 불량률 등 집계(요약)할 때 사용

사용 데이터 컬럼

컬럼명	역할
machine_id	설비 구분 (예: M001)
production_count	각 설비의 생산량

Python 코드로 구현하기

1 설비별 생산량 합계 (그룹화)

```
machine_data = df.groupby("machine_id")["production_count"].sum()
```

2 생산량이 많은 순서로 정렬

```
machine_data = machine_data.sort_values(ascending=False)
```

3 상위 10개 설비 선택

```
machine_data = machine_data.head(10)
```

4 Plotly로 막대그래프 작성

```
import plotly.express as px
fig = px.bar(machine_data, x=machine_data.index, y=machine_data.values,
             text=machine_data.values, title="설비별 생산량 TOP 10")
st.plotly_chart(fig, use_container_width=True)
```

그래프 결과 예시

핵심 정리

- df.groupby()로 설비별 데이터를 묶고, sum()으로 생산량을 합산한다.
- sort_values(ascending=False)로 생산량이 많은 순서로 정렬한다.
- head(10)으로 상위 10개 설비를 선택하여 Plotly로 막대그래프로 시각화한다.

3교시

날짜·설비 필터 구현하기

사용자가 원하는 설비와 기간을 선택하면, 해당 조건에 맞는 데이터만 분석하고 생산지표와 그래프가 함께 변경됩니다.

학습 목표

- 설비번호 선택 기능 구현
- 시작 날짜와 종료 팟호 선택
- 선택한 조건의 데이터만 분석
- 생산량·불량률·그래프 자동 변경

필터링(Filtering)의 개념

전체 데이터에서 사용자가 지정한 조건에 맞는 데이터만 선택하는 과정입니다.

- 원본 데이터를 삭제하지 않습니다.
- 설비, 날짜 등 조건으로 필요한 데이터만 추출하여 분석합니다.

예시: 필터링 결과

전체 데이터 (일부)

날짜	설비번호	생산량
11월 1일	M001	100
11월 1일	M002	120
11월 2일	M001	150
11월 2일	M002	130
11월 3일	M001	110

필터 조건

- 설비: M001
- 기간: 11월 1일 ~ 11월 3일

필터링 결과 (조건에 맞는 데이터만 선택)

날짜	설비번호	생산량
11월 1일	M001	100
11월 2일	M001	150
11월 3일	M001	110
총생산량		360

Streamlit UI (필터 영역)

조건 선택

설비 선택

전체

시작 날짜

2026-11-01

종료 날짜

2026-11-05

Python 코드 (Streamlit)

```python
# 1. 설비 선택 메뉴 만들기
machines = ["전체"] + sorted(df["machine_id"].unique().tolist())
machine = st.selectbox("설비 선택", machines)

# 2. 날짜 선택 메뉴 만들기
start_date = st.date_input("시작 날짜", df["timestamp"].min().date())
endd_date = st.date_input("종료 날짜", df["timestamp"].max().date())

# 3. 조건에 맞는 데이터 필터링
filtered = df[(df["timestamp"].dt.date >= start_date) &
              (df["timestamp"].dt.date <= end_date)].copy()
 if machine != "전체":
    filtered = filtered[filtered["machine_id"] == machine)
```

코드 설명

1 설비 선택 — df["machine_id"].unique()로 설비 목록을 가져와 st.selectbox()로 선택 메뉴를 생성합니다.

2 날짜 선택 — st.date_input()으로 시작 날짜와 종료 날짜를 선택합니다.

3 날짜 필터 — 선택한 기간의 데이터만 추출합니다. (df["timestamp"].dt.date >= 시작) & (<= 종료)

4 설비 필터 — "전체"가 아니면 선택한 설비의 데이터만 추가로 추출합니다.

핵심 정리

- st.selectbox()로 설비를 선택하고, st.date_input()으로 날짜 범위를 지정한다.
- pandas의 조건식으로 날짜와 설비 조건을 만족하는 데이터만 선택한다.
- 필터링된 데이터(filtered)를 사용해 생산량, 불량률, 그래프를 다시 계산한다.

3교시 | 생산 목표 달성률 계산하기

사용자가 목표 생산량을 입력하고, 실제 생산량과 비교하여 목표 달성률 계산하고 진행률 막대로 확인합니다.

계산 원리

$$\text{목표 달성률(\%)} = \frac{\text{실제 생산량}}{\text{목표 생산량}} \times 100$$

예시

- 목표 생산량: 10,000개
- 실제 생산량: 8,000개

$$\text{목표 달성률} = \frac{8{,}000}{10{,}000} \times 100 = 80\%$$

목표 생산량 입력 (Streamlit)

```
target = st.number_input(
    "목표 생산량",
    min_value=1,  value=10000,  step=100)
```

매개변수	의미
st.number_input()	숫자를 입력받는 위젯
min_value=1	최소 입력값 (1 이상만 입력 가능)
value=10000	기본값 (처음 표시되는 값)
step=100	증감 단위 (100씩 증가/감소)

달성률 계산 및 표시

1. 달성률 계산 — `achievement = total / target * 100`
2. 숫자로 표시 — `st.metric("목표 달성률", f"{achievement:.1f}%")`
3. 진행률 막대 — `st.progress(min(achievement / 100, 1.0))`

왜 min()을 사용할까요?

실제 생산량이 목표보다 많으면 달성률이 100%를 초과해도, 진행률 막대는 100%까지만 표시하기 위해 min(달성률/100, 1.0)을 사용합니다.

전체 처리 흐름

필터 적용 (설비·날짜) → 실제 생산량 계산 (total) → 목표와 비교 및 달성률 표시 (%)

실행 결과 예시

① 목표 미달 (80%)

목표 생산량	10,000개
실제 생산량	8,000개
목표 달성률	80.0%

80%

② 목표 달성 (100%)

목표 생산량	10,000개
실제 생산량	10,000개
목표 달성률	100.0%

100%

③ 목표 초과 (120%)

목표 생산량	10,000개
실제 생산량	12,000개
목표 달성률	120.0%

100%

핵심 정리

- 목표 달성률 = (실제 생산량 ÷ 목표 생산량) × 100 으로 계산한다.
- st.number_input()으로 목표 생산량을 입력받아 사용자가 변경할 수 있다.
- 달성률이 100%를 넘을 수 있으므로, 진행률 막대는 min(달성률/100, 1.0)로 100%까지만 표시한다.

3교시 | 바이브코딩 추가 실습 (기능 확장하기)

기본 기능을 완료한 후, AI에게 원하는 기능을 직접 요청하여 프로그램을 개선하는 활동입니다.

실습 목표

- AI에게 기능을 직접 설명하고 프로그램을 수정해보기
- 그래프 조건을 변 경해보기 (상위 개수 조정)
- 새로운 분석 그래프 추가하기 (제품별 생산량)
- 다른 관점에서 데이터 분석하기 (생산량이 적은 설비 확인)

추천 진행 방법

1. 원하는 기능을 생각하기 (무엇을, 어떻게 바꿀지 정하기)
2. AI에게 명확하게 요청하기 (프롬프트 작성)
3. 생성된 코드를 실행하고 결과 확인하기
4. 잘 동작하면 다음 기능으로 확장하기

추가 과제 A

생산량 그래프 변경

생산량이 많은 설비 10개 → 5개로 변경

프롬프트 예시

"설비별 생산량 그래프에서 상위 10개 대신 **상위 5개 설비만** 표시하도록 수정해줘. 나머지 기능은 변경하지 마."

결과 예시: 설비별 생산량 TOP 5

추가 과제 B

제품별 생산량 분석

제품(product_id)별 총생산량 그래프 추가

프롬프트 예시

기존 생산 현황 매뉴에 **제품별 생산량 그래프**를 추가해줘. product_id를 기준으로 production_count를 합산하고, 생산량이 높은 제품부터 막대그래프로 표시해줘."

결과 예시: 제품별 생산량 TOP 5

추가 과제 C

생산량이 가장 적은 설비 확인

생산량이 가장 적은 설비 5개 그래프 추가

프롬프트 예시

생산량이 **가장 적은 설비 5개**를 확인할 수 있도록 막대그래프를 추가해줘, 기존 TOP 10 그래프는 유지해줘."

결과 예시: 생산량이 가장 적은 설비 5개

핵심 포인트

- AI에게 수정하고 싶은 기능을 자연어로 설명하여 기존 프로그램을 개선할 수 있다.
- groupby()와 sum()을 활용하면 설비별, 제품별로 생산량을 쉽게 계산할 수 있다.
- 분석 목적에 따라 상위 N개, 하위 N개 등 다양한 관점으로 데이터를 시각화할 수 있다.

4교시

품질 관리의 기본 개념 이해하기

품질 관리 (Quality Management)란?

생산된 제품이 정해진 **품질 기준**을 만족하는지 확인하고, 문제가 발생하는 **원인**을 파악하여 **개선**하는 활동입니다.

예시: 생산설비별 생산 현황

설비번호	생산량	불량 수량	정상 제품 (생산량 – 불량 수량)
M001	100	3	97
M002	200	10	190
M003	150	12	138
M004	250	5	245

공장 관리자의 주요 질문

- 어떤 설비에서 **불량이 가장 많이** 발생할까?
- 어떤 설비의 **품질 상태**가 가장 좋을까?
- 특정 날짜에 **불량률이 증가**했을까?
- 불량률이 기준을 초과하면 **자동으로 경고**할 수 있을까?

불량 수량과 불량률 차이

불량 수량 (Defect Count)	불량으로 판정된 제품의 개수
불량률 (Defect Rate)	전체 생산량 중 불량 제품이 차지하는 비율

불량률 계산식

$$R = \frac{\text{불량 수량}(D)}{\text{생산량}\ (P)} \times 100(\%)$$

예시로 비교하기

설비	생산량 (P)	불량 수량 (D)	불량률 (R)
A	1,000	30	3%
B	100	10	10%

계산 과정

- A 설비: $R_A = \frac{30}{1{,}000} \times 100 = 3\%$
- B 설비: $R_B = \frac{10}{100} \times 100 = 10\%$

학생 확인 질문

- **Q1** 불량 수량이 더 많은 설비는? → **A 설비 (30개)**
- **Q2** 불량률이 더 높은 설비는? → **B 설비 (10%)**
- **Q3** 어느 설비의 품질 상태가 더 좋을까요? → **A 설비 (불량률 3%)**

해석 포인트

불량률만 비교하면 A 설비의 품질이 더 좋다고 볼 수 있습니다.
단, 실제 품질 평가에서는 **제품 종류, 생산 조건, 표본 크기** 등도 함께 고려해합니다.

핵심 정리

불량 수량은 불량 제품의 개수이고, **불량률**은 전체 생산량 대비 불량 제품의 비율입니다.
불량 수량이 많다고 반드시 불량률도 높은 것은 아닙니다. 생산량이 다른 설비를 비교할 때는 **불량 수량**과 **불량률**을 **함께 확인**해야 합니다.

4교시

실습 1. 설비별 불량률 계산하기

CSV 데이터를 이용하여 설비별로 생산량과 불량 수량 을 합산하고, 각 설비의 불량률을 계산합니다.

실습 목표

- 설비별 데이터를 묶기(그룹화)
- 설비별 생산량 합산하기
- 설비별 불량 수량 합산하기
- 설비별 불량률 계산하기

사용할 데이터

컬럼명	의미
machine_id	설비번호
production_count	생산량
defect_count	불량 수량

불량률 계산 원리

전체 불량률

$$R = \frac{D}{P} \times 100(\%)$$

설비별 불량률

$$R_m = \frac{D_m}{P_m} \times 100(\%)$$

- P : 전체 생산량 $(\sum p_i)$
- D : 전체 불량 수량 $(\sum d_i)$
- R : 전체 불량률(%)
- P_m : 설비 m의 총생산량
- D_m : 설비 m의 총불량 수량
- R_m : 설비 m의 불량률(%)

예제: 설비별 불량률 계산

설비번호	생산량 (P_m)	불량 수량 (D_m)	불량률 (R_m)
M001	100	3	3%
M002	200	10	5%
M003	150	12	8%
M004	250	5	2%

계산 과정 예시

- M003의 불량률

$$R_{M003} = \frac{12}{150} \times 100 = 8\%$$

- M004의 불량률

$$R_{M004} = \frac{5}{250} \times 100 = 2\%$$

학생 확인 문제

- Q1 불량률이 가장 높은 설비는? → M003
- Q2 불량률이 가장 낮은 설비는? → M004
- Q3 불량 수량이 가장 많은 설비는? → M003

중요한 이해

불량 수량이 가장 많다고
항상 불량률도 높은 것은 아닙니다.
생산량이 다른 설비를 비교할 때는
불량 수량과 불량률을 함께 확인해야 합니다.

Python으로 구현하기

1 설비별 생산량과 불량 수량 합산 (그룹화)

```
machine_quality = df.groupby("machine_id")[
    ["production_count", "defect_count"]
].sum()
# 같은 설비번호끼리 묶고
# 생산량과 불량 수량을 합산
```

2 설비별 불량률 계산

```
machine_quality["defect_rate"] = (
    machine_quality["defect_count"] /
    machine_quality["production_count"] * 100
)
# 불량 수량을 생산량으로 나누어 백분율(%)로 계산
```

3 생산량이 0인 경우 처리 (계산 불가)

```
import numpy as np
machine_quality["defect_rate"] = np.where(
    machine_quality["production_count"] > 0,
    machine_quality["defect_count"] /
      machine_quality["production_count"] * 100,
    np.nan   # 계산 불가로 표시
)
```

핵심 정리

- 설비별로 생산량과 불량 수량을 합산한 후 불량률을 계산한다.
- 불량률은 **(불량 수량 ÷ 생산량) × 100** 으로 계산한다.
- 생산량이 0인 경우 불량률은 계산하지 않고 **'계산 불가'**로 처리한다.

4교시

실습 2. 설비별 불량률 비교 그래프 만들기

설비별 불량률을 계산한 후, 불량률이 높은 상위 10개 설비를 막대그래프로 시각화합니다.

학습 목표

- 설비별 불량률을 그래프로 시각화
- 불량률이 높은 설비를 빠르게 확인
- 데이터 정렬과 Top N 선택
- Plotly로 막대그래프 구현

사용할 데이터

컬럼명	의미
machine_id	설비번호
production_count	총생산량
defect_count	불량 수량
defect_rate	불량률(%) (계산된 값)

처리 과정 (Step-by-Step)

1 불량률 기준 정렬

설비별 불량률을 높은 순서로 정렬합니다.

정렬 결과 (일부)

설비번호	불량률(%)
M100	4.13
M088	4.07
M031	4.01
...	

2 상위 10개 선택

정렬된 데이터에서 상위 10개 설비만 선택합니다.

상위 10개 데이터 (일부)

설비번호	불량률(%)
M100	4.13
M088	4.07
M031	4.01
...	

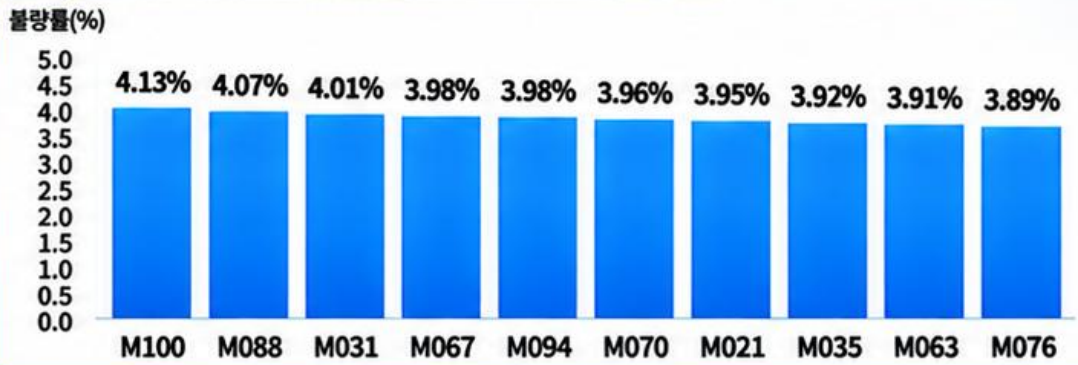
3 막대그래프 시각화

Plotly로 설비별 불량률을 막대그래프로 표시하고, 막대 위에 불량률(%)을 함께 보여줍니다.

주요 Python 코드 (Streamlit + Plotly)

```python
# 1. 불량률이 높은 순으로 정렬하고 상위 10개 선택
top10 = (machine_quality
         .sort_values("defect_rate", ascending=False)
         .head(10)
         .reset_index())
# 2. 막대그래프 생성 (Plotly)
fig = px.bar(top10, x="machine_id", y="defect_rate",
             title="설비별 불량률 TOP10", text="defect_rate")
# 3. 막대 위에 불량률을 소수점 둘째 자리까지 표시
fig.update_traces(texttemplate="%{y:.2f%}")
st.plotly_chart(fig, use_container_width=True)
```

실행 결과 예시 : 설비별 불량률 TOP 10



핵심 정리

- sort_values(ascending=False)로 불량률이 높은 순서로 정렬한다.
- head(10)으로 상위 10개 설비만 선택하여 그래프를 간단하고 명확하게 만든다.
- Plotly의 bar()로 설비별 불량률을 시각화하고, texttemplate로 막대 위에 수치를 표시한다.

4교시 | 실습 3. 날짜별 불량률 변화 분석하기

날짜별로 생산량과 불량 수량을 합산하여, 시간에 따라 불량률이 어떻게 변한는지 분석합니다.

학습 목표

- 날짜별 생산량과 불량 수량 집계하기
- 날짜별 불량률 계산하기
- 불량률 변화를 선그래프로 시각화하고 해석하기

데이터 예시 (일부)

timestamp	생산량	불량 수량
2026-11-01 09:00	100	3
2026-11-01 10:00	120	5
2026-11-02 09:00	150	6
2026-11-02 10:00	130	4
…	…	…

처리 흐름

1. 날짜 추출 (timestamp → date)
2. 날짜별 합계 (생산량, 불량 수량)
3. 불량률 계산
4. 선그래프로 시각화

계산 원리

날짜별 불량률

$$R_d = \frac{D_d}{P_d} \times 100(\%)$$

- P_d : 날짜 d의 총생산량
- D_d : 날짜 d의 총불량 수량
- R_d : 날짜 d의 불량률(%)

계산 예제

11월 1일

생산량 $P_{11/1} = 100 + 120 = 220$

불량 수량 $D_{11/1} = 3 + 5 = 8$

$$R_{11/1} = \frac{8}{220} \times 100 \approx \mathbf{3.64\%}$$

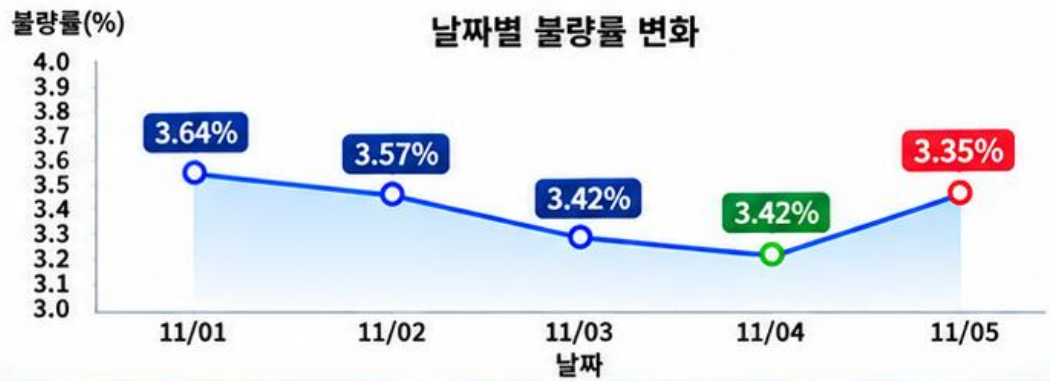
11월 2일

생산량 $P_{11/2} = 150 + 130 = 280$

불량 수량 $D_{11/2} = 6 + 4 = 10$

$$R_{11/2} = \frac{10}{280} \times 100 \approx \mathbf{3.57\%}$$

실행 결과 예시: 날짜별 불량률 변화



Python 코드 (날짜별 불량률 & 선그래프)

```python
# 1. timestamp를 날짜 형식으로 변환하고 날짜만 추출
df["timestamp"] = pd.to_datetime(df["timestamp"])
df["date"] = df["timestamp"].dt.date

# 2. 날짜별 생산량과 불량 수량 합계
daily_quality = df.groupby("date")[
    ["production_count", "defect_count"]].sum()

# 3. 날짜별 불량률 계산 (생산량이 0이면 계산하지 않음)
daily_quality["defect_rate"] = (
    daily_quality["defect_count"] /
    daily_quality["production_count"].replace(0, float("nan")) * 100

# 4. 선그래프로 시각화 (Plotly)
fig = px.line(daily_quality,  reset_index(),
              x="date", y="defect_rate", markers=True,
              title="날짜별 불량률 변화")
st.plotly_chart(fig, use_container_width=True)
```

분석 질문

- **Q1** 불량률이 가장 높은 날짜는? → **11월 5일 (3.53%)**
- **Q2** 불량률이 가장 낮은 날짜는? → **11월 4일 (3.35%)**
- **Q3** 날짜에 따라 불량률이 일정한가? → **아니요 (조금씩 변동)**
- **Q4** 특정 날짜의 불량률 증가 이유를 데이터만으로 확정할 수 있는가? → **아니요 (추가 원인 분석 필요)**

핵심 정리

- 날짜별로 생산량과 불량 수량을 합산한 후 **불량률 = (불량 수량 ÷ 생산량) × 100** 으로 계산한다.
- Plotly의 **line()**으로 날짜별 불량률 변화를 **선그래**프로 **시각화**할 수 있다.
- 그래프의 변화를 통해 품질 추세를 파악할 수 있지만, 원인 분석을 위해서는 온도, 설비 상태 등 추가 데이터가 필요하다.

4교시

실습 4. 품질 경고 기능 구현하기

사용자가 허용 불량률을 설정하고, 기준을 초과하는 설비를 자동으로 경고하는 기능을 구현합니다.

학습 목표

- 허용 불량률 설정하기
- 불량률 기준으로 정상·경고 판정하기
- 경고 설비 개수 확인하기
- 품질 상비 메시지 표시하기

예제 데이터와 결과

설비	불량률	품질 상태 (허용 기준 5%)
M001	3%	정상
M002	5%	정상
M003	8%	경고
M004	2%	정상

불량률이 정확히 5%인 경우는 '정상'으로 처리합니다.

품질 경고의 원리

품질 상태 판정식

$$S(R_m) = \begin{cases} \text{정상}, & R_m \le R_0 \\ \text{경고}, & R_m > R_0 \end{cases}$$

- R_m : 설비 m의 설제 불량률(%)
- R_0 : 허용 불량률(%)

실행 결과 예시

허용 불량률 5.0% (기본값)

허용 불량률(%) : 5.0

0.0% 5.0% 10.0%

전체 불량률	경고 설비 수
3.41%	0대

모든 설비가 허용 기준 이내입니다.

허용 불량률 4.0% (기준 강화)

허용 불량률(%) : 4.0

0.0% 4.0% 10.0%

전체 불량률	경고 설비 수
3.41%	3대

경고 설비가 3대 있습니다.

기준별 경고 설비 수 비교 (실제 데이터)

허용 불량률	5.0%	4.0%	3.5%
경고 설비 수	0대	3대	기준 초과 설비 발생

주요 Python(Streamlit) 코드

1 허용 불량률 설정(슬라이더)

```
threshold = st.slider(
    "허용 불량률(%)", min_value=0.0, max_value=10.0,
    value=5.0, step=0.1)
```

2 품질 상태 판정(정상/경고)

```
machine_quality["quality_status"] = "정상"
machine_quality.loc[
    machine_quality["defect_rate"] > threshold,
    "quality_status"] = "경고"
```

3 경고 설비 개수 및 메시지

```
warning_count = len(machine_quality[
    machine_quality["quality_status"] == "경고"])

st.metric("경고 설비 수", warning_count)

if warning_count > 0:
    st.error(f"경고 설비가 {warning_count}대 있습니다.")
else:
    st.success("모든 설비가 허용 기준 이내입니다.")
```

핵심 정리

- 허용 불량률 R_0를 설정하고, 설제 불량률 R_m과 비교하여 '정상($\le R_0$)' 또는 '경고($> R_0$)'로 판정한다.
- Streamlit의 st.slider()로 기준을 변경하면 품질 상태와 경고 설비 수가 자동으로 갱신된다.
- 이번 데이터의 최대 불량률은 약 4.13%이므로, 기준 5%에서는 경고가 없고, 4%로 낮추면 경고가 발생한다.

4교시

실습 5. 설비별 품질 순위 표시하기

설비별 불량률을 비교하여 품질 순위를 매기고, 생산량·불량 수량·품질 상태를 표로 확인합니다.

학습 목표

- 설비별 불량률로 품질 순위 매기기
- 불량률이 낮은 설비부터 정렬하기
- 생산량·불량 수량·품질 상태 함께 확인하기

예제 데이터

설비	생산량	불량 수량	불량률
M001	100	3	3%
M002	200	10	5%
M003	150	12	8%
M004	250	5	2%

학생 확인 질문

- Q1 품질 순위 1위 설비는?
- Q2 품질 순위가 가장 낮은 설비는? (품질이 가장 나쁜 설비는?)
- Q3 순위는 어떤 기준으로 정해졌나요?

품질 순위 계산 과정

1. **정렬하기** — 불량률을 낮은 순서로 정렬 (ascending=True)
2. **순위 추가하기** — 정렬된 데이터에 순위(1, 2, 3, ...) 부여
3. **결과 표시하기** — 생산량, 불량 수량, 불량률, 품질 상태를 표로 출력

실행 결과 예시 : 설비별 품질 순위표

순위	설비	총생산량	불량 수량	불량률	품질 상태
1	M004	250	5	2.00%	정상
2	M001	100	3	3.00%	정상
3	M002	200	10	5.00%	정상
4	M003	150	12	8.00%	경고

주요 Python 코드

```python
# 1. 불량률 기준으로 오름차순 정렬
quality_rank = machine_quality.sort_values(
    "defect_rate", ascending=True)

# 2. 순위 부여 (1부터 시작)
quality_rank = quality_rank.reset_index()
quality_rank["순위"] = range(1, len(quality_rank) + 1)

# 3. 결과 테이블 출력 (Streamlit)
st.dataframe(quality_rank)
```

코드 설명

코드	의미
.sort_values()	특정 기준(불량률)으로 정렬
ascending=True	작은 값부터 큰 값으로 정렬 (불량률이 낮은 순서)
.reset_index()	인덱스를 일반 컬럼으로 변경
range(1, ...)	순위 번호(1, 2, 3, ...) 생성
st.dataframe()	표 형태로 결과를 화면에 표시

핵심 정리

- 설비별 불량률을 기준으로 낮은 순서로 정렬하여 품질 순위를 매긴다.
- 순위와 함께 **총생산량**, **불량 수량**, **불량률**, **품질 상태(정상/경고)**를 함께 확인한다.
- 동일한 불량률이 있을 경우 기본적으로 표시 순서로 순위를 부여한다.

심화 학습

생산량과 불량률을 함께 고려하기

불량률만 비교하지 않고, 생산량(표본의 크기)도 함께 고려하여 더 신뢰성 있는 품질 평가를 합니다.

핵심 개념

- 불량률은 전체 생산량 대비 불량 수량의 비율이다.
- 생산량이 적으면 소수의 불량도 불량률에 큰 영향을 준다.
- 품질을 정확히 판단하려면 불량률과 생산량을 함께 확인해야 한다.

향상 과제

"생산량이 일정 기준 이상인 설비만 품질 순위에 포함하기"

요구 기능

- 최소 생산량 기준을 설정하고
- 해당 기준 이상인 설비만 필터링하여
- 품질 순위(불량률 낮은 순)를 표시한다.

예제 비교: 같은 불량 수라도 생산량에 따라 불량률이 달라요

설비	생산량 (P)	불량 수량 (D)	불량률 (R)
A	10	1	10%
B	1,000	50	5%

→ A 설비: 불량률 10% (생산량이 적어 변동이 큼)

→ B 설비: 불량률 5% (생산량이 많아 변동이 작음)

불량 1개 증가 시 불량률 변화량(ΔR) 비교

생산량이 적을수록 불량률 변동이 매우 큽니다!

수학적 설명: 불량 1개 증가 시

$$\Delta R = \frac{1}{P} \times 100 (\%p)$$

- 생산량 10개일 때

$$\Delta R = \frac{1}{10} \times 100 = 10\%p$$

- 생산량 1,000개일 때

$$\Delta R = \frac{1}{1000} \times 100 = 0.1\%p$$

실행 결과 예시 : 최소 생산량 필터 적용

최소 생산량(개) : 1,000 → 기준 이상 설비 수 : **97대** (전체 100대 중)

순위	설비	생산량	불량 수량	불량률	품질 상태
1	M054	11,476	311	2.71%	정상
2	M082	11,985	333	2.78%	정상
3	M029	10,381	290	2.79%	정상
	…	…	…	…	…

바이브코딩 프롬프트 예시

"품질 순위표를 개선하고 싶어.
최소 생산량 기준을 설정할 수 있게 하고,
총생산량이 1,000개 이상인 설비만
품질 순위표에 표시해줘.

또한 기준을 만족하는 설비 수를 함께 보여줘."

핵심 정리

- 불량률은 생산량에 따라 크게 달라질 수 있으므로, 반드시 생산량(표본 크기)도 함께 확인한다.
- 생산량이 적은 설비의 높은 불량률은 통계적으로 변동이 크므로 신중하게 해석한다.
- 최소 생산량 기준을 적용하면 더 안정적이고 신뢰성 있는 품질 순위를 만들 수 있다.

5교시

설비 모니터링의 기본 개념 이해하기

공장 설비의 상태를 지속적으로 확인하고, 센서 데이터를 통해 이상 상황을 미리 파악하는 활동입니다.

설비 모니터링이란?

공장에서 사용하는 기계나 생산**설비의 상태를 지속적으로 확인**하고,
수집한 데이터를 분석하여
이상 상황에 대응하는 활동입니다.

설비 모니터링으로 확인할 수 있는 질문

- 지금 설비의 온도와 습도는 정상인가?
- 특정 설비의 상태가 이상하지 않은가?
- 온도가 기준을 초과하면 경고할 수 있는가?
- 설비 상태 변화를 시간에 따라 확인할 수 있는가?

설비에서 발생하는 데이터

구분	데이터	설명
설비정보	설비번호	설비를 구분하는 번호
온도정보	온도	설비 또는 공정에서 측정한 온도
환경정보	습도	주변 환경의 습도
시간정보	측정 시간	데이터가 기록된 시각
가동정보	설비 상태	가동, 정지, 점검 상태
생산정보	생산량	설비가 생산한 제품 수
품질정보	불량 수량	불량으로 판정된 제품 수

설제 데이터(production_data.csv)의 구성

컬럼명	의미	이번 교시 활용
timestamp	데이터 기록 시간	최신 기록 및 시간별 그래프
machine_id	설비번호	설비 선택
product_id	제품번호	(추가 분석)
production_count	생산량	(기준 생산 현황)
defect_count	불량 수량	(기준 품질 관리)
temperature	공정 온도	온도 모니터링
humidity	습도	습도 모니터링
operating_status	설비 가동 상태	설비 상태 표시

데이터 특징

- 설비 수 100대
- 기록 항목 8개 컬럼
- 활용 컬럼 온도, 습도, 측정 시간, 설비 상태

핵심 정리

- 이번 교시에서는 **온도, 습도, 측정 시간, 설비 상태**를 이용하여 설비 모니터링 시스템을 구현합니다.
- 100대의 모든 설비를 한 번에 표시하기보다는, 사용자가 **설비를 선택하여 확인하는 방식** 으로 구성합니다.

5교시

실습 1. 설비별 최근 상태 표시하기

CSV 데이터에서 각 설비의 가장 최근 기록을 찾아 온도, 습도, 가동 상태를 화면 표시합니다.

학습 목표

- 설비별 최신 데이터(Latest Data) 이해
- CSV에서 최근 기록 추출하기
- 설비 선택 메뉴 만들기
- 최근 온도·습도·가동 상태 표시하기

사용할 데이터(컬럼)

컬럼명	역할
machine_id	설비 구분
timestamp	최신 기록 판단
temperature	최근 공정 온도
humidity	최근 습도
operating_status	최근 가동 상태

최신 데이터란?

같은 설비의 여러 기록 중
가장 최근 시간의 데이터를 의미합니다.

timestamp	machine_id	온도(℃)	습도(%)
2026-11-01 09:00	M001	35	42
2026-11-01 10:00	M001	36	43
2026-11-01 11:00	M001	38	44
2026-11-01 12:00	M001	37	42

- 가장 높은 온도: 11:00 (38℃)
- 가장 최근 온도: 12:00 (37℃)

최신 값은
가장 높은 값이 아니라,
가장 최근에 기록된 값입니다.

주요 Python 코드

```
# 1. 시간 형식 변환
df["timestamp"] = pd.to_datetime(df["timestamp"])

# 2. 시간순 정렬
df_sorted = df.sort_values("timestamp")

# 3. 설비별 가장 최근 기록 선택(각 그룹의 마지막 행)
latest_data = df_sorted.gropuby("machine_id").tail(1)

# 4. 설비 선택 매뉴
machines = sorted(df["machine_id"].unique())
selected_machine = st.selectbox("설비 선택", machines)

# 5. 선택한 설비의 최신 기록 표시
latest = latest_data[latest_data["machine_id"] == selected_machine].iloc[0]
```

1. 시간 형식 변환
2. 시간순 정렬
3. 설비별 최신 기록 추출
4. 설비 선택 매뉴
5. 값 표시(온도, 습도 등)

실행 화면 예시 (Streamlit)

설비 모니터링

설비 선택

M003

최근 온도 **43.0℃**

최근 습도 **48.0%**

최근 기록 시간 2026-11-01 12:00

설비 가동 상태 RUNNING (가동 중)

참고

본 화면의 '최근'은 CSV에 저장된
가장 최근 기록을 의미합니다.
(실시간 센서값이 아닙니다.)

핵심 정리

- 최신 데이터(Latest Data)는 설비별로 가장 최근 시간의 기록을 의미한다.
- pd.to_datetime(), sort_values(), groupby(), tail(1)로 각 설비의 최신 기록을 추출할 수 있다.
- 사용자가 설비를 선택하면 해당 설비의 최근 온도, 습도, 기록 시간, 가동 상태를 확인할 수 있다.

5교시 | 설비별 온도·습도 변화 그래프 만들기

선택한 설비의 센서 데이터를 이용하여 시간에 따른 온도와 습도의 변화를 선그래프로 시각화합합니다.

학습 목표

- 설비의 시간별 센서 데이터 확보
- 온도 변화 선그래프 만들기
- 경고 기준선(40°C) 추가하기
- 습도 변화 선그래프 만들기

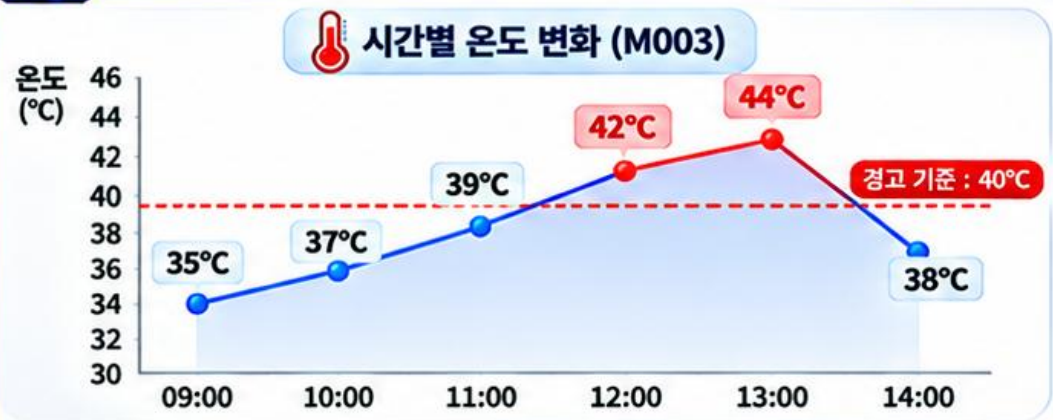
예제 데이터 (M003)

시간	설비	온도(°C)	습도(%)
09:00	M003	35	45
10:00	M003	37	46
11:00	M003	39	47
12:00	M003	42	48
13:00	M003	44	50
14:00	M003	38	47

학생 질문

- Q1 온도가 가장 높은 시간은?
- Q2 온도가 가장 낮은 시간은?
- Q3 40°C를 초과한 시간은?
- Q4 14:00의 온도 상태는?

실행 결과 : 시간별 온도·습도 변화 그래프



분석 결과

- 가장 높은 온도: 13:00 (44°C)
- 가장 낮은 온도: 09:00 (35°C)
- 40°C 초과 시간: 12:00, 13:00
- 14:00 온도 상태: 38°C → 정상

주요 Python 코드 (Plotly)

```python
# 1. 선택한 설비의 데이터 추출 및 시간 정렬
machine_data = df[df["machine_id"] == selected_machine]
machine_data = machine_data.sort_values("timestamp")

# 2. 온도 변화 그래프 생성
fig = px.line(machine_data, x="timestamp",
    y="temperature", title="시간별 온도 변화",
    markers=True)

# 3. 40°C 경고 기준선 추가
fig.add_hline(y=40, line_dash="dash",
    line_color="red", annotation_text="온도 경고 기준: 40°C")

# 4. 습도 변화 그래프 생성
fig_humidity = px.line(machine_data, x="timestamp",
   y="humidity", title="시간별 습도 변화", markers=True)
```

코드 설명

코드	의미
df[df["machine_id"]==...]	선택한 설비의 데이터만 추출
.sort_values("timestamp")	시간 순서로 정렬
px.line()	선그래프(시계열) 생성
markers=True	각 데이터 지점에 점 표시
add_hline(y=40, ...)	경고 기준선(40°C) 추가

핵심 정리

- 시간별 센서 데이터를 선그래프로 그리면 온도와 습도의 변화 추세를 쉽게 파악할 수 있다.
- 그래프에 경고 기준선(40°C)을 추가하면 기준을 초과한 구간을 한눈에 확인할 수 있다.

5교시

실습 3. 온도 이상 경고 시스템 구현하기

설비의 온도가 기준을 초과하면 경고를 표시하고, 기준 이하로 돌아오면 정상으로 표시하는 기능을 구현합니다.

학습 목표

- 온도 기준을 이용한 이상 경고 원리 이해
- 조건문으로 정상·경고 판단하기
- Streamlit으로 경고 메시지 표시하기
- 슬라이더로 경고 기준 변경하기
- 가동 상태와 온도 경고 구분하기

온도 경고의 판단 원리

$$S(T) = \begin{cases} \text{정상}, & T \le T_0 \\ \text{경고}, & T > T_0 \end{cases}$$

T: 설비 온도
T_0: 온도 경고 기준(예: 40℃)

온도 T	조건 (T_0 = 40℃)	판단 결과
35℃	35 ≤ 40	정상
39℃	39 ≤ 40	정상
40℃	40 ≤ 40	정상
41℃	41 > 40	경고
45℃	45 > 40	경고

중요!

온도가 정확히 40℃이면 경고가 발생하지 않습니다.

경고 조건은 '40℃ 이상'이 아니라 '40℃ 초과'입니다.

주요 Python / Streamlit 코드

① 온도 기준 설정(사용자 변경 가능)

```
threshold = st.slider(
    "온도 경고 기준(℃)",
    min_value=30, max_value=50,
    value=40, step=1
)
```

② 온도 상태 판단 및 메시지 표시

```
temperature = latest["temperature"]

if temperature > threshold:
    st.error(f"경고: {selected_machine} 설비의
             온도가 기준을 초과했습니다.")

else:
    st.success(f"{selected_machine} 설비의
             온도가 정상 범위입니다.")
```

온도 경고 기준 설정

온도 경고 기준(℃) 40℃

30℃ ——●—— 50℃

기준 변경에 따른 결과 예시

기준: 40℃	기준: 45℃
현재 온도: 43℃	현재 온도: 43℃
경고	정상

설비 가동 상태와 온도 경고는 다른 정보

설비	가동 상태 (operating_status)	온도	온도 경고 (기준 40℃)
M001	RUNNING (가동 중)	35℃	정상
M002	RUNNING (가동 중)	43℃	경고
M003	STOPPED (정지)	38℃	정상
M004	MAINTENANCE (점검 중)	42℃	경고

- 가동 상태: 설비가 가동 중인지 여부 (RUNNING, STOPPED, MAINTENANCE)
- 온도 경고: 온도가 기준을 초과했는지 여부(정상, 경고)

핵심 정리

- 온도 경고는 $T > T_0$일 때 발생하며, $T = T_0$는 정상입니다.
- st.slider()로 경고 기준을 사용자가 변경할 수 있으며, 기준 변경 시 결과가 즉시 갱신됩니다.
- 설비 가동 상태(가동/정지/점검)와 온도 경고 상태(정상/경고)는 서로 다른 정보로 구분하여 표시합니다.

5교시

실습 4. 가상 센서 데이터 생성하기

버튼을 클릭하면 새로운 온도·습도 값을 생성하여 화면에 즉시 반영하고, 기준을 초과하면 경고 표시합니다.

학습 목표

- 가상 센서의 개념 이해
- 버튼 클릭으로 센서값 생성하기
- 생성된 온도·습도를 화면에 표시하기
- 온도 기준과 비교하여 정상·경고 판단하기
- st.session_state로 값 유지하기

실제 센서와 가상 센서의 차이

구분	실제 센서	가상 센서
데이터 생성	물리적 센서 측정	프로그램에서 생성
온도값	실제 측정 온도	임의 생성값
데이터 수집	센서·통신 장치 활용	Python 코드 활용
활용 목적	실제 설비 모니터링	기능 개발 및 테스트
이번 수업	사용하지 않음	사용 (시뮬레이션)

가상 센서 데이터 생성 원리

Python의 random.uniform()로 범위 내의 무작위 값을 생성합니다.

온도 범위 30 ~ 45°C

습도 범위 35 ~ 60%

```
import random
temperature = random.uniform(30, 45)
humidity = random.uniform(35, 60)
```

random.uniform(a, b)
a와 b 사이의 실수 값을 무작위로 생성하는 함수

버튼으로 센서값 생성하기 (Streamlit)

```
# 버튼을 클릭하면 새로운 센서값 생성
if st.button("재로운 센서 대이터 생성"):
    temperature = random.uniform(30, 45)
    humidity = random.uniform(35, 60)
```

세션 상태로 값 유지하기

```
# 세션 상태 초기화 (최초 실행 시)
if "temperature" not in st.session_state:
    st.session_state.temperature = 35.0
    st.session_state.humidity = 45.0

# 버튼 클릭 시 세션 상태에 새로운 값 지장
if st.button("새로운 센서 대이터 생성"):
    st.session_state.temperature = random.uniform(30, 45)
    st.session_state.humidity = 45.0
```

실행 흐름 (프로그램 동작 과정)

1. **버튼 클릭** (새로운 센서 데이터 생성)
2. **무작위 온도·습도 생성** (온도: 30~45°C 습도: 35~60%)
3. **세션 상태에 저장** (값 유지)
4. **기준 온도와 비교** (예: 40°C)
5. **결과 표시** (정상 또는 경고)

실행 예시

새로운 센서 데이터 생성

생성된 가상 센서 데이터

온도 43.1°C

습도 48.2%

경고: M003 설비의 온도가 기준(40°C)을 초과했습니다.

핵심 정리

- random.uniform(a, b)로 범위 내의 가상 센서값을 생성할 수 있습니다.
- st.session_state를 사용하면 버튼 클릭 후 생성한 값이 화면을 다시 실행해도 유지됩니다.
- 생성된 온도를 기준 온도(T_0)와 비교하여 $T > T_0$이면 '경고', 그 외에는 '정상'으로 표시합니다.

5교시

Streamlit 세션 상태(Session State) 이해하기

버튼을 클릭해 생성한 센서값(온도 · 습도)을 화면에서 계속 유지하는 방법을 학습합니다.

Session State란?

Streamlit에서 사용자가 웹을 사용하는 동안
(같은 세션) 데이터를 저장하고 유지하는 기능입니다.

Session State를 사용하면 기준을 변경해도
생성한 온도(43°C)가 사라지지 않습니다.

일반 변수 vs Session State

구분	일반 변수	Session State
데이터 저장	현재 코드 실행 중만	사용자 세션 동안
화면 재실행	값이 초기화될 수 있음	저장한 값 유지
버튼 클릭 결과	유지 관리가 필요	유지하기 쉬움
데이터베이스 저장	아님	아님

중요한 점

- Session State는 웹브라우저의 사용자 세션 동안만 데이터를 보관합니다.
- 데이터베이스에 저장하는 기능이 아닙니다.
- 브라우저를 닫거나 세션이 종료되면 저장한 데이터가 사라질 수 있습니다.

1 세션 상태 초기화하기

처음 실행할 때 초기값을 설정합니다.

```
# 온도 초기값 설정
if "temperature" not in st.session_state:
    st.session_state.temperature = 35.0

# 습도 초기값 설정
if "humidity" not in st.session_state:
    st.session_state.humidity = 45.0
```

2 버튼 클릭 시 센서값 변경하기

버튼을 클릭하면 새로운 온도와 습도를 생성하여
세션 상태에 저장합니다.

```
import random
# 버튼 클릭 시 무작위 센서값 생성
if st.button("새로운 센서 데이터 생성"):
    st.session_state.temperature = random.uniform(30, 45)
    st.session_state.humidity = random.uniform(35, 60)

# 생성된 값을 화면에 표시
st.metric("가상 센서 온도", f"{st.session_state.temperature:.1f}°C")
st.metric("가상 센서 습도", f"{st.session_state.humidity:.1f}%")
```

3 설비별로 센서값 관리하기

여러 설비의 값을 딕셔너리 형태로 저장하여
설비별로 독립적인 값을 유지합니다.

```
# 설비별 센서값 저장 예시
st.session_state.sensor_data = {
    "M001": {"temperature": 35.0,  "humidity": 42.0},
    "M002": {"temperature": 38.0,  "humidity": 45.0}
}

# 선택한 설비의 센서값만 변경 (예: M001)
st.session_state.sensor_data["M001"]["temperature"] = 43.1
```

학생 확인 활동

버튼을 3번 클릭하고,
온도·습도와 상태가 어떻게 바뀌는지
확인해봅니다.

클릭	온도(예시)	습도(예시)	상태 (기준: 40°C)
1회	35.2°C	42.5%	정상
2회	43.1°C	48.2%	경고
3회	37.6°C	44.8%	정상

핵심 정리

- Session State로 생성한 값을 화면에서 계속 유지할 수 있다.
- 설비별로 값을 따로 저장해 독립적으로 관리할 수 있다.
- 버튼, 슬라이더를 변경해도 이전에 생성한 값이 유지된다.

5교시

실습 5. 경고 발생 및 해제 실험하기

사용자가 온도를 직접 입력하고, 기준을 변경하면서 경고가 정상적으로 발생하고 해제되는지 확인합니다.

학습 목표

- 사용자 온도 직접 입력 기능 구현
- 온도에 따른 정상·경고 판단 확인
- 기준 변경에 따른 상태 변화 확인
- 경고 발생 및 해제 동작 검증
- 센서값과 기준의 관계 이해

사용자 온도 입력 및 판단 코드

```python
# 온도 직접 입력 (Streamlit)
test_temperature = st.number_input(
    "테스트 온도 입력(°C)",
    min_value=0.0, max_value=100.0,
    value=35.0, step=1.0)

# 기준과 비교하여 정상/경고 표시
if test_temperature > threshold:
    st.error("온도 경고")
else:
    st.success("온도 정상")
```

실행 화면 예시 (Streamlit)

온도 경고 기준(°C) 40°C

30°C 50°C

테스트 온도 입력(°C) 41

온도 경고

입력한 온도(41°C)가 기준(40°C)을 초과했습니다.

판단 원리

$$S(T) = \begin{cases} \text{정상}, & T \le T_0 \\ \text{경고}, & T > T_0 \end{cases}$$

- T : 입력한 온도
- T_0 : 온도 경고 기준(예: 40°C)
- $T \le T_0$: 정상
- $T > T_0$: 경고

경고 발생 테스트 (기준: 40°C)

테스트 번호	입력 온도	판단 결과
1	35°C	정상
2	39°C	정상
3	40°C	정상
4	41°C	경고
5	45°C	경고

경고 해제 테스트

실험 1. 온도 변경 (기준: 40°C 유지)

온도: 43°C 경고 → 온도: 37°C 정상

실험 2. 기준 변경 (온도: 43°C 유지)

기준: 40°C, 온도: 43°C 경고 → 기준: 45°C, 온도: 43°C 정상

실험 결과 정리

온도	경고 기준	결과
43°C	40°C	경고
37°C	40°C	정상
43°C	45°C	정상
43°C	42°C	경고

학생 질문

- Q1 40°C는 왜 정상인가요?
- Q2 41°C는 왜 경고인가요?
- Q3 기준을 45°C로 변경하면 왜 정상으로 바뀌나요?

핵심 정리

- 온도 경고는 $T > T_0$일 때 발생하며, $T \le T_0$이면 정상입니다.
- 센서값(온도)이 바뀌어도, 기준(T_0)이 바뀌어도 경고 상태가 달라질 수 있습니다.
- 이 실습을 통해 센서값과 경고 기준이 서로 다른 변수임을 이해할 수 있습니다.

추가 심화 · 선택 실습

센서 모니터링 프로그램에 자동 갱신 기능 넣기

버튼을 누르지 않아도 일정 간격마다 새로운 가상 센서값을 받아 보는 실제 모니터링 방식 흉내 내기

이번 교시 · 기본 실습

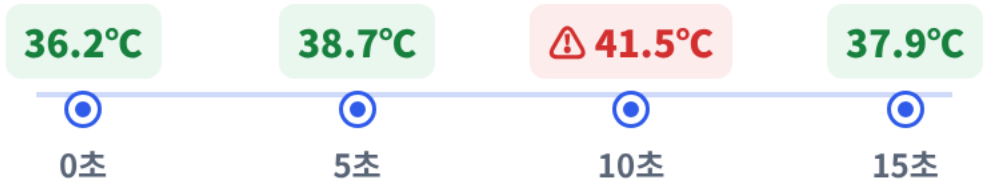
수동 갱신 🔄 새 값 생성 → 클릭할 때마다 센서값 생성

사용자가 **매번 버튼을 클릭**해야 데이터가 바뀝니다.

추가 심화 · 자동 갱신

클릭 없이 자동 생성

자동 갱신을 켜면 **설정한 간격(5초)**마다 새로운 가상 센서 데이터를 생성합니다.



40°C 이상이면 경고 표시

⚙️ 사용자 설정 항목

자동 갱신 기능 사용 여부	켜기 / 끄기
갱신 간격 새 데이터 생성 주기	5초
온도 범위 가상 온도 생성 범위	30 ~ 45°C
습도 범위 가상 습도 생성 범위	35 ~ 60%
경고 기준 이 온도 이상이면 경고	40°C

! 자동 갱신 구현 방식은 **사용 중인 Streamlit 버전에 따라 달라질 수 있습니다.**
처음부터 넣으면 구조가 복잡해지므로, 기본 실습 완성 후 **별도 심화 과제**로 진행하세요.

① 기본 실습 완성 → ② 자동 갱신 심화

데이터 관리 시스템 · 기본 개념

데이터 관리에서 중요한 네 가지 기능

C · R · U · D = Create · Read · Update · Delete

이번 교시

C

생성

Create

새로운 데이터 입력

이번 교시

R

조회

Read

저장된 데이터 확인

DB 연결 후

U

수정

Update

기존 데이터 변경

DB 연결 후

D

삭제

Delete

기존 데이터 제거

✅ 이번 교시에서 구현

Create · 새로운 생산 기록 추가
Read · 기존 생산 기록 조회

⏭ 다음 단계에서 추가

Update와 Delete는
데이터베이스를 연결한 이후 추가

학생들에게 질문

이 작업은 CRUD 중 어떤 기능일까요?

공장에서 일어나는 상황을 보고 알맞은 기능을 떠올려 보세요

Q1 공장에서 새로운 생산 기록을 입력하는 것은?

C Create 생성

Q2 어제 생산한 기록을 확인하는 것은?

R Read 조회

Q3 잘못 입력한 생산량을 수정하는 것은?

U Update 수정

Q4 중복 입력된 기록을 삭제하는 것은?

D Delete 삭제

🎯 이번 수업에서는 새로운 데이터를 입력하는 **Create**와 기존 데이터를 조회하는 **Read**부터 구현합니다.

실습 1 · 전체 생산 데이터 조회하기

CSV 생산 기록을 웹 화면 표로 조회하기

🎯 기존 CSV 파일의 생산 기록을 **Streamlit 웹 화면에서 표로 조회**합니다

📄 CSV 파일 → 🐼 DataFrame → 🖥️ 웹 화면의 데이터 표

📄 production_data.csv **10,000건**

컬럼명 (8개)	의미	표시
timestamp	생산 기록 시간	✔
machine_id	설비번호	✔
product_id	제품번호	✔
production_count	생산량	✔
defect_count	불량 수량	✔
temperature	공정 온도	✔
humidity	습도	✔
operating_status	설비 가동 상태	✔

데이터 관리 메뉴에서 **전체 기록**을 확인할 수 있도록 구현합니다.

1 pandas로 CSV 읽기

```
import pandas as pd
df = pd.read_csv("production_data.csv")
```

`len(df)` → **10000 건 (행)**

`len(df.columns)` → **8 개 (열)**

df = CSV 데이터를 저장한 **DataFrame**

2 Streamlit에서 표 표시

```
st.header("데이터 관리")
st.subheader("전체 생산 데이터")
st.write("전체 데이터 개수:", len(df))
st.dataframe(df)
```

`header` 메뉴 제목
`subheader` 세부 제목
`write` 결과 출력
`dataframe` 표 형태로 표시

⚠️ 화면에서 표를 **정렬**하는 것 ≠ 원본 데이터를 실제로 **수정**하는 것

3 데이터 개수 ≠ 전체 생산량

혼동 주의

데이터 개수
CSV에 저장된 기록 수

10,000 건

`len(df)`

≠

전체 생산량
각 기록의 생산 수량 합계

1,118,576 개

`df["production_count"].sum()`

4 학생 확인 질문

Q1 CSV에 10,000개의 행이 있으면 제품을 10,000개 생산했다는 뜻인가요?

아닙니다. 하나의 기록에 생산량 **100개**가 저장되어 있을 수도 있습니다.

Q2 전체 생산량은 어떻게 계산할까요?

`production_count` 컬럼의 값을 **모두 합산**합니다.

💡 지난 시간엔 **계산 결과**만 → 이번엔 계산에 쓴 **원본 데이터 자체**를 표시 · 개수와 생산량의 차이를 이해하는 것이 **데이터 관리의 기초**

실습 2 · 데이터 검색 및 정렬 기능 만들기

원하는 생산 기록만 찾고, 원하는 순서로 보기

🎯 학습 목표 · 전체 생산 기록 중 **사용자가 원하는 데이터만** 검색하도록 만듭니다

🔍 필터링 Filtering

조건에 맞는 데이터만 **선택**

M001 100	M001 100
M002 120	M001 150
M001 150 →	
M003 90	
M002 130	

예: M001 설비만 조회

개수 줄어들 수 있음

1 설비번호 검색

```
selected_machine = st.selectbox(
    "설비번호 선택", machines)
if selected_machine != "전체":
    filtered = filtered[
        filtered["machine_id"] == selected_machine]
```

'전체' 선택 시 모든 기록 · **원본 CSV는 변경되지 않음**

2 날짜별 검색

```
df["timestamp"] = pd.to_datetime(
    df["timestamp"])
start_date = st.date_input("시작 날짜", ...)
end_date = st.date_input("종료 날짜", ...)
filtered = filtered[
    (filtered["timestamp"].dt.date >= start_date) &
    (filtered["timestamp"].dt.date <= end_date)]
```

시작 > 종료이면 **st.error()** + **st.stop()**

↕️ 정렬 Sorting

데이터의 **순서를 변경**

M001 100	M001 150
M002 120	M002 130
M001 150 →	M002 120
M003 90	M001 100
M002 130	M003 90

예: 생산량 높은 순 개수 유지

sort_values()

3 생산량 정렬

```
sort_order = st.radio("정렬 방식",
    ["생산량 높은 순", "생산량 낮은 순"])
filtered = filtered.sort_values(
    "production_count",
    ascending=False)
```

높은 순 **ascending=False** · 낮은 순 **True**

4 검색 결과 표시

```
st.write("검색 결과:", len(filtered), "건")
if filtered.empty:
    st.warning("검색 조건에 해당하는 데이터가 없습니다.")
else:
    st.dataframe(filtered)
```

결과가 없는 것은 **오류가 아님** → 안내 메시지로 처리

💡 **필터링**은 데이터를 **선택**하는 과정, **정렬**은 데이터의 **순서를 바꾸는** 과정 · 데이터가 **없는 상황**도 프로그램에서 처리합니다.

실습 3 · 새로운 생산 데이터 입력하기

새로운 생산 기록을 직접 입력하는 화면 만들기

🎯 기존: CSV에 저장된 데이터를 **읽기** → 이번: 사용자가 새 생산 기록을 **직접 입력**
실제 공장에서는 새로운 생산 기록이 계속 발생하므로 입력 화면이 필요합니다

👷 공장 관리자의 요청

"오늘 생산한 제품 정보를 **프로그램에 직접 입력**하고 싶습니다."

"생산량과 불량 수량을 입력하면 **자동으로 저장**되면 좋겠습니다."

"입력한 데이터도 **생산 현황에 반영**되도록 만들어 주세요."

📋 입력 항목 = 기존 CSV와 같은 8개 컬럼

입력 항목	CSV 컬럼	예시
생산 날짜·시간	timestamp	2026-11-07 10:00
설비번호	machine_id	M001
제품번호 추가	product_id	P001
생산량	production_count	100
불량 수량	defect_count	5
공정 온도	temperature	38.5
습도	humidity	45
설비 가동 상태 추가	operating_status	RUNNING

같은 8개 컬럼 구조 → 이후 데이터 통합·분석이 쉬움

❓ 학생 확인 질문

- **Q1** 생산량을 -100개로 입력할 수 있나요? — 제한해야 함
- **Q2** 불량 수량을 -5개로 입력할 수 있나요? — 제한해야 함
- **Q3** 생산량 100개, 불량 수량 120개? — 불량 ≤ 생산량

Q3 조건은 다음 실습에서 검증합니다

1 입력 화면 구성 · st.form()

`date_input()` 날짜 입력	`time_input()` 시간 입력
`selectbox()` 목록에서 선택	`number_input()` 숫자 입력
`form()` 여러 입력을 하나의 폼으로	`form_submit_button()` 제출 버튼

st.form()으로 묶으면 저장 버튼을 클릭할 때 **한 번에 처리**

2 날짜 + 시간 → timestamp

```
production_date = st.date_input(
    "생산 날짜", value=datetime.now().date())
production_time = st.time_input("생산 시간", value=...)
timestamp = datetime.combine(
    production_date, production_time)
```

📅 2026-11-07 + 🕙 10:00 → timestamp 2026-11-07 10:00

날짜와 시간을 합쳐 기존 CSV의 **timestamp**와 같은 형태로

3 설비·제품·가동 상태는 목록에서 선택

```
machine_id = st.selectbox("설비번호",
    sorted(df["machine_id"].unique()))
product_id = st.selectbox("제품번호",
    sorted(df["product_id"].unique()))
operating_status = st.selectbox("설비 가동 상태",
    ["RUNNING", "STOPPED", "MAINTENANCE"])
```

RUNNING	STOPPED	MAINTENANCE
가동 중	정지	점검 중

직접 입력하면 M001 → MOO1 오타 위험 → 등록된 값 중 선택

4 숫자 입력 · number_input()

```
production_count = st.number_input("생산량",
    min_value=0, value=100, step=1)
defect_count = st.number_input("불량 수량",
    min_value=0, value=0, step=1)
temperature = st.number_input("공정 온도(℃)",
    value=38.5, step=0.1)
humidity = st.number_input("습도(%)", min_value=0.0,
    max_value=100.0, value=45.0, step=0.1)
```

수량: **정수, 음수 불가(min_value=0)** · 온도·습도: 소수, 습도 0~100%

실습 4 · 생산 데이터 입력값 검증하기

저장하기 전에 입력값이 올바른지 검증하기

🎯 입력값 검증(Validation) = 입력한 데이터가 **정해진 조건을 만족하는지 확인**하는 과정 · 잘못된 값이 저장되면 **생산량·불량률 계산이 틀어집니다**

📋 검증 조건과 잘못된 입력 예시

항목	검증 조건	잘못된 입력 예시	이번
생산량	0 이상	-100	①
불량 수량	0 이상	-5	②
불량 수량	생산량 이하	생산량보다 큰 값	③
온도	숫자	숫자가 아닌 문자	
습도	숫자, 0~100%	120%	
설비번호	등록된 설비	등록되지 않은 번호	
제품번호	등록된 제품	—	
가동 상태	지정된 상태 중 하나	—	

이번 실습: 가장 중요한 ①②③ 세 가지 조건부터 구현

Σ 수학적으로 표현하기

P 생산량 · D 불량 수량

$$P \geq 0 \qquad D \geq 0 \qquad D \leq P$$

$$\boldsymbol{0 \leq D \leq P}$$

불량 수량은 0 이상이고 전체 생산량 이하

🧪 예제

생산량 P	불량 D	결과
100	5	정상
100	0	정상
100	100	정상
100	120	오류
0	0	정상
0	5	오류

Q1 **P=100, D=100이면 잘못된 데이터?**
반드시 그렇지 않음 · **모두 불량**일 수 있음

Q2 **P=0, D=0이면 잘못된 데이터?**
이번 규칙에선 **허용** · 단, 불량률은 계산 불가 → 별도 처리

🐍 Python 조건문으로 검증하기

```
if production_count < 0:
    st.error("생산량은 0 이상이어야 합니다.")
elif defect_count < 0:
    st.error("불량 수량은 0 이상이어야 합니다.")
elif defect_count > production_count:
    st.error("불량 수량은 생산량보다 클 수 없습니다.")
else:
    st.success("입력값이 정상입니다.")
```

`if`	`elif`	`else`	`st.error()`	`st.success()`
첫 조건	다음 조건	모두 통과	오류	정상

🌡️ 앞서 배운 **온도 경고**: 기준 초과 여부 확인 ↔ 이번: 생산량·불량 수량이 **허용 범위**인지 확인 → 둘 다 **조건문** 활용

💾 저장 전 검증

입력값
P=100, D=120 →

❌ **검증 실패 → `st.error()` · 저장 안 함**

✅ **검증 통과 → `else` 안에서만 저장**

```
if defect_count > production_count:
    st.error("불량 수량은 생산량보다 클 수 없습니다.")
else:
    st.success("정상적인 데이터입니다.")
    # 검증을 통과한 경우에만 저장
```

🔒 오류 메시지를 보여주는 것뿐 아니라 **잘못된 데이터를 실제로 저장하지 않는 것**이 중요합니다 · 잘못된 데이터는 세션에 추가하지 않습니다

실습 5 · 새로운 생산 데이터를 임시 저장하기

검증을 통과한 생산 기록을 session_state에 임시 저장하기

🎯 입력값 검증을 통과한 새 생산 기록을 5교시에서 배운 **st.session_state**에 임시 저장합니다

🔁 기존 학습 내용 복습

5교시 · 숫자 하나

```
st.session_state.temperature = 35.0
```

↓ 이번 · 생산 기록 여러 개

그래서 **리스트 [ ]**를 사용합니다

1 저장할 공간 만들기

```
if "new_records" not in st.session_state:
    st.session_state.new_records = []
```

[ ] **new_records** = 새로 입력한 기록을 저장하는 공간 · 처음엔 **빈 리스트**

✍️ 학생 확인 활동

새로운 생산 기록을 **세 번 입력**한 뒤 **len()**으로 개수 확인

```
st.write("새로 입력한 데이터 개수:",
         len(st.session_state.new_records))
```

예상 결과 ▶ 3 건

2 새로운 생산 기록 만들기

딕셔너리(Dictionary) { }

```
new_data = {
    "timestamp": timestamp,
    "machine_id": machine_id,
    "product_id": product_id,
    "production_count": production_count,
    "defect_count": defect_count,
    "temperature": temperature,
    "humidity": humidity,
    "operating_status": operating_status
}
```

입력 예시 → 하나의 생산 기록

"timestamp"	2026-11-07 10:00
"machine_id"	M001
"product_id"	P001
"production_count"	100
"defect_count"	5
"temperature"	38.5℃
"humidity"	45%
"operating_status"	RUNNING

"하나의 생산 기록에는 **여러 정보가 함께** 들어 있습니다."

"생산 시간, 설비번호, 생산량, 불량 수량 등을 **하나로 묶어서** 저장하겠습니다."

3 Session State에 추가 · append( )

```
st.session_state.new_records.append(new_data)
```

append() = 리스트의 **마지막**에 새로운 데이터를 추가

4 입력한 데이터를 표로 표시

```
new_df = pd.DataFrame(st.session_state.new_records)
st.dataframe(new_df)
```

리스트 → **DataFrame**으로 변환 → 직접 입력한 생산 기록을 **표로 확인**

🖥️ 화면 예시

timestamp	machine_id	production_count	defect_count	…
2026-11-07 10:00	M001	100	5	…
…	…	…	…	…

⚠️ 입력한 데이터는 **CSV 파일에 저장된 것이 아니라** 현재 세션 안에 **임시 저장**된 것 · 브라우저 세션 종료나 서버 재시작 시 **사라질 수 있음** → 입력 기능 구현의 **중간 단계**

실습 6·7 · 이번 교시의 가장 중요한 기능

기존 CSV + 신규 데이터 합치기 → 생산 현황 갱신하기

🎯 새로 입력한 데이터까지 포함한 **전체 데이터(all_df)**로 분석하고, 생산 현황 메뉴의 값도 바뀌도록 만듭니다

실습 6 기존 CSV와 신규 데이터 합치기

데이터 처리 구조

기존 CSV **10,000** n_{csv} + 신규 입력 **1** n_{new} → 전체 데이터 **10,001** n_{total}

$n_{total} = n_{CSV} + n_{new}$ · 신규 3건 입력 시 → **10,003건**

pandas에서 합치기 · pd.concat()

```
new_df = pd.DataFrame(st.session_state.new_records)
all_df = pd.concat([df, new_df], ignore_index=True)
```

`pd.DataFrame()` 신규 기록을 표로 | `pd.concat()` 두 DataFrame 연결 | `ignore_index=True` 행 번호 새로 부여

⚠️ **컬럼 이름이 같아야 합니다** · machine_id ↔ machine처럼 다르면 서로 다른 컬럼으로 처리 → 기존 8개 컬럼명 그대로 사용

🔒 df는 **원본 CSV로 유지**하고, 분석용 전체 데이터는 all_df로 따로 만들기

데이터 개수 확인

```
st.write("기존 CSV 데이터:", len(df), "건")
st.write("새로 입력한 데이터:", len(new_df), "건")
st.write("전체 데이터:", len(all_df), "건")
```

기존 CSV	신규 입력	전체 데이터
10,000건	1건	10,001건

실습 7 새로운 데이터로 생산 현황 갱신하기

신규 입력 ▶ 생산량 100 | 불량 수량 5 | 정상 제품 95

전체 생산량 P_{new}
1,118,576 + 100
1,118,676

전체 불량 수량 D_{new}
38,088 + 5
38,093

정상 제품 수 N_{new}
1,080,488 + 95
1,080,583

전체 불량률 R_{new}
38,093 ÷ 1,118,676 × 100
≈ 3.41%

불량률은 소수점 둘째 자리까지 보면 기존과 같은 3.41%로 보일 수 있지만, **실제 계산에는 신규 데이터가 포함**되어 있습니다

기존 프로그램 → 변경할 프로그램

기존 · CSV만 사용

```
total = df["production_count"].sum()
```

변경 · 신규 데이터까지 포함

```
total = all_df["production_count"].sum()
defect = all_df["defect_count"].sum()
normal = total - defect
defect_rate = (defect / total * 100 if total > 0 else 0)
```

❗ 중요한 사항

설비별 그래프·날짜 필터·품질 관리 그래프도 **all_df를 공통 분석 데이터**로 사용하도록 수정

전체 데이터 → 필터 적용 → 생산지표 계산

예: **11월 7일** 데이터를 추가했는데 화면에서 **11월 1일**만 선택했다면 신규 데이터는 계산에서 제외 → **정상적인 동작**

심화 학습 · 기본 기능을 빠르게 완료한 학생용

검색 결과 다운로드 기능 추가하기

🎯 사용자가 검색한 데이터를 **CSV 파일로 다운로드**합니다 · 예: M001 설비를 검색한 뒤 해당 설비의 생산 기록만 저장

💻 코드 · st.download_button() 사용

```
csv_data = filtered.to_csv(
    index=False
).encode("utf-8-sig")

st.download_button(
    label="검색 결과 다운로드",
    data=csv_data,
    file_name="filtered_production_data.csv",
    mime="text/csv"
)
```

🖥️ 화면 예시

📋 코드 설명

코드	의미
`to_csv()`	DataFrame을 CSV 형식으로 변환
`index=False`	행 번호 제외
`encode("utf-8-sig")`	한글 데이터의 **엑셀 호환성** 향상
`st.download_button()`	다운로드 버튼 생성

💡 이 기능을 활용하면 학생들이 **직접 검색한 결과**를 **CSV 파일로 저장**할 수 있습니다.

7교시 · Supabase 데이터베이스 구축

프로그램을 종료해도 데이터가 남도록 만들기

CSV·세션 임시 저장 → **데이터베이스에 영구 저장** · 저장한 데이터를 조회하고 대시보드와 연동합니다

🏁 7교시 결과물

- Supabase 데이터베이스
- 생산 데이터 저장
- 생산 데이터 조회
- 대시보드 연동

🎯 학습 목표

1. 데이터베이스의 필요성 이해
2. 테이블과 컬럼 이해
3. Supabase 프로젝트 생성
4. CSV 데이터 저장
5. Streamlit 데이터베이스 연결
6. 데이터 입력 및 조회

⏱️ 60분 수업 구성

0~10분	CSV와 데이터베이스 차이
10~20분	Supabase 프로젝트 및 테이블 확인
20~30분	CSV 데이터 등록
30~40분	Streamlit과 데이터베이스 연결
40~50분	데이터 저장
50~60분	데이터 조회 및 검증

👩‍🏫 강사는 수업 전에 **Supabase 프로젝트 생성**과 **테이블 구조**를 검증해 두기

🗄️ 테이블 production_data

처음엔 테이블 1개

컬럼	자료형	설명
id	bigint	고유번호
timestamp	timestamptz	기록 시간
machine_id	text	설비번호
production_count	integer	생산량
defect_count	integer	불량 수량
temperature	numeric	온도
humidity	numeric	습도

🔗 CSV ↔ 데이터베이스

CSV의 한 행	=	DB의 한 레코드
CSV의 한 열	=	DB의 한 컬럼

💾 **DB에 저장하면 프로그램을 종료해도 데이터가 남습니다**

1 데이터베이스 테이블 생성

💬 프롬프트 25

production_data 테이블 · 컬럼별 자료형 지정 · SQL Editor용 `CREATE TABLE` 코드 + 초보자용 설명

⚠️ 실행 전 강사가 **컬럼명·자료형·제약조건** 확인

2 기존 데이터 등록

💬 프롬프트 26

CSV → **production_data** 저장 · CSV와 DB **컬럼명 일치** 확인 · **중복 저장** 주의

🔟 처음엔 **10개만** 등록 → 성공하면 더 많이

3 데이터베이스 연결

💬 프롬프트 27

테이블 **조회** 기능 · 연결 정보는 `Streamlit Secrets`로 관리 · 기존 CSV 기능 유지

🔒 비밀 키를 코드·GitHub에 노출 금지 · **모든 사용자가 모든 데이터를 수정**하는 구조는 피하기

4 데이터 저장

💬 프롬프트 28

저장 버튼 → 테이블에 **새 기록 추가** · 성공·실패(원인) **메시지** 표시 · **대시보드**에도 반영

5 데이터 저장 검증

학생 직접 입력

설비 **M001** 생산 **100** 불량 **5** 온도 **38** 습도 **45** → **Supabase에서 등록 확인** → **Streamlit에서 다시 조회**

1 데이터베이스의 정의

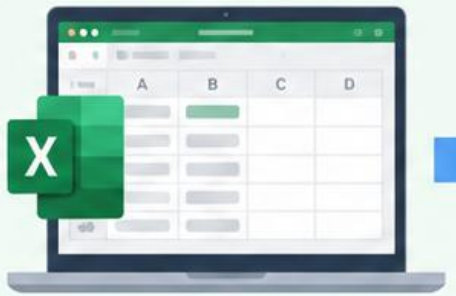
데이터베이스(Database, DB)는 **여러 데이터를 일정한 규칙에 따라 체계적으로 저장하고 관리하는 공간**입니다.

예시: 회사의 공용 기자재 관리

회사가 보유한 기자재는 다음과 같습니다.

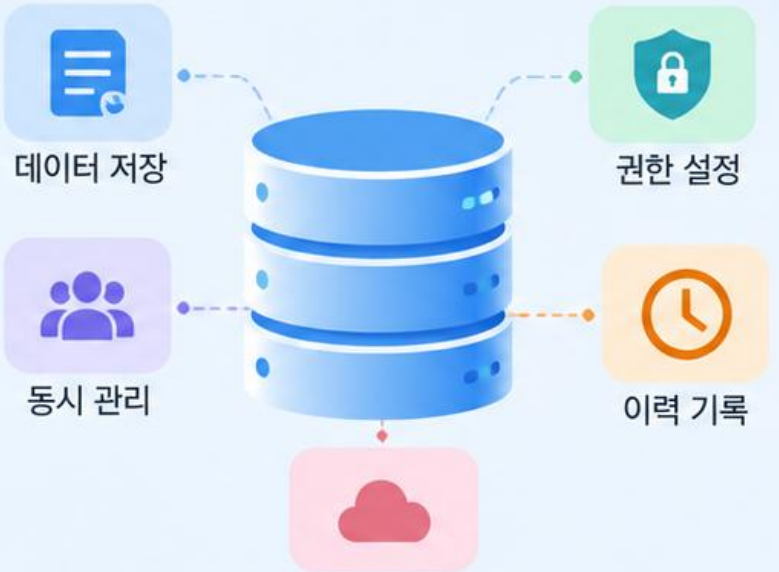
기자재 번호	기자재명	종류	상태
101	보조배터리	IT	사용 가능
102	멀티탭	사무용	대여 중
103	노트북	IT	사용 가능
104	전동드릴	공구	점검 중
105	테스트 스마트폰	IT	대여 중

엑셀로 관리하면...



엑셀 파일로도
관리할 수 있지만...

데이터베이스를 사용하면!



엑셀만으로 관리할 때 발생하는 문제

- 여러 사람이 동시에 데이터를 수정해야 합니다.
- 특정 사용자의 수정 권한을 제한해야 합니다.
- 누가 언제 기자재를 대여했는지 기록해야 합니다.
- 다른 프로그램에서도 동일한 데이터를 조회해야 합니다.
- 데이터가 잘못 수정되거나 삭제되는 것을 방지해야 합니다.

2 데이터베이스와 DBMS의 차이

데이터를 저장하고 활용하는 과정에서 **각각의 역할이** 다릅니다.

데이터베이스는 **'데이터가 저장된 공간'** 이고,
DBMS는 그 데이터를 **관리하는 소프트웨어**입니다.

Application

데이터를 이용하는 프로그램

예시: 기자재 대여·반납 웹앱

사용자가 데이터를 입력하고 조회하는 화면을 제공합니다.

데이터 요청 →
← 결과 전달

Database Service

데이터베이스를 쉽게 구축·운영하도록 제공하는 서비스

Supabase

- 데이터베이스 (PostgreSQL)
- 인증 (Auth)
- 파일 저장 (Storage)
- API (REST)

예시: Supabase

데이터베이스뿐 아니라 인증, 파일 저장, API 등 다양한 기능을 한 번에 제공합니다.

DBMS

데이터베이스를 관리하는 소프트웨어

PostgreSQL

예시: PostgreSQL, MySQL

데이터를 저장·조회·수정·삭제할 수 있도록 관리하는 프로그램입니다.

Database

데이터를 저장하는 공간

기자재 데이터

기자재 번호	기자재명	종류	상태
101	보조배터리	IT	사용 가능
102	멀티탭	사무용	대여 중
103	노트북	IT	사용 가능
104	전동드릴	공구	점검 중
105	테스트 스마트폰	IT	대여 중

주요 DBMS의 예시

PostgreSQL · MySQL · SQLite · Microsoft SQL Server

Supabase는 무엇이 다른가요?

Supabase

- PostgreSQL을 기반으로 제공
- 데이터베이스 + 인증 + 파일 저장 + API 등 다양한 기능 포함
- 별도의 서버 구축 없이 빠르게 시작 가능
- 웹앱, 모바일앱 동과 쉽게 연동 가능

3 엑셀과 데이터베이스는 무엇이 다른가?

엑셀도 데이터를 저장할 수 있지만, **여러 사용자가 동시에 사용하고 관리하는 웹 서비스**를 만들 때는 데이터베이스가 **더 적합**합니다.

비교 항목	Excel (엑셀)	Database (데이터베이스)
주요 목적	데이터 정리·분석	데이터 저장·관리·공유
데이터 구조	시트와 셀	테이블과 관계
여러 사용자 접근	공동 편집 기능 제공	동시 접근과 처리에 특화
데이터 검색	필터, 함수	SQL 및 API
데이터 관계	함수·참조로 구현	PK·FK 동으로 관리
데이터 검증	데이터 유효성 검사	제약조건 및 트랜잭션
사용자 권한	파일·시트 중심	역할·행 단위까지 설정 가능
웹 서비스 연동	추가 연동 필요	API와 직접 연결 가능

기업 사례 동시에 같은 기자재를 대여하려는 경우

엑셀로 관리하는 경우

기록이 늦여 중복 대여가 방정할 수 있습니다.

기록이 늦거나 동시에 수정하면
동일한 기자재가 중복으로 대여될 수 있습니다.

데이터베이스를 사용하는 경우

제약조건과 트랜잭션으로 중복 대여를 방지할 수 있습니다.

중요한 점!

데이터베이스를 사용한다고 자동으로 중복 대여가 방지되는 것은 아닙니다.
중복 대여를 막기 위해서는 **개발자가 올바른 규칙(제약조건, 트랜잭션 등)을 구현**해야 합니다.

4 데이터베이스의 기본 구성요소

관계형 데이터베이스에서는 데이터를 **테이블(Table)** 형태로 저장하며, 테이블은 여러 **행(Row)**과 **열(Column)**로 구성됩니다.

1 테이블 (Table)

같은 종류의 데이터를 저장하는 단위입니다.
예시: 기자재 테이블 (equipment)

equipment

- 엑젤의 시트와 유사하지만,
- 각 열의 자료형과 데이터 제약조건을 정의할 수 있습니다.

equipment 테이블 예시

열(Column) / 행(Row)

id INTEGER	name TEXT	category TEXT	status TEXT
1	보조배터리	IT	사용 가능
2	멀티탭	사무용	대여 중
3	노트북	IT	사용 가능
4	전동드릴	공구	점검 중
5	테스트 스마트폰	IT	대여 중

2 행 (Row)

하나의 데이터 기록을 의미합니다.
행은 **레코드(Record)**라고도 합니다.

id	name	category	status
1	보조배터리	IT	사용 가능

이 한 행은 '1번 보조배터리'에 대한 하나의 데이터입니다.

3 열 (Column)

데이터의 속성을 의미합니다.

Column	의미
id	기자재 고유번호
name	기자재명
category	기자재 종류
status	현재 상태

4 자료형 (Data Type)

테이블의 각 **열**에는 **자료형**이 있으며,
데이터의 종류에 맞게 지정해야 합니다.

자료형	설명	예시
INTEGER	정수	100
NUMERIC	정확한 수치	10.25
TEXT	문자열	보조배터리
BOOLEAN	참·거짓	true
DATE	날짜	2026-10-29
TIMESTAMPTZ	시간대 정보를 포함하는 날짜·시간	대여 시각
UUID	고유 식별자	사용자 ID

꼭 기억하세요!

날짜를 문자열로 저장하는 것과 **날짜** 자료형으로 저장하는 것은 다릅니다. 날짜 자료형을 사용하면 날짜 비교, 기간 계산, 정렬 등을 안정적으로 수행할 수 있습니다.

문자열로 저장한 날짜

"2026-1-5"
"2026-10-29"
"2026-2-3"

올바른 날짜 순서로 정렬되지 않을 수 있습니다.

날짜 자료형으로 저장한 날짜

2026-01-05
2026-02-03
2026-10-29

날짜 비교, 기간 계산, 정렬을 정확하게 할 수 있습니다.

4 데이터베이스에서 가장 중요한 개념: Primary Key

Primary Key는 테이블의 **각 행을 구분하는 고유한 값**입니다.

4.1 Primary Key (PK)

Primary Key는 테이블의 각 행을 구분하는 **고유한 값**입니다.

예시 다음과 같은 기자재 데이터가 있다고 가정해보겠습니다.

id	name
1	보조배터리
2	보조배터리
3	보조배터리

기자재 이름은 모두 같습니다.

그렇다면 톡정 보조배터리를 어떻게 구분할까요?
→ 바로 **id**를 사용합니다.

동일한 데이터도 구분할 수 있어요!

같은 이름의 데이터라도 Primary Key를 사용하면 **각각의 행을 정확하게 구분할 수 있습니다.**

Primary Key의 조건

1. **중복될 수 없습니다.**
   같은 값이 두 번 이상 존재할 수 없습니다.
2. **NULL 값을 가질 수 없습니다.**
   비어 있는 값은 허용되지 않습니다.
3. **각 행을 고유하게 식별할 수 있어야 합니다.**
   테이블의 모든 행이 서로 다른 값을 가져야 합니다.

일반적으로 어떤 값을 사용할까요?

- **숫자** (예: 1, 2, 3, 4 ...)

또는

- **UUID** (예: 550e8400-e29b-41d4-a716-446655440000)

핵심 정리

Primary Key는 단순한 식별자가 아니라, 데이터의 **정확성, 안정성, 그리고 신뢰성**을 보장하는 데이터베이스의 핵심 요소입니다.

5 Foreign Key: 데이터 간 관계

Foreign Key(FK)는 **다른 테이블의 데이터를 참조하는 값**으로, 여러 테이블을 연결하는 역할을 합니다.

1 직원 테이블 (employees)

직원에 대한 기본 정보를 저장합니다.

employee_id PK (직원 고유번호)	name (직원명)	department (부서)
1	김철수	생산팀
2	이영회	구매팀
3	박민수	인사팀
…	…	…

1 : N

한 명의 직원이 여러 번 대여할 수 있습니다.

3 대여 기록 테이블 (rentals)

누가 어떤 기자재를 언제 대여했는지 기록합니다.

rental_id PK (대여번호)	employee_id FK (직원 고유번호)	equipment_id FK (기자재 고유번호)	rental_date (대여일)
1	1	101	2026-10-27
2	2	102	2026-10-28
3	1	103	2026-10-29
…	…	…	…

N : 1

하나의 기자재가 여러 번 대여될 수 있습니다.

2 기자재 테이블 (equipment)

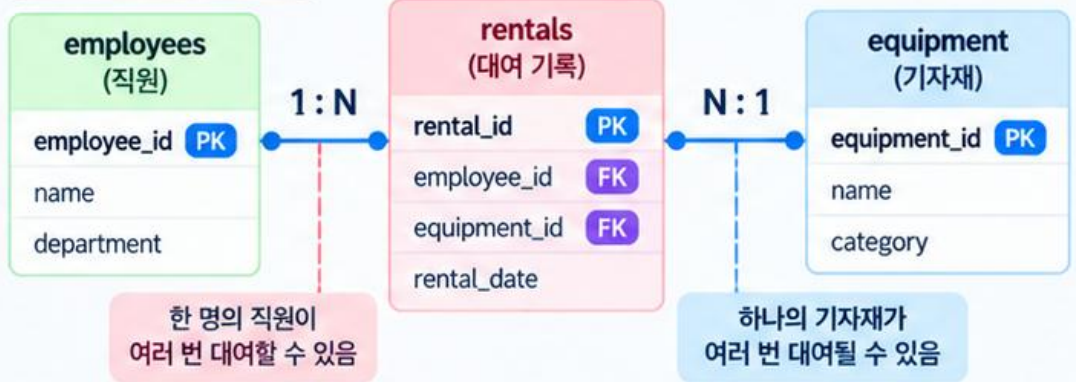
기자재에 대한 기본 정보를 저장합니다.

equipment_id PK (기자재 고유번호)	name (기자재명)	category (종류)
101	보조배터리	IT
102	멀티탭	사무용
103	전동드릴	공구
…	…	…

Foreign Key (FK)란?

- 다른 테이블의 Primary Key를 참조하는 값입니다.
- 테이블 간의 관계를 연결해 줍니다.
- 예) rentals.employee_id → employees.employee_id
- 예) rentals.equipment_id → equipment.equipment_id

테이블 간 관계도



중요한 점!

- FK는 다른 테이블에 실제로 존재하는 값을 참조해야 합니다.
- 존재하지 않는 직원이나 기자재를 대여 기록에 입력할 수 없습니다.
- 이러한 규칙을 통해 **데이터의** 일관성과 정확성을 유지할 수 있습니다.

6 데이터베이스 관계의 종류

관계형 데이터베이스에서는 일반적으로 **세 가지 관계(1:1, 1:N, N:M)**를 사용합니다.

1 일대일 (1:1)

하나의 데이터가 다른 테이블의 하나의 데이터와 연결됩니다.

예시 직원 1명 ↔ 직원별 보안 설정 1개

employees (직원)

employee_id	name	department
1	김철수	생산팀
2	이영희	구매팀
3	박민수	인사팀

security_settings (보안 설정)

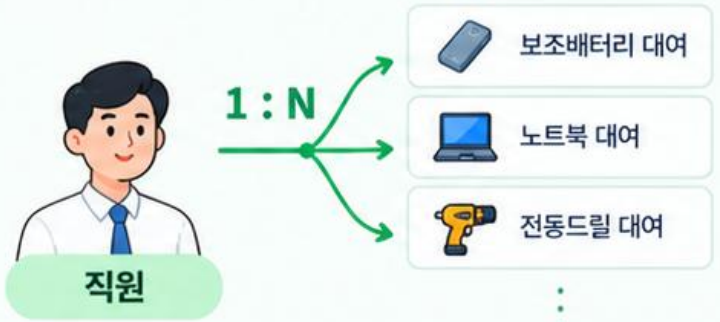
employee_id	role	two_factor
1	일반	사용
2	관리자	사용
3	일반	미사용

직원 1명은 보안 설정 1개만 가집니다.
보안 설정의 employee_id는 employees를 참조하는 Foreign Key입니다.

2 일대다 (1:N)

하나의 데이터가 여러 데이터와 연결됩니다.

예시 직원 1명 → 여러 대여 기록



employees (직원)

employee_id	name	department
1	김철수	생산팀
2	이영회	구매팀
3	박민수	인사팀

rentals (대여 기록)

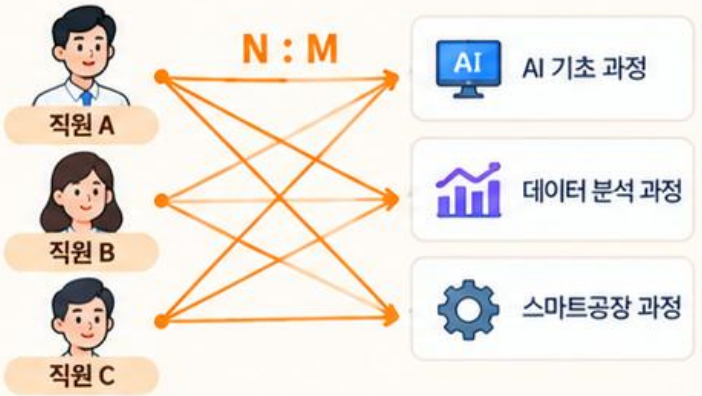
rental_id	employee_id	equipment_id
1	1	101
2	1	102
3	2	103
4	1	104
…	…	…

직원 1명은 여러 번 기자재를 대여할 수 있습니다.
rentals의 employee_id는 employees를 참조하는 Foreign Key입니다.

3 다대다 (N:M)

여러 데이터가 서로 여러 데이터와 연결됩니다.

예시 직원 여러 명 ↔ 교육과정 여러 개



employees (직원)

employee_id	name
1	김철수
2	이영회
3	박민수
…	…

employee_courses (중간 테이블)

employee_id	course_id
1	1
1	2
2	1
3	3
…	…

courses (교육과정)

course_id	title
1	AI 기초
2	데이터 분석
3	스마트공장
…	…

직원 여러 명이 여러 교육과정에 참여할 수 있으므로 중간 **테이블**(employee_courses)을 만들어 대대다 관계를 표현합니다.

7 정규화 (Normalization)는 왜 필요한가?

데이터 중복을 줄이고, 대이터를 효율적이고 일관되게 관리하기 위한 방법입니다.

1 정규화 전 (하나의 테이블에 모든 정보 저장)

직원, 부서, 기자재, 대여 정보를 한 테이블에 저장합니다.

대여번호	직원명	부서	기자재명	대여일
1	김철수	생산팀	보조배터리	10/27
2	김철수	생산팀	멀티탭	10/28
3	김철수	생산팀	노트북	10/29

김철수가 구매팀으로 이동하면 모든 기록의 부서를 수정해야 할까요?

동일한 부서 정보가 여러 행에 중복 저장

- 부서 변경 시 여러 행을 모두 수정해야 함
- 데이터 중복으로 오류 발생 가능
- 데이터 관리가 비효율적

2 정규화 후 (정보를 테이블로 분리하여 저장)

각 정보를 테이블로 나누고, 테이블 간 관계를 연결합니다.

직원 테이블 (employees)

employee_id	name	department
1	김철수	구매팀
2	이영희	생산팀
3	박민수	인사팀
…	…	…

대여 기록 테이블 (rentals)

rental_id	employee_id	equipment_id	rental_date
1	1	101	10/27
2	1	102	10/28
3	1	103	10/29
…	…	…	…

외래키 (employee_id)

외래키 (equipment_id)

기자재 테이블 (equipment)

equipment_id	name	category
101	보조배터리	IT
102	멀티탭	사무용
103	노트북	IT
…	…	…

이제 직원 테이블에서 부서만 한 번 수정하면 모든 대여 기록은 자동으로 연결됩니다!

정규화의 장점

1 데이터 중복 감소
같은 정보를 여러 번 저장하지 않아 데이터 크기가 줄어듭니다.

2 데이터 일관성 유지
한 곳만 수정하면 전체 데이터가 일관되게 관리됩니다.

3 데이터 관리 효율성 향상
오류를 줄이고, 데이터 변경이 쉬워집니다.

! 추가 고려사항

과거 대여 당시의 부서를 기록해야 하는 경우에는 대여 테이블에 당시 부서를 별도로 저장하거나 이력 테이블을 만들어야 합니다.

8 SQL이란 무엇인가?

SQL은 Structured Query Language의 약자로, 관계형 데이터베이스에서 데이터를 조회하거나 수정하는 데 사용하는 언어입니다.
학생들에게는 SQL을 **데이터베이스에 명령하는 언어**라고 설명하면 이해하기 쉽습니다.

SQL의 역할

SQL은
데이터베이스에
명령을 보내서
데이터를 조회, 추가,
수정, 삭제하는 언어입니다.

CRUD (데이터베이스의 기본 기능)

데이터베이스의 기본 기능은 CRUD입니다. 이 네 가지는 번드시 이해해야 합니다.

기능	의미	SQL
Create	데이터 생성	INSERT
Read	데이터 조회	SELECT
Update	데이터 수정	UPDATE
Delete	데이터 삭제	DELETE

예제 테이블: equipment (기자재 테이블)

id	name	category	status
1	보조배터리	IT	사용 가능
2	멀티탭	사무용	대여 중
3	노트북	IT	사용 가능
4	전동드릴	공구	점검 중
5	테스트 스마트폰	IT	대여 중

1 CREATE : 데이터 추가 (INSERT)

새로은 기자재를 추가합니다.

```
INSERT INTO equipment (name, category, status)
VALUES ('보조배터리', 'IT', '사용 가능');
```

↓

id	name	category	status
6	보조배터리	IT	사용 가능

2 READ : 데이터 조회 (SELECT)

모든 기자재를 조회합니다.

```
SELECT *
FROM equipment;
```

↓

id	name	category	status
1	보조배터리	IT	사용 가능
2	멀티탭	사무용	대여 중
3	노트북	IT	사용 가능
...	...	...	...

3 UPDATE : 데이터 수정 (UPDATE)

1번 기자재의 상태를 '대여 중'으로 변경합니다.

```
UPDATE equipment
SET status = '대여 중'
WHERE id = 1;
```

↓

id	name	category	status
1	보조배터리	IT	대여 중

4 DELETE : 데이터 삭제 (DELETE)

1번 기자재를 삭제합니다.

```
DELETE FROM equipment
WHERE id = 1;
```

↓

id	name	category	status
2	멀티탭	사무용	대여 중
3	노트북	IT	사용 가능
...	...	...	...

주의사항

- UPDATE나 DELETE에서 **WHERE 조건을 빠뜨리면** 의도하지 않은 **어러 행이 변경되거나 삭제될** 수 있습니다.
- 실제 대여 기록은 **감사와 이력 관리가 필요하**므로, 물리적으로 삭제하는 대신 **취소 상태를 기록하는** 방식도 고려해야 합니다.

핵심 포인트

- SQL로 데이터를 생성, 조회, 수정, 삭재할 수 있다.
- WHERE 조건을 사용하여 특정 데이터만 대상으로 한다.
- 데이터 삭제는 신중하게 처리하고, 이력 관리 방법을 고려한다.

PostgreSQL

PostgreSQL은 데이터를 체계적으로 저장하고 관리하는
오픈소스 관계형 데이터베이스 관리 시스템(RDBMS)입니다.

SQL이라는 언어를 사용하여 데이터를 저장·조회·수정·삭제할 수 있도록 해주는 소프트웨어입니다.

시스템 구조

예시 기자재 대여·반납 시스템

직원들이 기자재를 대여하면 다음과 같은 정보가 PostgreSQL에 저장됩니다.

저장되는 데이터 예시

equipment (기자재 테이블)

id	기자재명	대여자	상태
1	보조배터리	김철수	대여 중
2	멀티탭	이영회	대여 중
3	노트북	-	사용 가능

주요 특징

- **강력한 안정성** — ACID 준수, 트랜잭션, 무결성 보장
- **확장성** — 대용량 데이터와 복잡한 워크로드 처리
- **풍부한 기능** — JSON, GIS, Full Text Search, Window Function 등 지원
- **보안** — 역할 기반 권한, SSL, 암호화, 감사 기능
- **동시 사용자 지원** — 여러 사용자가 동시에 접근해도 데이터 일관성 유지
- **오픈소스** — 무료로 사용할 수 있으며 다양한 환경에서 활용 가능

데이터 타입 예시

분류	대표 타입
문자열	TEXT, VARCHAR(n), CHAR(n)
숫자	INTEGER, BIGINT, NUMERIC, REAL
날짜/시간	DATE, TIME, TIMESTAMP
불리언	BOOLEAN
JSON	JSON, JSONB
UUID	UUID
배열	ARRAY
기타	BYTEA, INET, MACADDR, XML 등

기본 SQL 예제

```sql
-- 테이블 생성
CREATE TABLE equipment (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  status VARCHAR(20) DEFAULT '사용 가능'
);

-- 데이터 추가
INSERT INTO equipment (name, category, status)
VALUES ('보조배터리', 'IT', '사용 가능');

-- 데이터 조회
SELECT *
FROM equipment;

-- 조건 검색
SELECT *
FROM equipment
WHERE status = '사용 가능';

-- 데이터 수정
UPDATE equipment
SET status = '대여 중'
WHERE id = 1;

-- 데이터 삭제
DELETE FROM equipment
WHERE id = 1;
```

활용 환경 및 확장 기능

- **웹 애플리케이션** — Django, Rails, Node.js, Spring 등
- **데이터 분석 / BI** — Redash, Metabase, Grafana 등
- **GIS / 지도 서비스** — PostGIS 기반 공간 데이터 서비스
- **AI / ML / 벡터 검색** — pgvector 확장으로 벡터 유사도 검색
- **확장 기능 (Extensions)** — PostGIS, pg_trgm, TimescaleDB 등

SQL과 PostgreSQL의 차이

구분	SQL (Structured Query Language)	PostgreSQL (포스트그레스큐엘)
정의	데이터베이스를 조작하는 언어	데이터베이스 관리 소프트웨어
역할	데이터 조회·추가·수정·삭제 명령	SQL 실행 및 데이터 관리
종류	프로그래밍 언어의 일종	관계형 DBMS (Database Management System)
예시	`SELECT, INSERT, UPDATE`	PostgreSQL 서버

동작 예시 다음 SQL을 실행하면, PostgreSQL이 해당 명령을 처리하여 테이블의 데이터를 반환합니다.

1 SQL 작성

사용자가 SQL 명령을 작성합니다.

SQL 명령 전달

```
SELECT *
FROM equipment;
```

2 PostgreSQL이 명령 처리

PostgreSQL이 SQL 명령을 해석하고 데이터를 조회합니다.

PostgreSQL
(데이터베이스 서버)

조회 결과 반환

3 조회 결과 확인

equipment 테이블의 데이터가 화면에 반환됩니다.

equipment 테이블 (조회 결과)

id	기자재명	category	status
1	보조배터리	IT	사용 가능
2	멀티탭	사무용	대여 중
3	노트북	IT	사용 가능
...	...	...	...

SQL은 명령을 작성하는 언어이고, PostgreSQL은 그 명령을 실행하는 시스템입니다.

PostgreSQL의 주요 특징

오픈소스 | 무료로 사용할 수 있는 데이터베이스 소프트웨어

관계형 데이터베이스 | 여러 테이블을 관계로 연결하여 관리

SQL 지원 | 데이터를 조회·추가·수정·삭제할 수 있음

동시 사용자 지원 | 여러 사용자의 데이터 접근과 변경 처리

트랜잭션 지원 | 여러 작업을 안전하게 처리 (일관성 보장)

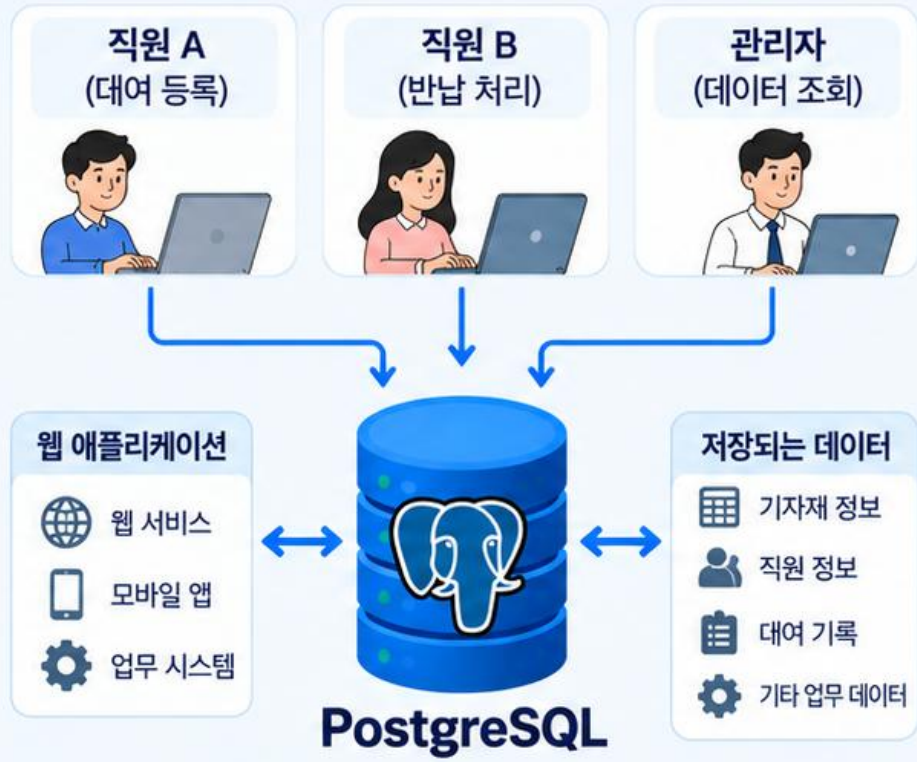
데이터 무결성 | PK, FK, UNIQUE 등 데이터 제약조건 지원

확장성 | 다양한 데이터 유형 및 확장 기능 지원 (PostGIS, JSON, 등)

보안 | 사용자별 권한 및 접근 제어

기업 환경에서의 활용 예시

여러 사용자가 동시에 데이터를 사용하는
웹 서비스를 개발할 때 PostgreSQL이 유용합니다.



PostgreSQL은 다양한 기능과 높은 안정성을 갖춘 오픈소스 관계형 데이터베이스로,
기업에서 **여러 사용자가 동시에 데이터를 사용하는 웹 서비스를 개발할 때 매우 유용**합니다.

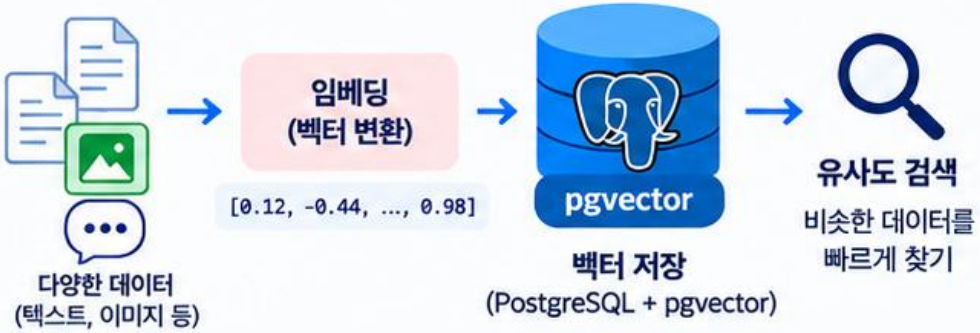
PostgreSQL pgvector 와 AI 기능

PostgreSQL에서 벡터 저장 · 유사도 검색은 물론, AI 기능까지 하나의 데이터베이스에서!

하나의 시스템에서 데이터 + AI 처리 | 강력한 확장성과 신뢰성 | 고성능 벡터 검색으로 빠른 AI 응답 | 추가 시스템 없이 비용 절감

1 pgvector란?

PostgreSQL의 확장 기능으로, 고차원 벡터 데이터를 저장하고 유사도 검색(벡터 검색)을 지원합니다.



2 임베딩 백터 저장과 유사도 검색 예시

임베딩 백터 저장 (예: 이미지 임베딩)

id	내용	embedding (vector)
1	고양이 사진	[0.12, -0.44, ..., 0.98]
2	강아지 사진	[-0.21, 0.31, ..., 0.77]
3	바다 풍경	[0.33, -0.15, ..., 0.66]
...	...	...

유사도 검색 예시 (코사인 유사도)

검색 백터 (고양이 사진)

[0.11, -0.40, ..., 0.97]

유사도 검색

검색 결과 (유사한 순)

순위	내용	유사도
1	고양이 사진	0.98
2	강아지 사진	0.85
3	바다 풍경	0.42
...	...	...

지원 연산자 <-> (L2 거리) <=> (코사인 유사도) <#> (내적) <+> (L1 거리)

3 PostgreSQL에서 바로 사용하는 AI 기능

AI 임베딩 생성

내장 함수 또는 확장을 통해 텍스트를 벡터로 변환

```
SELECT ai.embed('고양이 사진')
    AS embedding;
```

유사도 검색 (RAG, 검색 증강 생성)

pgvector로 유사한 데이터를 검색하여 AI 모델에 컨텍스트로 제공

```
SELECT content
FROM docs
ORDER BY embedding <=> :query
LIMIT 5;
```

AI 요약 / 생성

LLM을 호출하여 요약, 생성, 분류 등 수행 (확장 또는 외부 모델 연동)

```
SELECT ai.generate(
  '다음 문서를 요약해줘: ' || :text
);
```

AI 함수 연동

다양한 AI 모델 및 서비스를 SQL 함수처럼 간편하게 호출

```
SELECT ai.chat_completion(
  '질문: PostgreSQL의 장점은?'
);
```

4 RAG 파이프라인 예시

5 주요 활용 사례

RAG 기반 지식 검색 챗봇 | 문서 / 이미지 유사도 검색 | 개인화 추천 | 이상 탐지 및 유사 패턴 분석 | AI 요약, 분류, 감성 분석 등 | 멀티모달 검색 (텍스트 + 이미지)

6 왜 PostgreSQL + pgvector + AI 인가?

- 데이터 저장, 벡터 검색, AI 처리까지 한 번에
- ACID 보장으로 안전하고 신뢰할 수 있는 AI 서비스 구축
- 확장 가능하고 오픈소스로 비용 효율적
- SQL 기반으로 개발 생산성 향상

7 시작하기 (예시)

```
-- pgvector 설치
CREATE EXTENSION IF NOT EXISTS vector;

-- 벡터 컬럼 사용 예시
CREATE TABLE docs (
  id serial PRIMARY KEY,
  content text,
  embedding vector(1536)   -- 차원 수
);
```

Supabase란 무엇인가?

Supabase는 **PostgreSQL**을 기반으로 **데이터베이스, 인증, API, 파일 저장** 등의 기능을 제공하는 **클라우드 백엔드 개발 플랫폼**입니다.

1 Supabase의 정의

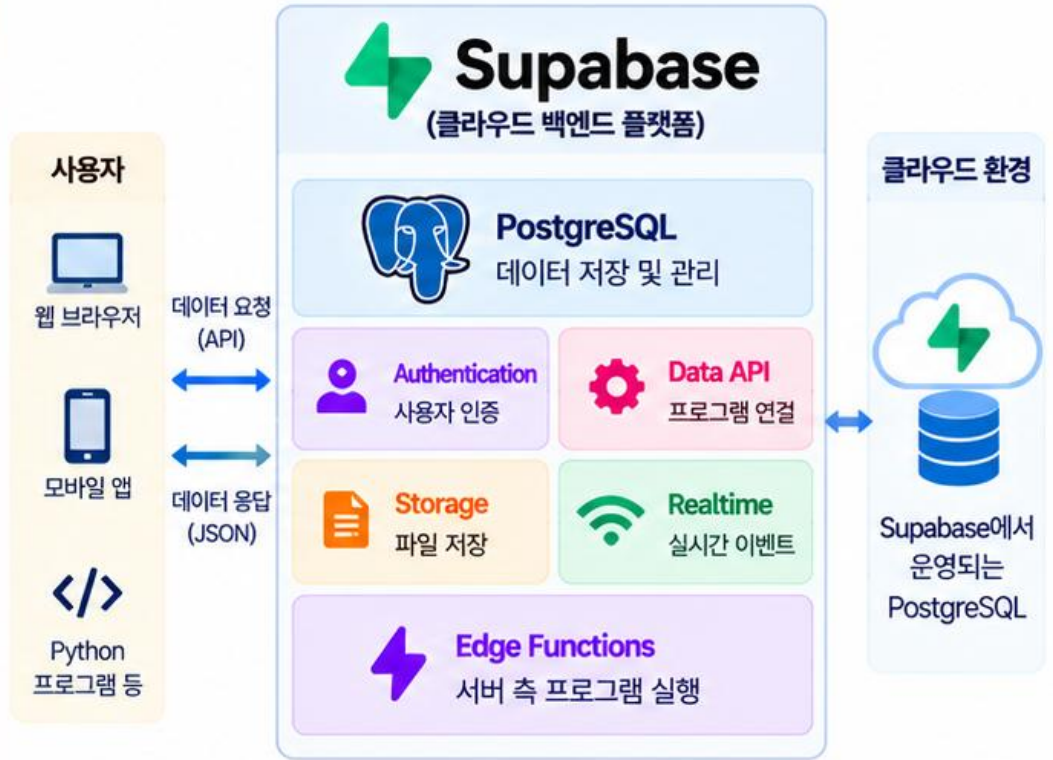
기존 PostgreSQL은 사용자가 직접 컴퓨터에 설치하고 관리했습니다.
반면 Supabase를 사용하면 **클라우드에서 PostgreSQL 데이터베이스를 생성**하고 **웹 브라우저에서 관리**할 수 있습니다.

2 Supabase의 주요 서비스

기능	설명	기업 프로젝트 활용
Database	PostgreSQL 데이터베이스 제공	기자재 · 직원 · 대여 기록 저장
Table Editor	데이터베이스 테이블 관리	기자재 등록 및 데이터 확인
SQL Editor	SQL 명령어 실행	테이블 생성 및 데이터 조회
Data API	데이터베이스 접근 API 제공	Python · 웹앱 연동
Authentication	사용자 인증 및 로그인	직원 로그인
Storage	파일 저장	기자재 사진 및 문서 저장
Realtime	데이터 변경 이벤트 전달	대여 현황 실시간 갱신
Edge Functions	서버 측 프로그램 실행	알림 및 업무 자동화

3 PostgreSQL과 Supabase의 관계

두 기술은 서로 다른 데이터베이스가 아닙니다.
Supabase 내부에서 PostgreSQL을 사용젯합니다.



4 기존 PostgreSQL과 동일한 SQL 사용 가능

Supabase는 PostgreSQL을 사용하므로 지금까지 학습한 SQL 문법을 그대로 사용할 수 있습니다.
예) CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, JOIN, PK, FK 등

```sql
SELECT * FROM equipment;
UPDATE equipment SET status = '대여 중' WHERE id = 1;
INSERT INTO rentals (employee_id, equipment_id) VALUES (1, 3);
DELETE FROM equipment WHERE id = 5;
```

Supabase란? · Streamlit과 역할 나누기

⚡ Supabase의 개념

PostgreSQL 데이터베이스를 중심으로 인증·파일 저장·API 등의 기능을 제공하는 **백엔드 개발 플랫폼**

프로젝트를 생성하면 PostgreSQL DB가 제공되고, 웹 대시보드의 **Table Editor**와 **SQL Editor**에서 테이블과 데이터를 관리합니다.

이번 수업에서 사용할 기능 · 그중 Database 기능 중심

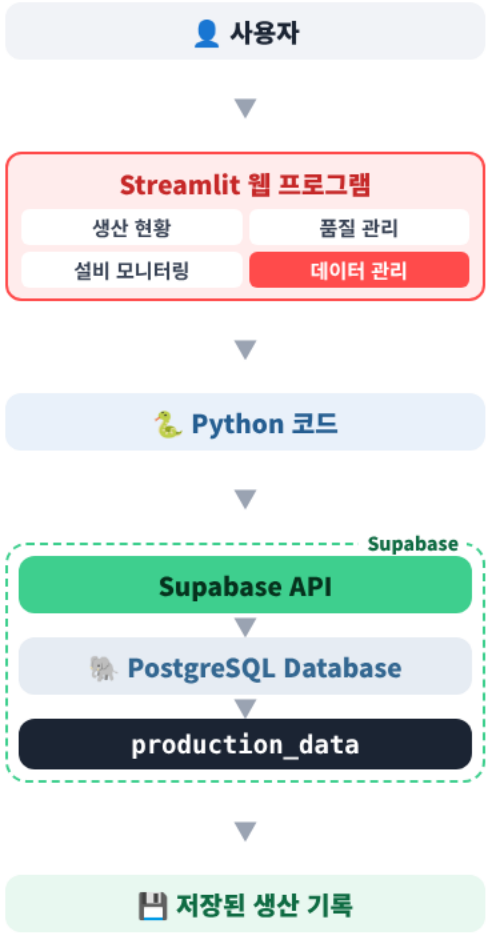
기능	설명	이번 교시
Database	생산 데이터 저장	사용
Table Editor	테이블과 데이터 확인	사용
SQL Editor	SQL 명령 실행	사용
API	Python 프로그램과 데이터 연동	사용
Authentication	사용자 로그인 및 인증	후속 심화
Realtime	데이터 변경 구독	후속 심화
Storage	이미지·파일 저장	사용하지 않음

🔗 공식 사이트 supabase.com · Database 문서 supabase.com/docs/guides/database

🧩 도구별 역할

- 🖥️ **Streamlit** — 사용자에게 웹 화면 제공
- 🐍 **Python** — 입력값 처리 및 데이터 분석
- ⚡ **Supabase** — 데이터 저장 및 조회
- 🐘 **PostgreSQL** — Supabase에서 사용하는 관계형 DB
- 🐙 **GitHub** — 프로그램 코드 관리
- ☁️ **Streamlit Community Cloud** — 웹 프로그램 실행 및 배포

🏗️ 전체 구조



💬 "**Streamlit**은 우리가 보는 **화면**을 만들고, **Supabase**는 **데이터를 저장**합니다."

로컬 PostgreSQL과 Supabase의 차이

Supabase는 클라우드에서 제공되는 PostgreSQL 기반의 백엔드 플랫폼입니다.

1 로컬 PostgreSQL (내 컴퓨터)

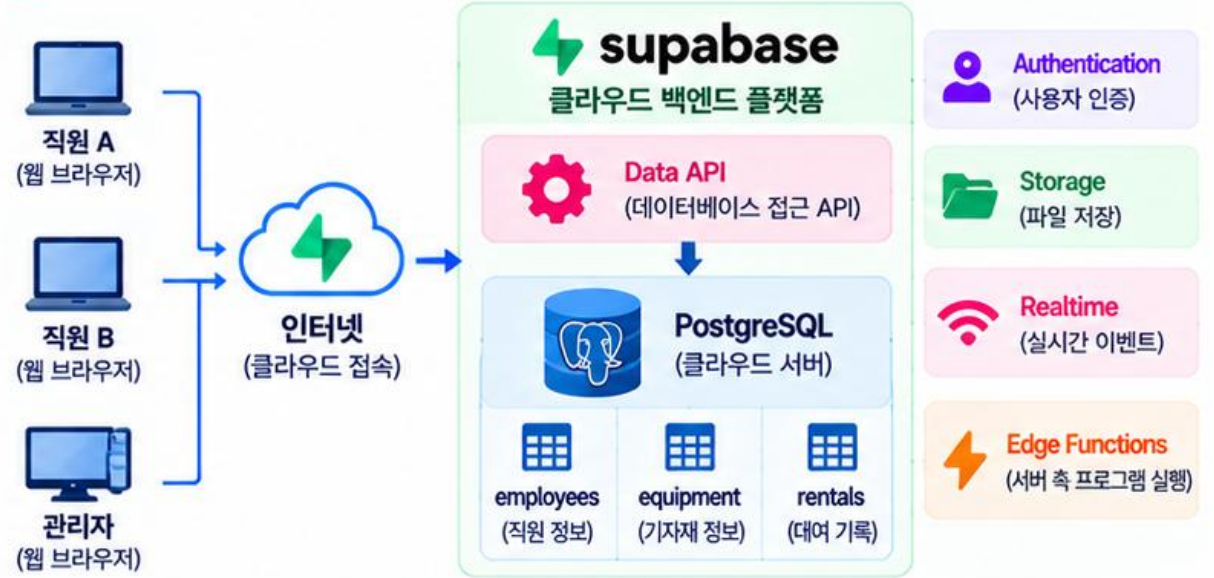
접속 정보 예시

```
host = localhost
port = 5432
dbname = practice_db
```

로컬 환경의 특징

- 자신의 컴퓨터에서 실습하기 편리합니다.
- 인터넷 연결 없이도 사용할 수 있습니다.
- 데이터베이스를 직접 관리할 수 있습니다.
- 다른 컴퓨터에서 접속하려면 네트워크 설정이 필요합니다.

2 Supabase 클라우드 PostgreSQL



클라우드 환경의 특징

- 여러 사용자가 인터넷을 통해 동일한 데이터베이스에 접근할 수 있습니다.
- 별도의 서버 설정 없이 바로 사용할 수 있습니다.
- 사용자 인증과 접근 권한을 설정하여 보안을 강화할 수 있습니다.
- API를 통해 웹앱, 모바일앱, Python 프로그램 등과 쉽게 연동할 수 있습니다.

3 주요 비교

비교 항목	로컬 PostgreSQL	Supabase
설치	직접 설치	클라우드 프로젝트 생성
데이터 저장 위치	로컬 컴퓨터	클라우드 서버
관리 도구	pgAdmin 4	Supabase Dashboard
SQL 실행	Query Tool	SQL Editor
외부 접근	네트워크 설정 필요	API 및 연결 기능 제공
사용자 인증	별도 구현 필요	Supabase Auth 제공
API	별도 개발 가능	자동 생성 Data API
여러 사용자 이용	서버 구성 필요	클라우드 환경에서 이용 가능
비용	로컬 운영 비용	사용량 및 요금제에 따라 달라짐

핵심

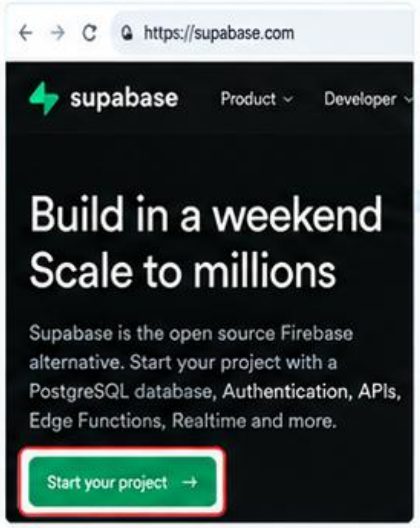
기존 PostgreSQL에서는 데이터베이스를 직접 운영했다면, **Supabase**에서는 **클라우드에서 관리되는 PostgreSQL**을 **활용**할 수 있습니다.

실습 1 — Supabase 프로젝트 생성

Supabase는 PostgreSQL을 기반으로 하는
클라우드 백엔드 플랫폼입니다.
이번 실습에서는 **클라우드 데이터베이스 프로젝트**를 생성합니다.

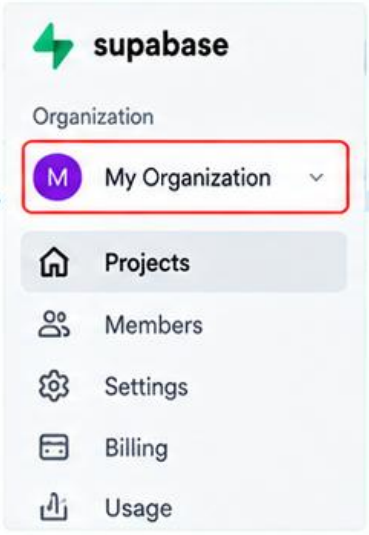
1 Supabase 접속

공식 홈페이지에 접속하여
계정을 생성하기다 로그인합니다.



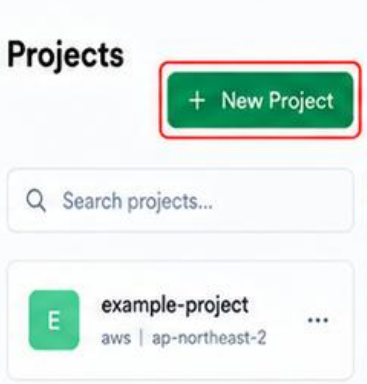
2 Organization 선택

프로젝트를 생성할
Organization을 선택합니다.



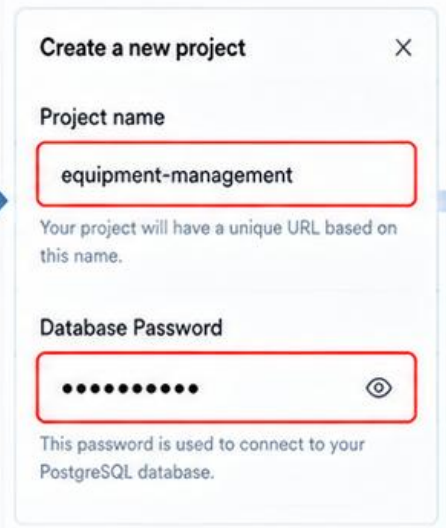
3 New Project 선택

대시보드에서
'New Project' 버튼을 클릭합니다.



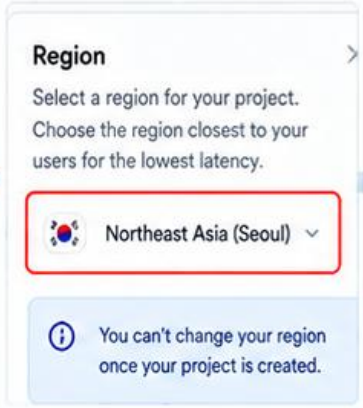
4 프로젝트 정보 입력

프로젝트 이름과
데이터베이스 비밀번호를 입력합니다.



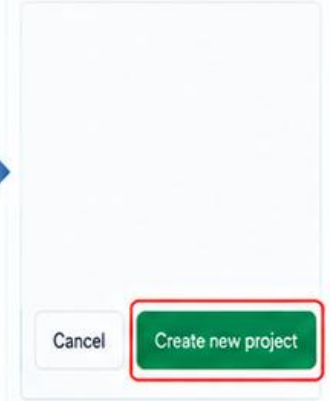
5 Region 선택

한국에서 지연 시간이 적은
지원 지역을 선택합니다.



6 프로젝트 생성

모든 정보를 확인한 후
'Create new project'를
클릭합니다.



프로젝트 설정 예시

항목	실습 설정
Project Name	equipment-management
Database Password	개별 설정
Region	한국에서 지연 시간이 적은 지원 지역
Database	PostgreSQL
용도	기업 공용 기자재 관리

데이터베이스 비밀번호 관리

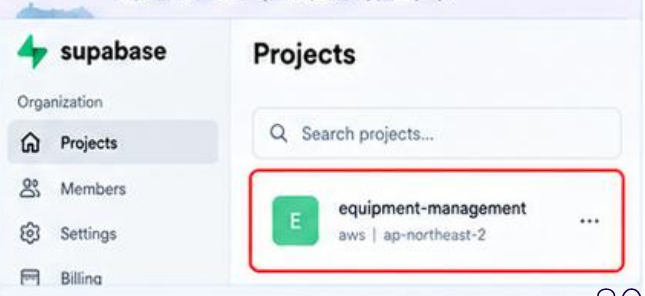
- 데이터베이스 비밀번호는 이후 PostgreSQL 직접 연결에 필요합니다.
- 다른 사람과 공유하지 말고 안전하게 관리합니다.
- 분실한 경우 Supabase에서 비밀번호를 재설정할 수 있습니다.

주의 사항

- 수업에 사용하는 프로젝트에는 기업의 실제 적원 게인정보나 비공개 자료를 입력하리 않습니다.
- 가상 데이터만 사용하여 실습합니다.
- 무료 요금제는 사용량 제한이 있으므로 과도한 데이터 저장은 피합니다.

생성된 프로젝트 확인

프로젝트가 생성되면 대시보드에
새로운 프로젝트가 표시됩니다.



실습 2 — Supabase Dashboard 이해하기

프로젝트를 생성하면 데이터베이스를 관리할 수 있는 Supabase Dashboard가 나타납니다.
이번 실습에서는 주요 메뉴를 살펴보고 사용 방법을 익힙니다.

1 Table Editor
- 테이블과 데이터 관리
- 기자재 목록 확인

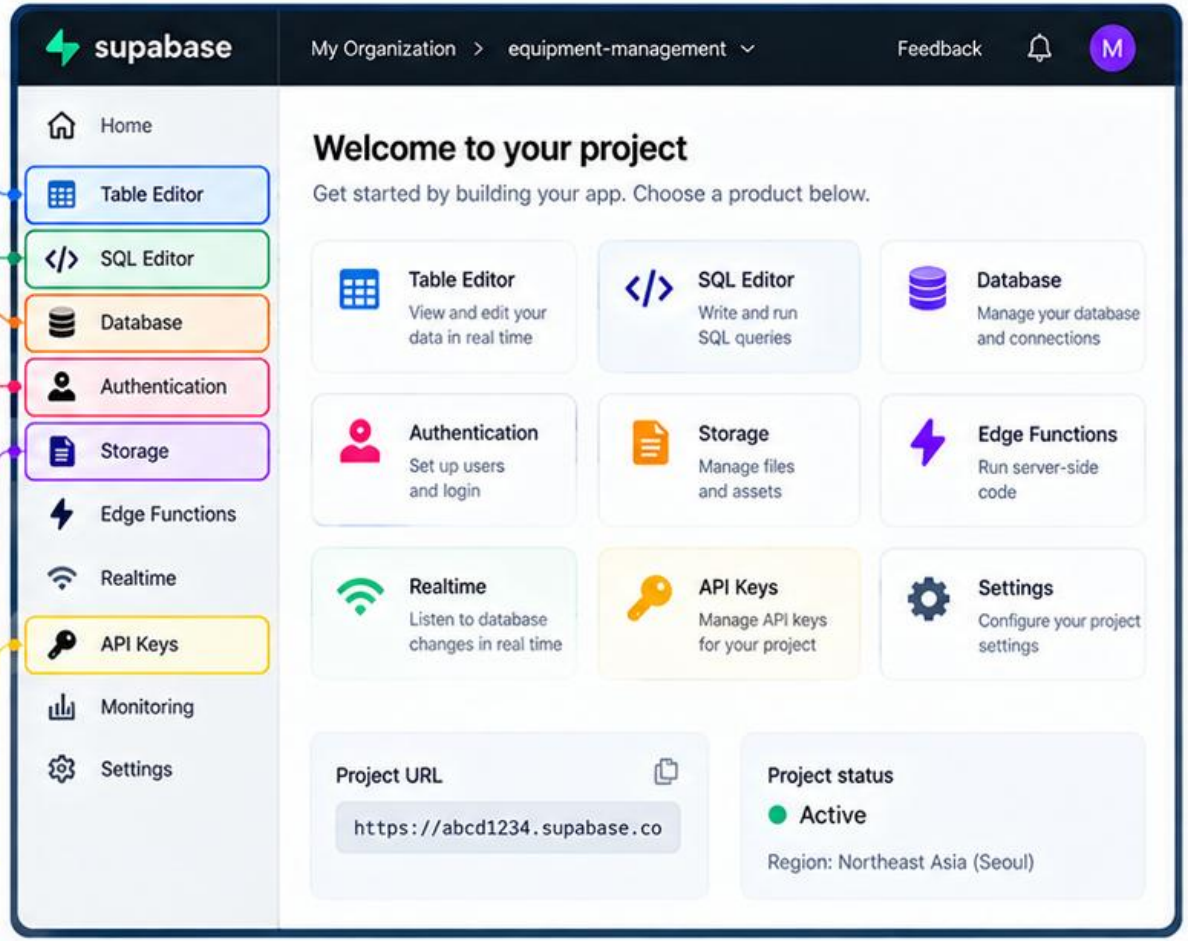
2 SQL Editor
- SQL 명령어 실행
- 테이블 생성 및 데이터 조회

3 Database
- DB 구조와 연결 설정
- PK, FK 및 접속 정보 확인

4 Authentication
- 사용자 인증 및 로그인
- 직원 로그인 기능

5 Storage
- 파일 관리
- 기자재 사진 저장

6 API Keys
- API 키 확인
- Python 연동



테이블 확인 예시 (Table Editor)

schema: public

Search tables...

- employees
- equipment
- rentals

employees

Filter | Sort | + Insert

employee_id (int4)	name (text)	department (text)
1	김철수	생산팀
2	이영희	구매팀
3	박민수	인사팀
4	최지은	개발팀

SQL 실행 예시 (SQL Editor)

```
1  SELECT *
2  FROM employees
3  ORDER BY employee_id;
4
```

Results | Chart | Export

	employee_id	name	department
1	1	김철수	생산팀
2	2	이영희	구매팀
4	3	박민수	인사팀
5	4	최지은	개발팀

pgAdmin 4 와 Supabase 비교

비교 항목	pgAdmin 4	Supabase
SQL 실행 도구	Query Tool	SQL Editor
테이블 관리	Tables	Table Editor
데이터베이스 서버	로컬 또는 원격 PostgreSQL	Supabase PostgreSQL (클라우드)
데이터 조회/수정	가능	가능
접근 방식	직접 연결 (psycopg 등)	Data API 또는 직접 연결
사용 환경	데스크톱 프로그램	웹 브라우저 (Dashboard)

핵심 정리

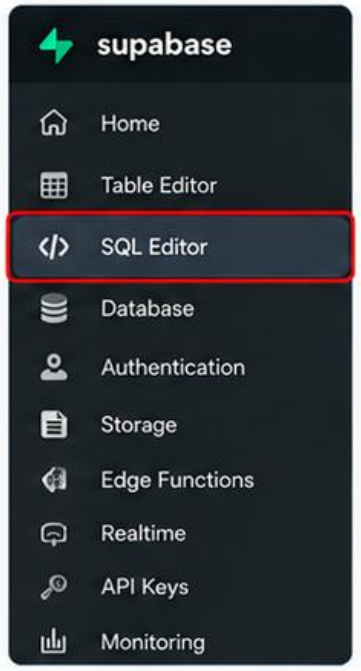
- Supabase는 웹 브라우저에서 PostgreSQL을 편리하게 관리할 수 있는 클라우드 플랫폼입니다.
- Table Editor와 SQL Editor는 pgAdmin 4의 기능과 유사합니다.
- Python에서는 Data API(권장) 또는 psycopg로 직접 연결할 수 있습니다.
- 프로젝트 URL과 API Keys를 활용하여 프로그램에서 쉽게 연동할 수 있습니다.

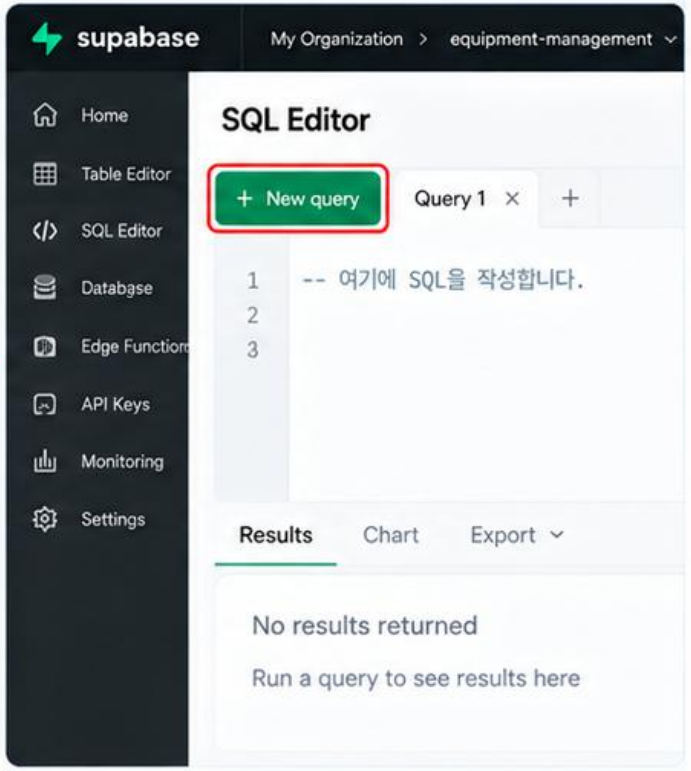
실습 3 — Supabase에서 테이블 생성

기존 PostgreSQL에서 사용했던 **equipment** 테이블을 Supabase의 플라우드 PostgreSQL에 생성합니다.

1 SQL Editor 실행

Supabase Dashboard에서
왼쪽 메뉴의 **SQL Editor**를 선택합니다.





2 테이블 생성 SQL 실행

다음 SQL을 입력하고 Run을 클릭합니다.

Run Save

```sql
CREATE TABLE public.equipment (
    id INTEGER GENERATED ALWAYS AS IDENTITY
     PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT '사용 가능',

    CONSTRAINT valid_equipment_status
       CHECK (
          status IN ('사용 가능', '대여 중', '점검 중')
    )
);
```

실행 결과

Success. No rows returned

3 RLS 활성화

API를 통해 테이블에 접근할 때는
보안 설정이 중요합니다.
다음 SQL을 실행하여 RLS를
활성화합니다.

```sql
ALTER TABLE public.equipment
 ENABLE ROW LEVEL SECURITY;
```

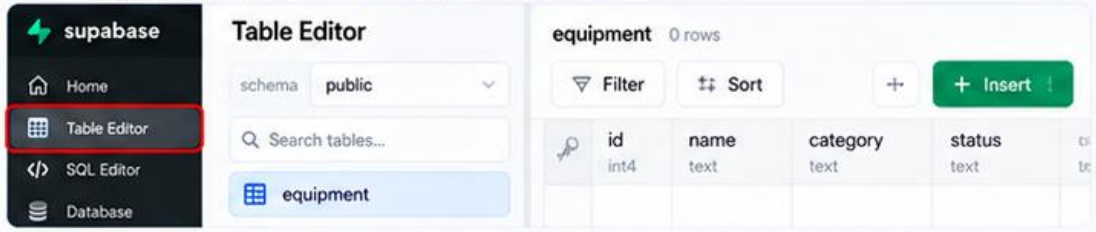
실행 결과

Success. No rows returned

RLS (Row Level Security)
사용자 역할 및 정책에 따라
데이터 접근을 제어하는
PostgreSQL 기능입니다.

4 생성된 테이블 확인

왼쪽 메뉴의 **Table Editor**에서
equipment 테이블이 생성되었는지 확인합니다.



테이블 구조

컬럼	자료형	설명
id	INTEGER	기자재 고유번호 (자동 증가, 기본키)
name	VARCHAR(100)	기자재 이름
category	VARCHAR(50)	기자재 중류
status	VARCHAR(20)	기자재 현재 상태 (기본값: 사용 가능)

기자재 상태 값

status 컬럼은 다음 값만 허용됩니다.

허용 값		
사용 가능	사용 가능	대여할 수 있는 상태
대여 중	대여 중	현재 대여된 상태
점검 중	점검 중	수리 또는 점검이 필요한 상태

실습 5 — Supabase에서 SELECT 실행

기존 PostgreSQL에서 배웠던 SQL을
Supabase SQL Editor에서도 그대로 사용할 수 있습니다.

실습 목표

- Supabase의 SQL Editor에서 다양한 SELECT 문을 실행하고 결과를 확인합니다.
- 기자재 테이터를 조건에 따라 조회하고, 상태별 개수를 집계합니다.

기자재 테이블 (equipment)

id	name	category	status
1	보조배터리	IT	사용 가능
2	멀티탭	사무용	대여 중
3	노트북	IT	사용 가능
4	전동드릴	공구	점검 중
5	테스트 스마트폰	IT	대여 중

핵심 정리

- Supabase에서도 기존 PostgreSQL과 동일한 SELECT 문법을 사용합니다.
- 조회, 조건 검색(WHERE), 집계(GROUP BY)까지 모두 동일하게 동작합니다.

8.1 전체 기자재 조회

등록된 모든 기자재를 조회합니다.

```sql
SELECT *
FROM public.equipment;
```

id	name	category	status
1	보조배터리	IT	사용 가능
2	멀티탭	사무용	대여 중
3	노트북	IT	사용 가능
4	전동드릴	공구	점검 중
5	테스트 스마트폰	IT	대여 중

8.2 사용 가능한 기자재 조회

상태가 '사용 가능'인 기자재만 조회합니다.

```sql
SELECT *
FROM public.equipment
WHERE status = '사용 가능';
```

id	name	category	status
1	보조배터리	IT	사용 가능
3	노트북	IT	사용 가능

8.3 IT 기자재 조회

카테고리가 'IT'인 기자재를 조회합니다.

```sql
SELECT *
FROM public.equipment
WHERE category = 'IT';
```

id	name	category	status
1	보조배터리	IT	사용 가능
3	노트북	IT	사용 가능
5	테스트 스마트폰	IT	대여 중

8.4 기자재 상태별 개수

기자재의 상태별 개수를 집계합니다.

```sql
SELECT
  status,
  COUNT(*) AS total
FROM public.equipment
GROUP BY status;
```

status	total
사용 가능	2
대여 중	2
점검 중	1

실습 6 — Supabase API 이해하기

Supabase Data API를 이용하여
PostgreSQL에 저장된 데이터에 접근할 수 있습니다.

1 Supabase API 주소

Supabase 프로젝트마다 고유한 API 주소가 제공됩니다.
일반적인 형태는 다음과 같습니다.

`https://프로젝트ID.supabase.co`

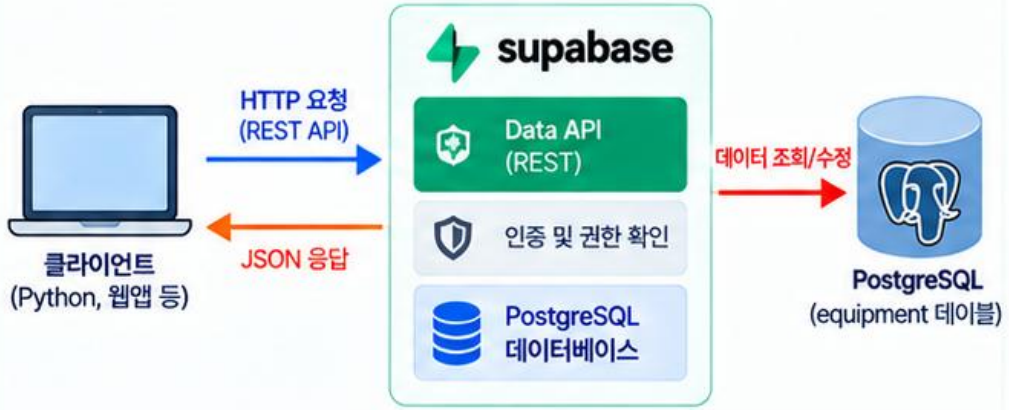
PostgreSQL의 **equipment** 테이블에 접근하는
REST API 경로는 다음과 같은 구조를 가집니다.

`https://프로젝트ID.supabase.co/rest/v1/equipment`

ⓘ 이 주소는 구조를 설명하기 위한 예시입니다.
실제 API 요청에는 유효한 API 키 및 적절한 접근 권한이 필요합니다.

2 Data API 구조와 동작 방식

Supabase에서 제공하는 Data API는 PostgreSQL에 저장된
데이터를 HTTP 요청으로 조회하거나 수정할 수 있는 기능입니다.



3 주요 API 경로 예시

작업	HTTP 메서드	API 경로 예시
데이터 조회	GET	/rest/v1/equipment
데이터 추가	POST	/rest/v1/equipment
데이터 수정	PATCH	/rest/v1/equipment
데이터 삭제	DELETE	/rest/v1/equipment

API 요청 시 다음이 필요합니다.

- 프로젝트의 API 주소
- API 키 (anon key 또는 service_role key)
- 적절한 접근 권한 (RLS 설정)

4 API를 통한 데이터 조회 (전체)

SQL에서 다음과 같이 작성합니다.

PostgreSQL (SQL)

```sql
SELECT *
FROM equipment;
```

Supabase Python

```python
response = (
    supabase.table("equipment")
    .select("*")
    .execute()
)
```

결과 예시 (JSON 형식)

```
{
  "data": [
    {"id": 1, "name": "보조배터리", "category": "IT", "status": "사용 가능"},
    {"id": 2, "name": "멀티탭", "category": "사무용", "status": "대여 중"},
    {...
  ]
}
```

5 조건을 사용한 데이터 조회

상태가 '사용 가능'인 기자재만 조회합니다.

PostgreSQL (SQL)

```sql
SELECT *
FROM equipment
WHERE status = '사용 가능';
```

Supabase Python

```python
response = (
    supabase.table("equipment")
    .select("*")
    .eq("status", "사용 가능")
    .execute()
)
```

결과 예시 (JSON 형식)

```
{
  "data": [
    {"id": 1, "name": "보조배터리", "category": "IT", "status": "사용 가능"},
    {"id": 3, "name": "노트북", "category": "IT", "status": "사용 가능"}
  ]
}
```

6 SQL과 Supabase Python 비교

PostgreSQL SQL	Supabase Python
SELECT	.select()
INSERT	.insert()
UPDATE	.update()
DELETE	.delete()
WHERE	.eq(), .gt() 등
ORDER BY	.ordert()
LIMIT	.limit()

Supabase Python 클라이언트의 명령어는
직접 SQL 문장을 전송하는 것이 아니라
Supabase Data API를 호출하는 방식입니다.

실습 8 — 데이터 조회를 위한 RLS 설정

RLS(Row Level Security)는 "누가, 어떤 데이터에, 어떤 작업을 할 수 있는지"를 제어하는 보안 기능입니다.

RLS란?

Row Level Security (행 수준 보안)

사용자 역할과 정책에 따라 테이블의 각 행(데이터)에 대한 접근을 제어하는 PostgreSQL 기능입니다.

왜 정책이 필요할까?

RLS를 활성화(ENABLE)만 하면 아무도 데이터를 볼 수 없습니다.

누가(역할), 어떤 작업(SELECT 등)을 할 수 있는지 정책(Policy)을 지정해야 합니다.

RLS 동작 흐름

1. RLS 활성화 (ENABLE RLS)
2. 정책 생성 (CREATE POLICY)
3. 권한 부여 (GRANT)
4. 데이터 조회 가능

1 RLS 활성화 (이미 수행)

```sql
ALTER TABLE public.equipment
ENABLE ROW LEVEL SECURITY;
```

2 조회 정책 생성 (실습용)

```sql
CREATE POLICY "equipment_read_demo"
ON public.equipment
FOR SELECT
TO anon
USING (true);
```

3 조회 권한 부여

```sql
GRANT SELECT
ON public.equipment TO anon;
```

결과

Publishable Key (anon)

비로그인 클라이언트(anon)가 기자재 목록 조회 가능

equipment

id	name	category	status
1	보조배터리	IT	사용 가능
2	멀티탭	사무용	대여 중
...	...	...	...

SQL 설명

SQL	의미
CREATE POLICY	접근 정책(Policy) 생성
ON public.equipment	정책을 적용할 테이블 지정
FOR SELECT	허용할 작업 지정 (여기서는 조회)
TO anon	적용 대상 역할 지정 (로그인하지 않은 클라이언트)
USING (true)	조회 조건 (true: 모든 행 조회 허용)
GRANT SELECT	해당 역할(anon)에 테이블 조회 권한 부여

이번 실습의 설정

- 가상의 기자재 데이터만 사용합니다.
- anon 역할에 조회(SELECT)만 허용하는 정책을 설정합니다.
- Publishable Key로 기자재 목록을 조회할 수 있습니다.

주의사항

- 이 정책은 실습용(공개 데이터) 입니다.
- 실제 기업의 기자재 현황은 외부에 공개하지 않아야 합니다.
- 운영 환경에서는 직원 인증(로그인) 후 조회하도록 RLS 정책을 설정하는 것이 일반적입니다.

실습 10 — Python에서 클라우드 데이터 조회

Supabase Python 클라이언트를 사용하여 클라우드(PostgreSQL)의 데이터를 조회합니다.

실습 목표

- Python에서 Supabase API를 사용하여 데이터를 조회하고,
- 조건을 적용한 조회 결과가 Table Editor의 데이터와 동일한지 확인합니다.

사전 준비

- Supabase 프로젝트 연결 (create_client() 완료)
- equipment 테이블에 데이터 5건이 동록된 상태

핵심 개념

- table("...") : 테이블 선택
- select("*") : 조회할 컬럼 지정
- order("컬럼") : 정렬
- eq("컬럼", "값") : 조건 조회
- execute() : API 요청 실행
- response.data : 조회 결과 (파이썬 리스트)

1 전체 기자재 조회

```python
response = (
    supabase.table("equipment")
    .select("*")
    .order("id")
    .execute()
)
print(response.data)
```

실행 결과 (예시)

```
[
    {
        "id": 1,
        "name": "보조배터리",
        "category": "IT",
        "status": "사용 가능"
    },
    {
        "id": 2,
        "name": "멀티탭",
        "category": "사무용",
        "status": "대여 중"
    },
    ...
]
```

코드 설명

코드	의미
.table("equipment")	equipment 테이블 선택
.select("*")	전체 컬럼 조회
.order("id")	ID 순으로 정렬
.execute()	API 요청 실행 (데이터 조회)
response.data	반환된 데이터 (파이썬 리스트 형태)

2 사용 가능한 기자재만 조회

```python
response = (
    supabase.table("equipment")
    .select("*")
    .eq("status", "사용 가능")
    .execute()
)
print(response.data)
```

실행 결과 (예시)

id	name	category	status
1	보조배터리	IT	사용 가능
3	노트북	IT	사용 가능

SQL과 비교

PostgreSQL (SQL)

```
SELECT *
FROM equipment
WHERE status = '사용 가능';
```

⇄

Supabase API (Python)

```
.eq("status",
    "사용 가능")
```

실습 확인

- Supabase Table Editor에서 확인한 데이터와 Python에서 조회한 데이터가 동일한지 확인합니다.
- 조건을 변경하여(category, status 등) 다양하게 조회해 보세요.

실습 12 – 클라우드 데이터 통계 분석

Supabase에서 가져온 데이터를 pandas로 분석하고, 통계와 그래프로 시각화합니다.

실습 목표

- Supabase에서 가져온 데이터를 pandas로 분석하고,
- 기자재 종류와 상태별 개수를 확인하며,
- 그래프로 결과를 시각화합니다.

사전 준비

- 실습 11에서 생성한 DataFrame(df)을 사용 (기자재 5개 데이터)

분석 흐름

1. 데이터 확인 (df.shape)
2. 카테고리별 개수 (value_counts)
3. 상태별 개수 (value_counts)
4. 그래프 시각화 (막대그래프)

1 데이터 크기 확인

```
df.shape
```

→ 결과 (5, 4)

→ 5개의 행(기자재), 4개의 열(id, name, category, status)

2 기자재 종류별 개수

```
df["category"].value_counts()
```

category	개수
IT	3
사무용	1
공구	1

해석

- IT 기자재가 3개로 가장 많습니다.
- 사무용과 공구는 각각 1개입니다.

3 기자재 상태별 개수

```
df["status"].value_counts()
```

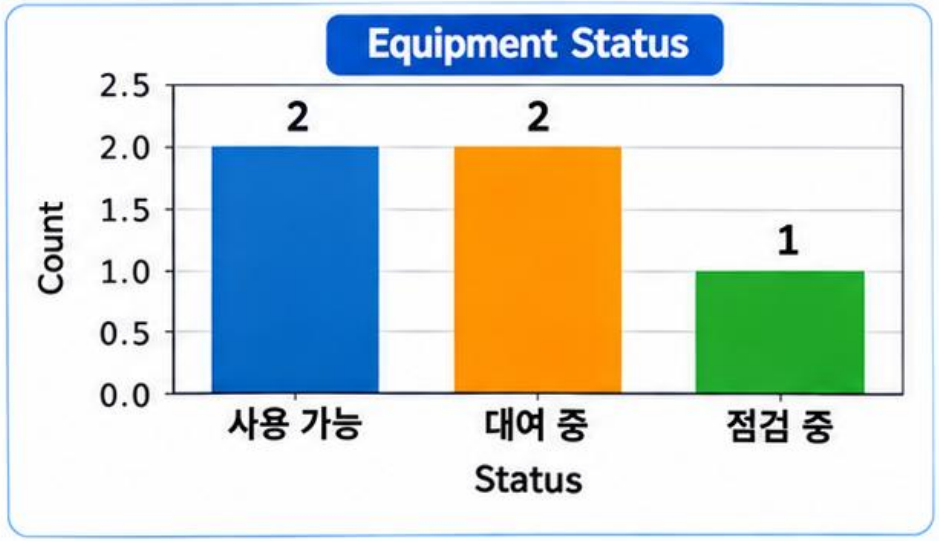
status	개수
사용 가능	2
대여 중	2
점검 중	1

해석

- 사용 가능: 2개
- 대여 중: 2개
- 점검 중: 1개

4 그래프 생성 (막대그래프)

```
1  import matplotlib.pyplot as plt
2
3  status_counts = df["status"].value_counts()
4  status_counts.plot(kind="bar")
5
6  plt.title("Equipment Status", fontsize=14)
7  plt.xlabel("Status")
8  plt.ylabel("Count")
9  plt.xticks(rotation=0)
10 plt.tight_layout()
11 plt.show()
```



핵심 정리

- 기존(local PostgreSQL)과 동일한 pandas 분석 코드를 그대로 사용할 수 있습니다.
- 데이터를 가져오는 위치만 Supabase(클라우드 PostgreSQL)로 바뀌었습니다.

추가 분석 아이디어

카테고리별 상태 비율(파이차트),
기자재 종류와 상태의 교차표(pd.crosstab)도 시도해 보세요.

실습 13 – Supabase에서 데이터 추가·수정·삭제

Supabase API를 사용하여 데이터를 추가(INSERT), 수정(UPDATE), 삭제(DELETE)합니다.

실습 목표

- Supabase API로 데이터를 추가, 수정, 삭제하는 방법을 이해하고,
- SQL과의 대응 관계를 비교하여 CRUD 흐름을 익힙니다.

사전 조건

- INSERT·UPDATE·DELETE 권한이 부여된 역할과 RLS 정책이 필요합니다.
- 현재 조회 전용(SELECT만 허용) 환경에서는 권한 오류가 발생할 수 있습니다.

CRUD 흐름

1. 데이터 추가 (INSERT)
2. 데이터 수정 (UPDATE)
3. 데이터 삭제 (DELETE)

1 데이터 추가 (INSERT)

Supabase Python

```python
response = (
    supabase.table("equipment")
    .insert({
        "name": "웹캠",
        "category": "IT",
        "status": "사용 가능"
    })
    .execute()
)
```

PostgreSQL SQL

```sql
INSERT INTO equipment
(name, category, status)
VALUES
('웹캠', 'IT', '사용 가능');
```

2 데이터 수정 (UPDATE)

Supabase Python

```python
response = (
    supabase.table("equipment")
    .update({"status": "대여 중"})
    .eq("id", 1)
    .execute()
)
```

PostgreSQL SQL

```sql
UPDATE equipment
SET status = '대여 중'
WHERE id = 1;
```

3 데이터 삭제 (DELETE)

Supabase Python

```python
response = (
    supabase.table("equipment")
    .delete()
    .eq("id", 6)
    .execute()
)
```

PostgreSQL SQL

```sql
DELETE FROM equipment
WHERE id = 6;
```

SQL과 Supabase API 비교

기능	PostgreSQL SQL	Supabase Python
조회	SELECT	.select()
추가	INSERT	.insert()
수정	UPDATE	.update()
삭제	DELETE	.delete()
조건	WHERE	.eq(), .gt(), .lt() 등
정렬	ORDER BY	.order()
행 수 제한	LIMIT	.limit()

주의사항

- INSERT·UPDATE·DELETE는 적절한 권한과 RLS 정책이 설정된 환경에서만 실행해야 합니다.
- 상태 변경(UPDATE)은 대여 이력 등 업무 흐름에 맞게 추가 처리(이력 저장)가 필요할 수 있습니다.
- 데이터 삭제(DELETE)는 실제 운영에서 주의가 필요하며, 가급적 테스트 데이터에만 사용하거나 '사용 종지(비활성화)' 방식으로 대체하는 것이 좋습니다.

핵심 정리

- Supabase Python 클라이언트의 insert(), update(), delete()를 사용하면 SQL과 동일한 CRUD 작업을 수행할 수 있습니다.
- 실제 운영에서는 권한 관리(RLS)와 데이터 무결성(이력 관리, 삭제 정책)을 함께 고려해야 합니다.

production_data 테이블 설계 · CSV 8개 컬럼 + id = 9개

기존 production_data.csv의 컬럼 구조를 그대로 데이터베이스 테이블로 옮깁니다

데이터베이스 컬럼 정의

테이블 이름: **production_data**

컬럼	설명	자료형	구분
id	고유번호, 자동 생성	bigint	+ 추가
timestamp	생산 기록 시간	timestamptz	CSV
machine_id	설비번호	text	CSV
product_id	제품번호	text	누락 보완
production_count	생산량	integer	CSV
defect_count	불량 수량	integer	CSV
temperature	공정 온도	numeric	CSV
humidity	습도	numeric	CSV
operating_status	설비 가동 상태	text	누락 보완

⚠ 기존 교육안 테이블에는 **product_id**와 **operating_status**가 빠져 있었습니다 → 이전 교시의 **제품별 분석**과 **설비 가동 상태** 기능을 연동하기 어렵기 때문에 보완합니다

CSV 8개 컬럼 + id = 총 9개 컬럼

자료형 설명

자료형	의미	예시
bigint	큰 정수	100000
integer	정수	100
numeric	소수 포함 숫자	38.5
text	문자	M001
timestamptz	시간대 정보를 반영하는 날짜·시간	2026-11-07 10:00+09:00

시간 데이터 주의사항

CSV 시간값에 **시간대가 없다면** 어느 나라 시간인지 먼저 정의해야 합니다 → 이번 수업은 **한국 표준시(KST, UTC+09:00)**로 가정

CSV 원본
2026-11-01 09:00

→

timestamptz 저장 · +09:00 명시
2026-11-01 09:00+09:00

timestamp without time zone
시간대 변환을 다루지 않는 기초 실습이라면 사용 가능

timestamptz 이번 선택
향후 **서비스 확장**까지 고려하여 사용

7교시 · 실습 2

production_data 테이블 생성하기

🎯 앞서 설계한 **9개 컬럼**으로 실제 데이터베이스 테이블을 생성합니다

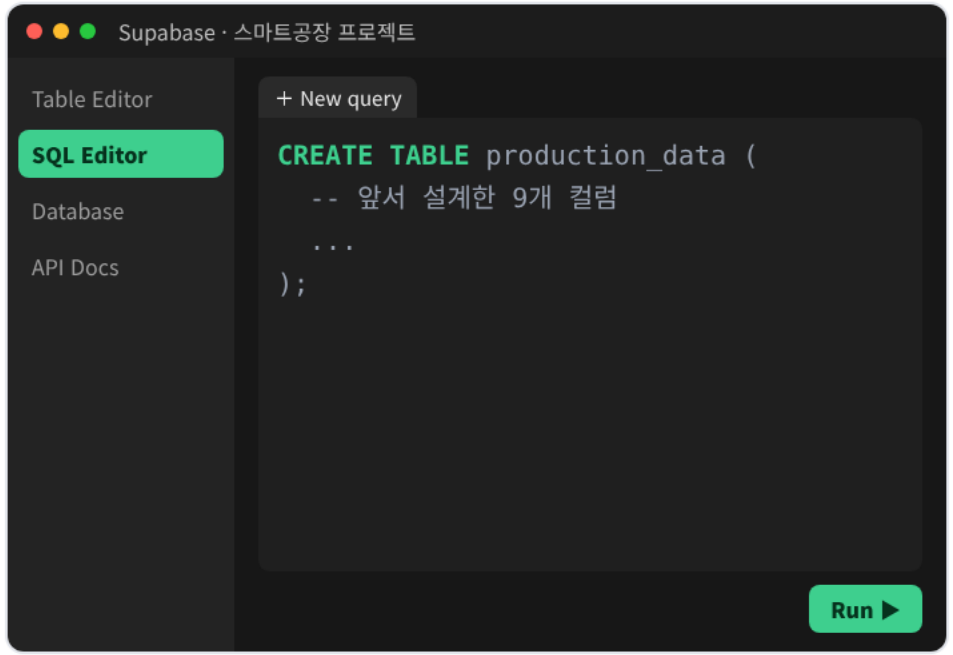
STEP 1 SQL Editor 실행

① **Supabase 프로젝트 열기**

② **SQL Editor 열기**

③ **새 SQL 쿼리 작성 화면으로 이동**

④ **테이블 생성 SQL 실행**



🗂️ SQL이란?

Structured **Q**uery **L**anguage · 관계형 데이터베이스에서 데이터를 **정의**하고 **조회**하거나 **변경**할 때 사용하는 언어

이번 실습에서 사용할 주요 SQL

`CREATE TABLE`	새로운 테이블 생성	`CREATE TABLE production_data (...);`
`SELECT`	데이터 조회	`SELECT * FROM production_data;`
`INSERT`	새로운 데이터 추가	`INSERT INTO production_data ...;`
`COUNT`	데이터 개수 계산	`SELECT COUNT(*) FROM production_data;`
`ORDER BY`	데이터 정렬	`... ORDER BY production_count DESC;`

🤖 **바이브코딩** 원하는 구조를 AI에게 설명 → 생성된 SQL 확인 → 실행 💡 처음에는 모든 SQL을 암기할 필요 없음

7교시 · 실습 3

기존 CSV 데이터를 Supabase에 등록하기

🎯 지금까지 사용하던 production_data.csv를 Supabase에 등록합니다

기존 CSV **10,000건** → 먼저 등록 **10건** 테이블 구조와 저장 과정 검증

🔗 CSV와 데이터베이스 컬럼 비교

컬럼	기존 CSV	DB 데이터 입력 방식
id	✕ 없음	⚙️ 자동 생성
timestamp	✔ 있음	CSV에서 가져오기
machine_id	✔ 있음	CSV에서 가져오기
product_id	✔ 있음	CSV에서 가져오기
production_count	✔ 있음	CSV에서 가져오기
defect_count	✔ 있음	CSV에서 가져오기
temperature	✔ 있음	CSV에서 가져오기
humidity	✔ 있음	CSV에서 가져오기
operating_status	✔ 있음	CSV에서 가져오기

💡 CSV에는 **id가 없어도** 됩니다 → 데이터베이스에서 **고유번호를 자동 생성**하기 때문입니다

STEP 1 처음 10개 데이터 준비하기

Google Colab

1 CSV 읽기

```
import pandas as pd
df = pd.read_csv("production_data.csv")
df.head(10)
```

2 처음 10개 데이터 선택

```
sample = df.head(10).copy()
```

3 한국시간(KST) 시간대 정보 추가

```
sample["timestamp"] = (
    pd.to_datetime(sample["timestamp"])
    .dt.tz_localize("Asia/Seoul")
    .astype(str)
)
```

2026-11-01 09:00 → **2026-11-01 09:00:00+09:00**

4 10개 데이터 CSV 생성

```
sample.to_csv(
    "production_sample.csv",
    index=False
)
```

📄 production_sample.csv · 처음 10개 기록

⚠️ 이번 수업은 CSV 시간을 **한국시간으로 가정**합니다 · 원본 데이터가 **이미 다른 시간대**의 시각이라면 그대로 사용하지 말고 **실제 데이터의 시간 기준을 확인**해야 합니다

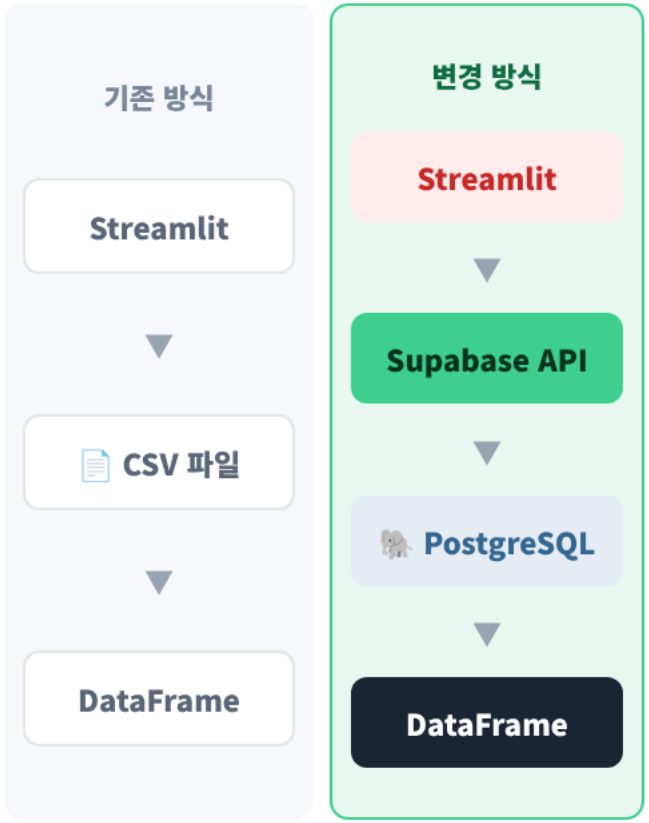
🗣️ "10,000건이 있지만 **우선 10건만 등록**해서 정상적으로 저장되는지 확인하겠습니다. 처음부터 많은 데이터를 등록한 뒤 문제가 생기면 **오류를 찾기 어렵기** 때문입니다."

7교시 · 실습 4

Streamlit과 Supabase 연결하기

🎯 기존 Streamlit 프로그램에서 **Supabase 데이터베이스에 접근**할 수 있도록 만듭니다

🔄 데이터를 가져오는 방식 변경



지금까지 CSV를 읽던 코드

```
df = pd.read_csv("production_data.csv")
```

➡️ 이번에는 **Supabase에 저장된 데이터를 조회**합니다

📦 Supabase Python 라이브러리 설치

Google Colab에서 실행

```
!pip install supabase
```

기존 프로젝트의 requirements.txt에 추가

📄 requirements.txt

```
streamlit
pandas
plotly
supabase ← 추가
```

🗣️ "Supabase라는 데이터베이스 서비스에 **Python으로 접속**하기 위해 필요한 라이브러리를 설치하는 과정입니다."

🔑 Supabase 연결 정보

정보	역할
Project URL	접속할 Supabase 프로젝트
API Key	프로젝트 API 접근에 사용하는 키

API 키 종류	설명	사용 위치
Publishable key	제한된 권한으로 API 접근	클라이언트 또는 서버
Secret key	높은 권한으로 API 접근	신뢰할 수 있는 **서버 전용**

📢 2026년 11월 기준 Supabase는 **Publishable key · Secret key** 체계를 안내 · 기존 anon·service_role 키는 폐지 절차 진행 중

🏫 이번 교육: 강사가 관리하는 **비공개 Streamlit 서버 측 코드**에서 **Secret key** 사용 (복잡한 사용자 인증은 구현하지 않음)

⚠️ Secret key는 **RLS를 우회**할 수 있는 높은 권한 → 공개 앱이라면 **서버 내부에만 보관**하고 **앱 접근 제한** 필요 · 학생별 프로젝트는 **각자 별도 프로젝트와 키** 사용

💼 실제 업무용 서비스: **Supabase Auth + 사용자별 RLS 정책** 구성이 더 적절

7교시 · Streamlit Secrets 이해하기

비밀 키는 코드가 아닌 Streamlit Secrets에 저장하기

GitHub에는 **프로그램 코드만**, 비밀 키는 **Streamlit Secrets**에 — 연결 정보를 코드와 분리합니다

1 왜 API 키를 코드에 넣으면 안 될까요?

❌ **이렇게 작성하면 위험**

```
SUPABASE_KEY = "실제_비밀키"
```

GitHub 업로드 → 비밀 키 노출 → DB 높은 권한 접근

특히 **Secret key**가 유출되면 데이터베이스에 **높은 권한으로 접근**할 수 있어 매우 위험합니다

✅ **이렇게 분리해서 관리**

GitHub

프로그램 코드만 저장

Streamlit Secrets

비밀 키 · 연결 정보 저장

2 Streamlit Secrets 설정하기

Community Cloud → 배포한 앱 열기 → 설정 → Secrets

Secrets 입력 형식

```
SUPABASE_URL = "https://your-project.supabase.co"
SUPABASE_KEY = "YOUR_SUPABASE_SECRET_KEY"
APP_PASSWORD = "YOUR_CLASS_APP_PASSWORD"
```

📝 위 값은 모두 **자리표시자** · 실제 URL과 키를 **공개된 문서에 적지 않도록** 안내

🔑 **APP_PASSWORD** = Streamlit 앱 자체에 접근 제한을 두기 위한 별도 비밀번호 · **Supabase DB 비밀번호와 다른 값**으로 설정

⚠️ **Secret key를 학생들에게 복사해서 배포하지 않습니다** · 강사가 하나의 프로젝트를 운영한다면 강사가 키를 관리하고, 학생은 **강사가 허용한 앱을 통해서만** 데이터 입력

3 Python에서 Secrets 가져오기

```
import streamlit as st
url = st.secrets["SUPABASE_URL"]
key = st.secrets["SUPABASE_KEY"]
```

코드	의미
`st.secrets`	Streamlit의 비밀 설정값 조회
`SUPABASE_URL`	프로젝트 주소
`SUPABASE_KEY`	서버 측에서만 사용하는 API 키

🙈 **실제 연결 정보가 화면에 출력되지 않도록 주의**

4 Supabase 연결 코드

```
import streamlit as st
from supabase import create_client

url = st.secrets["SUPABASE_URL"]
key = st.secrets["SUPABASE_KEY"]
supabase = create_client(url, key)
```

`create_client()` = Supabase 프로젝트에 접근하기 위한 **Python 클라이언트 생성** (공식 라이브러리 안내 방식)

7교시 · 실습 5

Supabase 데이터 조회하기

🎯 Supabase에 저장된 생산 데이터를 Python으로 가져옵니다 · 처음에는 전체가 아니라 **10건만** 조회

1 데이터 조회 코드

```
response = (
    supabase.table("production_data")
    .select("*")
    .order("id")
    .limit(10)
    .execute()
)
```

코드	의미
`table("production_data")`	조회할 테이블
`select("*")`	모든 컬럼 선택
`order("id")`	고유번호 순서로 정렬
`limit(10)`	10건만 가져오기
`execute()`	실제 명령 실행

2 조회 결과를 DataFrame으로 변환

```
import pandas as pd
data = response.data
df = pd.DataFrame(data)

st.dataframe(df)
```

예전 📄 CSV 파일 읽기 → DataFrame

이번 🗄️ 데이터베이스에서 가져오기 → DataFrame

🗣️ "예전에는 **CSV 파일**을 읽어서, 이번에는 **데이터베이스**에서 가져와 DataFrame을 만들겠습니다."

✅ 데이터를 **가져오는 위치만** 달라졌습니다 · 이후 활용 방법은 이전과 동일

`groupby()` `sum()` `Plotly`

3 데이터 개수 조회하기

```
response = (
    supabase.table("production_data")
    .select("id", count="exact", head=True)
    .execute()
)
st.write("전체 데이터 개수:", response.count)
```

⚠️ **기본 API 응답은 일반적으로 최대 1,000행**

10,000건을 분석하려고 **select("*")만** 쓰면 전체를 가져오지 못할 수 있음 → **range()**로 여러 번 나누어 조회

~~select("*") → 1,000행~~ `range(0, 999) · range(1000, 1999) …`

60분 수업에서 작업별 조회 방법

작업	조회 방법
DB 연결 확인	10건 조회
저장된 데이터 개수	COUNT
최근 생산 기록	최근 10건 조회
전체 데이터 분석	페이지 단위 조회
대시보드 전체 지표	전체 집계 쿼리 또는 페이지 단위 조회

Thank you.

스마트제조_1일차
ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.